

SZAKDOLGOZAT

Szoftverfejlesztő és Tesztelő

Bábi Levente

Nyikonyuk Gergő

Szlonkai Marcell

2025

VÁCI SZAKKÉPZÉSI CENTRUM

BORONKAY GYÖRGY

MŰSZAKI TECHNIKUM ÉS

GIMNÁZIUM



VIZSGAREMEK

CareCompass

Konzulens: Gyombolainé Cserny Zsuzsanna
Wiezl Csaba

Készítette: Bábi Levente
Nyikonyuk Gergő
Szlonkai Marcell

1 Hallgatói nyilatkozat

Alulírottak, ezúton kijelentjük, hogy a szakdolgozat saját, önálló munkánk, és korábban még sehol nem került publikálásra. Szakdolgozatunk a Váci Szakképzési Centrum Boronkay György Műszaki Technikum és Gimnázium Szoftverfejlesztő és tesztelő technikus képzésén készítettük. Tudomásul vesszük, hogy szakdolgozatunkat a Váci Szakképzési Centrum Boronkay György Műszaki Technikum és Gimnázium tárolja.

Bábi Levente

Nyikonyuk Gergő

Szlonkai Marcell

2 Konzultációs lap

Vizsgálók neve: Bábi Levente, Nyikonyuk Gergő, Szlonkai Marcell

Szaktervező neve: CareCompass

Program által nyújtott szolgáltatások:

- Egészségügyi magánszolgáltatásokhoz időpontfoglalás
- Egészségügyi magánszolgáltatások hozzáadása, törlése, módosítása
- Orvosok hozzáadása, törlése, adataik módosítása

Sorszám	A konzultáció időpontja	A konzulens aláírása
1.	2024. október 11.	
2.	2024. november 15.	
3.	2024. december 13.	
4.	2025. január 17.	
5.	2025. február 14.	
6.	2025. március 14.	

A szaktervező beadható:

A szaktervezőt átvettém:

Vác, 2025.

Vác, 2025.

.....

Konzulens

.....

A szakképzést folytató

intézmény felelőse

Tartalomjegyzék

1	Hallgatói nyilatkozat.....	3
2	Konzultációs lap.....	4
3	Bevezetés	9
3.1	Témaválasztás	9
3.2	Feladatspecifikáció.....	9
3.3	Tervek.....	10
3.3.1	Aktorok és feladataik	10
3.3.2	Munkaterv	11
3.3.3	Munkamegosztás, feladatkörök	11
4	Fejlesztési dokumentáció	13
4.1	Fejlesztői környezet	13
4.1.1	Visual Studio 2022	13
4.1.2	Visual Studio Code	13
4.1.3	Android Studio	13
4.1.4	phpmyadmin	14
4.2	Felhasznált programozási nyelvek és technológiák	14
4.2.1	HTML	14
4.2.2	PHP.....	14
4.2.3	CSS	15
4.2.4	JavaScript.....	15
4.2.5	C#	15
4.2.6	Kotlin Multiplatform	16
4.2.7	MySQL	16
4.2.8	JSON	16
4.2.9	jQuery.....	17
4.2.10	Postman.....	17
4.3	Adatbázis dokumentáció.....	18

4.3.1	Adatszerkezet.....	18
4.3.1.1	Adattáblák.....	18
4.3.1.2	Normálformák.....	18
4.3.1.3	Táblák közötti kapcsolatok.....	20
4.4	Jogosultságok, jogosultsági szintek.....	42
4.5	Mobil Applikáció fejlesztői dokumentáció.....	44
4.5.1	Bevezetés.....	44
4.5.2	Fejlesztési környezet és eszközök.....	44
4.5.3	Rendszerarchitektúra.....	45
4.5.4	Platform specifikus fejlesztés.....	50
4.5.5	Függőségek és harmadik féltől származó könyvtárak.....	50
4.5.6	Tesztelés.....	51
4.5.7	Telepítés és verziókezelés.....	51
4.6	Asztali alkalmazás fejlesztői dokumentáció.....	52
4.6.1	Bevezetés.....	52
4.6.2	Rendszerarchitectúra.....	53
4.6.3	Rendszer indulása.....	53
4.6.4	Adatmodell és API Integráció.....	54
4.6.4.1	API Kommunikáció és adatok kezelése.....	55
4.6.5	Témakezelés.....	59
4.6.5.1	Alapvető metódusok.....	59
4.6.6	Felület váltás.....	61
4.6.7	Hibakezelés, input ellenőrzés.....	63
4.6.7.1	Hibakezelés.....	63
4.6.7.2	Input ellenőrzés.....	63
4.6.8	Tesztelési terv.....	64
4.7	Weboldal fejlesztői dokumentáció.....	64
4.7.1	Bevezetés.....	64
4.7.2	Rendszer áttekintése.....	64

4.7.3	Architektúra	64
4.7.4	Frontend és backend kapcsolata	65
4.7.5	Útvonalkezelés az oldalon	69
4.7.6	Weboldal konfigurálása	71
4.7.7	Hibakezelés	72
4.7.8	API hozzáférés a weboldal számára	73
4.8	API dokumentáció	75
4.8.1	Bevezetés	75
4.8.2	api.php és .htaccess működése	75
4.8.3	API logikai vezérlők	78
4.8.4	API token bemutatása.....	79
4.8.5	Email küldés API-val	83
4.8.6	Adatvalidálás	85
4.8.7	API tesztelése	85
4.8.8	DELETE kérések tesztelése	91
5	Felhasználói dokumentációk	93
5.1	Telepítési útmutató.....	93
5.2	Weboldal felhasználói dokumentáció.....	94
5.2.1	Bevezetés	94
5.2.2	Kezdeti lépések	96
5.2.3	Alapvető funkciók	98
5.2.4	Haladó funkciók	100
5.3	Asztali alkalmazás felhasználói dokumentáció	103
5.3.1	Bevezetés	103
5.3.2	Kezdeti lépések	103
5.3.2.1	Rendszerkövetelmények.....	103
5.3.3	Haladó funkciók.....	104
5.3.3.1	Bejelentkezés	104
5.3.3.2	Orvos felhasználói felület.....	105
5.3.3.3	Kórház moderátor felhasználó műveletek.....	109

5.3.3.4	Cég moderátor felhasználó műveletek	113
5.3.3.5	Admin felhasználói műveletek	119
6	Konklúzió	128
7	Továbbfejlesztés	129
7.1	Értékelések	129
7.2	Blog	129
7.3	Orvos adatlapjának kibővítése	129
7.4	További statisztikák	130
7.5	Internacionális fejlesztések	130
7.6	Magasabb fokú személyre szabhatóság	130
7.7	Mobilalkalmazás	130
7.7.1	Időpontok megtekintése	130
7.7.2	Időpontfoglalás	131
8	Irodalomjegyzék	132
8.1	Applikáció (winform)	132
8.2	Statisztika	132
8.3	Theme changer (Alap): ChatGPT	132
8.4	Web applikáció	132
8.5	További linkek:	133

3 Bevezetés

3.1 Témaválasztás

A CareCompass ötlete abból a felismerésből született, hogy a magánegészségügyi szolgáltatások keresése és kiválasztása sok ember számára kihívást jelent. A célunk az volt, hogy egy olyan innovatív és felhasználóbarát megoldást hozzunk létre, amely segít eligazodni a magánegészségügyben elérhető lehetőségek között, és megkönnyíti a páciensek döntéshozatalát.

A témaválasztásunk egyik fő motivációja az volt, hogy olyan problémára adjunk választ, amely valós és széles körben érinti az embereket. Az egészség az egyik legfontosabb érték az életünkben, és szerettünk volna hozzájárulni ahhoz, hogy a megfelelő ellátáshoz való hozzáférés egyszerűbb és gyorsabb legyen.

Emellett a CareCompass számunkra nem csupán egy technológiai fejlesztés, hanem egy küldetés is. Az volt a célunk, hogy egy olyan szolgáltatást hozzunk létre, amely nemcsak informatív, hanem valódi segítséget nyújt mindazoknak, akik hatékonyan szeretnék intézni egészségügyi ügyeiket.

Hisszük, hogy a CareCompass egy olyan eszköz, amely átláthatóbbá és könnyebben elérhetővé teszi a magánegészségügyi szolgáltatásokat, és ezzel hozzájárul a felhasználók jobb egészségügyi élményéhez.

3.2 Feladatspecifikáció

A CareCompass három fő pillérre épül, amelyek különböző felhasználói csoportokat szolgálnak ki.

- Webalkalmazás: Elsősorban a szolgáltatást igénybe vevő felhasználók számára készült, lehetővé téve számukra az egészségügyi szolgáltatások keresését, időpontfoglalást és egyéb kapcsolódó funkciók elérését.

- **Asztali alkalmazás:** Az orvosok, kórházak és egészségügyi intézmények számára fejlesztett rendszer, amely segíti a szolgáltatások adminisztrációját, a foglalások kezelését és az adatok nyomon követését.
- **Mobilalkalmazás:** A webalkalmazás kiegészítéseként szolgál, főként tájékoztató jellegű funkciókat lát el, és gyors, kényelmes hozzáférést biztosít a felhasználók számára útközben is.

Ezek a komponensek együtt alkotnak egy átfogó rendszert, amely hatékonyan segíti mind a felhasználókat, mind az egészségügyi szolgáltatókat.

3.3 Tervek

3.3.1 Aktorok és feladataik

Webalkalmazás

- **Vendég:** Kezelések, egészségügyi cégek és azok szolgáltatásainak megtekintése (nem foglalhat időpontot csak megtekintő)
- **Felhasználó (regisztrált):** Időpontot foglalhat és előzményeit nézheti meg. (vendég + időpontfoglalás)
- **Orvos:** Rendelések időpontjainak megnézése (hozzá foglalt időpontok). (vendég + foglalás megtekintése)
- **Intézmény (kórház):** -
- **Cég:** -
- **Admin:** -

Asztali alkalmazás

- **Orvos:** Beírhatja a szabadságát és megtekintheti a naptárát és szerkesztheti és módosíthatja a lefoglalt időpontokat.
- **Intézmény (kórház):** Módosíthatja, szerkesztheti és törölheti az alá tartozó kezeléseket és azok leírását, az intézményben dolgozó orvosokat és az intézmény adatait. Új adatokat adhat hozzá az intézményhez.

- Cég: Szerkesztheti, módosíthatja és törölheti a hozzá tartozó intézményeket, valamint kezelheti azokat. A céghez és az alá tartozó kórházhoz új adatokat adhat hozzá.
- Admin: Szerkesztheti, módosíthatja és törölheti a rendszerben kezelt összes céget, intézményt, orvost, kezelést és felhasználót, új jogosultságokat hozhat létre. A rendszerhez új adatokat adhat hozzá.

3.3.2 Munkaterv

Határidők

- 2024.10.01 – Teljes adatbázis, tesztadatokkal
- 2024.10.10 – Weboldal terv
- 2024.11.01 – Asztali alkalmazás terv
- 2025.03.01 – Weboldal
- 2025.03.05 – Asztali alkalmazás
- 2025.03.23 – Dokumentáció

3.3.3 Munkamegosztás, feladatkörök

Bábi Levente

- Adatbázis megtervezése/előkészítése
- Adatbázis dokumentálása, normálformák leírása, kapcsolatok leírása
- Weblap dizájn
- Mobil alkalmazás programozása, dokumentálása

Nyikonyuk Gergő

- Weboldal frontend, backend programozása
- Weboldal dokumentációja
- Weblap dizájn
- API fejlesztése
- API dokumentálása
- Átfogó tesztelés

Szlonkai Marcell

- Asztali alkalmazás programozása
- Asztali alkalmazás dokumentációja
- Weblap dizájn
- API fejlesztése
- API tesztelése
- Átfogó tesztelés

4 Fejlesztői dokumentáció

4.1 Fejlesztési környezet

4.1.1 Visual Studio 2022

A Visual Studio egy teljes körű integrált fejlesztői környezet (IDE), amelyet a Microsoft fejlesztett ki. Lehetővé teszi a programozók számára, hogy alkalmazásokat tervezzenek, fejlesszenek, teszteljenek és hibakeresést végezzenek egyetlen platformon. Beépített fordítóval, kódszerkesztővel és hibakezelő eszközökkel rendelkezik. Elsősorban C# és C++ nyelvekhez optimalizált, de számos más programozási nyelvet is támogat. Több változatban érhető el, például Visual Studio Community, Professional és Enterprise, amelyek különböző fejlesztési igényekhez igazodnak.

4.1.2 Visual Studio Code

A Visual Studio Code (VS Code) egy ingyenes, nyílt forráskódú kódszerkesztő, amelyet a Microsoft fejlesztett. Kiváló teljesítményének és testreszabhatóságának köszönhetően az egyik legnépszerűbb fejlesztői eszköz. Alapértelmezetten támogatja a leggyakrabban használt programozási nyelveket, mint a C, C++, C#, Python, Java, JavaScript és még sok más. Bővíthető kiegészítőkkel, amelyek segítségével további nyelvek, keretrendszerek és fejlesztői eszközök integrálhatók. A felhasználói felület személyre szabható témákkal, billentyűparancsokkal és egyéb fejlesztői eszközökkel. A VS Code elérhető Windows, macOS és Linux rendszereken, így platformfüggetlen megoldást kínál a fejlesztők számára.

4.1.3 Android Studio

Az Android Studio egy hivatalos, integrált fejlesztői környezet (IDE) az Android alkalmazások fejlesztéséhez, amelyet a Google fejlesztett. Tartalmazza az összes szükséges eszközt a hatékony fejlesztéshez, beleértve a fejlett kódszerkesztőt, vizuális tervezőeszközöket, beépített emulátort és hibakeresőt. Az Android Studio támogatja a Kotlin, Java és C++ nyelveket, valamint számos beépített funkcióval segíti az alkalmazások optimalizálását és tesztelését. Lehetővé teszi az alkalmazások futtatását

virtuális vagy valós eszközökön, így a fejlesztők gyorsan ellenőrizhetik és finomhangolhatják munkájukat. Az Android Studio Windows, macOS és Linux rendszereken is elérhető.

4.1.4 phpmyadmin

A phpMyAdmin egy ingyenes, nyílt forráskódú webalapú eszköz, amelyet MySQL és MariaDB adatbázisok kezelésére használnak. Lehetővé teszi az adatbázisok létrehozását, módosítását, törlését, valamint az adatok importálását és exportálását egy könnyen kezelhető webes felületen keresztül. Emellett támogatja az SQL-utasítások futtatását, a jogosultságkezelést és a szerkezetek vizuális áttekintését. A phpMyAdmin egyik legnépszerűbb eszköz az adatbázis-adminisztrációhoz, amely széles körben elérhető különböző szerverkörnyezetekben.

4.2 Felhasznált programozási nyelvek és technológiák

4.2.1 HTML

A HTML (Hyper Text Markup Language) a szabványos jelölőnyelv, amelyet a weboldalak létrehozására és tervezésére használnak. A HTML meghatározza a weboldal struktúráját és tartalmát, lehetővé téve a különböző elemek, például szövegek, képek, linkek és táblázatok megjelenítését. A HTML-t gyakran kombinálják más webfejlesztési technológiákkal, mint a CSS (Cascading Style Sheets) és a JavaScript, hogy javítsák a weboldalak megjelenését és funkcionalitását. A CSS-t a HTML elemek formázására és stílusozására használják, míg a JavaScript a weboldalak interaktív és dinamikus viselkedésének megvalósítására szolgál. Együtt a HTML, a CSS és a JavaScript alkotják a modern webfejlesztés három alappillérét.

4.2.2 PHP

A PHP (Hypertext Preprocessor) egy népszerű, szerveroldali programozási nyelv, amely dinamikus weboldalak és webalkalmazások fejlesztésére szolgál. Lehetővé teszi az adatbázisokkal való kommunikációt, űrlapok kezelését és dinamikus tartalom

generálását. A PHP könnyen integrálható HTML-be, és a szerveren dolgozza fel a kódot, mielőtt azt a böngésző megjeleníti. A nyelv széles körben használt, például a WordPress és a Facebook is PHP-t használ, és számos keretrendszer (pl. Laravel, Symfony) segíti a gyors fejlesztést.

4.2.3 CSS

A CSS (Cascading Style Sheets) egy stíluslap nyelv, amelyet a weboldalak megjelenésének és formázásának kezelésére használnak. A CSS lehetővé teszi a HTML elemek stílusának, például a színek, betűtípusok, elrendezés és méretek meghatározását. A CSS segítségével különböző eszközökön és képernyőméreteken is rugalmasan alakíthatjuk a weboldalak kinézetét, így biztosítva a reszponzív design-t. A CSS-t gyakran alkalmazzák a HTML kódtól elkülönítve, hogy javítsák a kód tisztaságát és könnyen karbantarthatóvá tegyék a weboldalakat.

4.2.4 JavaScript

A JavaScript egy dinamikus szkriptnyelv, amelyet elsősorban webfejlesztésre használnak. Segítségével interaktív és dinamikus tartalmak hozhatók létre, amelyek reagálnak a felhasználói műveletekre. A JavaScript a frontend és a backend fejlesztésben egyaránt elterjedt, lehetővé téve böngészőoldali animációkat, űrlapkezelést, APIhívásokat és szerveroldali alkalmazásokat. Széles körben alkalmazzák modern webes technológiákban, például a Node.js és különböző keretrendszerek révén, így a webfejlesztés egyik alapvető nyelvéné vált.

4.2.5 C#

A C# (C-sharp) egy modern, objektumorientált programozási nyelv, amelyet elsősorban a .NET platformhoz fejlesztettek ki. Széles körben alkalmazzák Windows alkalmazások, webalkalmazások, játékok és szerveroldali szolgáltatások fejlesztésére. Egyszerű szintaxisa és erőteljes funkcionalitása révén népszerű választás üzleti szoftverek, mobilalkalmazások és nagy teljesítményű rendszerek készítéséhez. A C# támogatja az aszinkron programozást, a generikus típusokat és a modern fejlesztési paradigmákat, így sokoldalú és hatékony eszköz a fejlesztők számára.

4.2.6 Kotlin Multiplatform

A Kotlin Multiplatform (KMP) egy olyan fejlesztési megközelítés, amely lehetővé teszi ugyanazon kódbázis használatát több platformon, például Android, iOS, web és asztali alkalmazások számára. A KMP célja, hogy csökkentse a különböző platformokon történő fejlesztéshez szükséges erőforrásokat és időt, miközben lehetővé teszi a platform-specifikus funkciók és teljesítményjavítások integrálását. A Kotlin Multiplatform lehetővé teszi, hogy a közös üzleti logikát és adatkezelést egyetlen kódbázisból osszuk meg, míg az egyes platformokra specifikus UI-t és funkciókat külön kezelhetjük. A Kotlin természetes integrációt biztosít Android és iOS alkalmazásokkal, de más platformokhoz, mint például a backend vagy a webes alkalmazások, szintén használható. A KMP az egyik legújabb megoldás a multiplatform fejlesztésben, és jelentős előnyöket kínál a fejlesztők számára, mivel csökkenti a kód duplikációját és egyszerűsíti a karbantartást.

4.2.7 MySQL

A MySQL egy népszerű, nyílt forráskódú adatbázis-kezelő rendszer (DBMS), amelyet elsősorban a webalkalmazások és az online rendszerek adatainak tárolására használnak. Az SQL (Structured Query Language) nyelvet alkalmazza az adatok lekérdezésére, módosítására és kezelésére. A MySQL gyors, megbízható és könnyen kezelhető, ami lehetővé teszi, hogy nagy mennyiségű adatot tároljunk és dolgozzunk fel hatékonyan.

4.2.8 JSON

A JSON (JavaScript Object Notation) egy könnyen olvasható és gépi feldolgozásra alkalmas adatsere-formátum. Szöveges formátumban tárolja az adatokat, és lehetővé teszi olyan adatstruktúrák, mint az objektumok és tömbök reprezentálását. Egyszerű szintaxisa és széles körű támogatottsága miatt gyakran használják webes alkalmazásokban és API-kommunikációban.

4.2.9 jQuery

A jQuery egy könnyű, gyors és platformfüggetlen JavaScript könyvtár, amelyet a webfejlesztés egyszerűsítésére hoztak létre. Segítségével könnyebben kezelhetők a DOM-elemek, eseménykezelések, animációk és AJAX-hívások. Népszerűsége annak köszönhető, hogy leegyszerűsíti a JavaScript kódot, és kompatibilis a legtöbb böngészővel, így gyorsabb és hatékonyabb fejlesztést tesz lehetővé.

4.2.10 Postman

A Postman egy népszerű API-fejlesztési és tesztelési eszköz, amely megkönnyíti a RESTful és más típusú API-k kezelését. Segítségével fejlesztők könnyedén tesztelhetik, dokumentálhatják és automatizálhatják az API-kat, mindezt felhasználóbarát grafikus felületen keresztül. A Postman támogatja a HTTP-kérések (GET, POST, PUT, DELETE stb.) könnyű küldését, válaszok megjelenítését és a kérések testreszabását, így segítve a gyorsabb és hatékonyabb fejlesztést. Emellett lehetőséget biztosít a környezetek és változók kezelésére, a tesztek automatizálására és a kérések dokumentálására, amely még produktívabbá teszi a fejlesztési folyamatokat.

4.3 Adatbázis dokumentáció

4.3.1 Adatszerkezet

MySQL kliens verziója: -

Adatbázis neve: CareCompass

Tárolómotor: MariaDB

Alapértelmezett illesztés: utf8mb4_general_ci

4.3.1.1 Adattáblák

- api_endpoints
- company
- company_phonenumber
- company_service
- doctors
- doctor_company
- doctor_education
- doctor_hospital
- doctor_language
- doctor_phonenumber
- doctor_service
- educations
- endpoints
- genders
- holidays
- hospitals
- hospital_phonenumber
- hospital_service
- images
- languages
- link_tokens
- phonenumber
- reservations
- roles
- services
- tokens
- users
- user_company
- user_data
- user_hospital
- user_logs
- worktimes
- zip_code

4.3.1.2 Normálformák

Nulladik normálforma:

id, active, address, approved, birthdate, company_register_number, datetime, deleted, description, director_name, duration, education, email, end_date, event, gender, hasTB, introduction, name, newsletter, note, open_hours, password, phone, price, rating, start_date, tags, TAJ, tax_number, timestamp, type, verified

Első normálforma:

Users (id, name, public, deleted, company_register_number, tax_number, director_name, email, address, active, slug, birthdate, birthdate_visible, introduction, doctor_id, note, approved, open_hours, description, alt, title, owner_id, position)

Több (language, phone_number, type, price, timestamp, action, price, duration,)

Második normálforma:

api_endpoints (id, name, public, deleted)

company (id, name, company_register_number, tax_number, director_name, email, address, active, deleted)

doctors (user_id, slug, birthdate, birthdate_visible, introduction)

educations (id, name)

endpoints (id, endpoint, parent_id, position, name, controller, dynamic_source, menu_item, active, deleted)

genders (id, gender_name)

Harmadik normálforma:

api_endpoints(kész)

company(kész) doctors(kész)

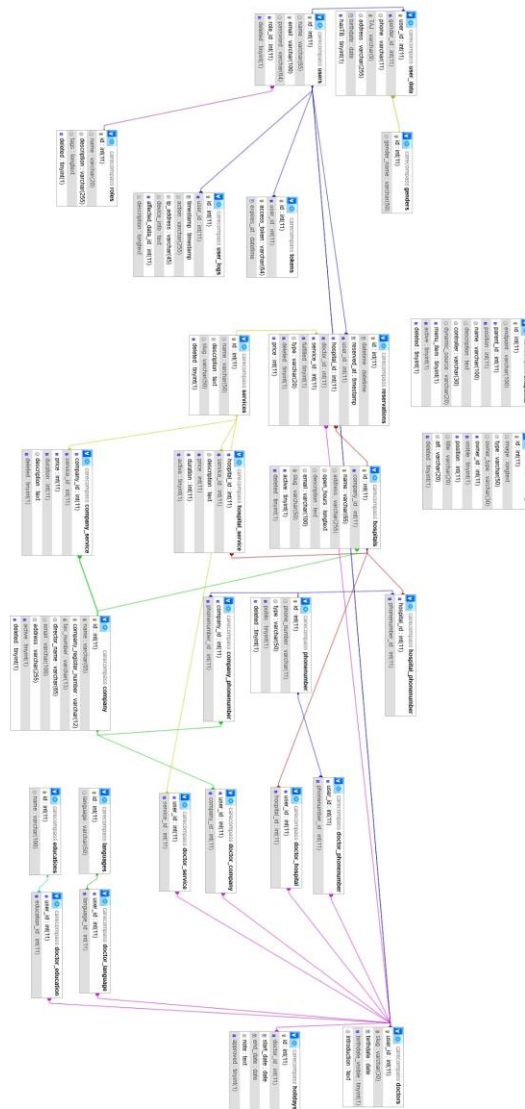
educations(kész)

endpoints(kész)

genders(kész)

hospitals (id, company_id, name, address, open_hours, description, email, slug, active, deleted)

4.3.1.3 Táblák közötti kapcsolatok



Az „api_endpoints” tábla

Ez a tábla tartalmazza az api végpontok adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id	 int(11)			Nem	Nincs		AUTO_INCREMENT
2	name	 varchar(200)	utf8mb4_general_ci		Nem	Nincs		
3	public	tinyint(1)			Nem	0		
4	deleted	tinyint(1)			Nem	0		

- id: az api végpont egyedi azonosítója, a tábla elsődleges kulcsa
- name: az api végpont elérési útvonala, unique
- public: publikus-e az adott api végpont

- deleted: törölt-e az adott végpont

Tábla létrehozása:

```
CREATE TABLE 'api_endpoints' ( 'id' int(11) NOT NULL AUTO_INCREMENT ,
'name' varchar(200) NOT NULL, 'public' tinyint(1) NOT NULL DEFAULT 0,
'deleted' tinyint(1) NOT NULL DEFAULT 0, PRIMARY KEY ('id'), UNIQUE
KEY 'name' ('name') ) ENGINE=InnoDB AUTO_INCREMENT=129 DEFAULT
CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „company” tábla

Ez a tábla tartalmazza a cégek adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	name 	varchar(65)	utf8mb4_general_ci		Nem	Nincs		
3	company_register_number 	varchar(12)	utf8mb4_general_ci		Nem	Nincs		
4	tax_number 	varchar(13)	utf8mb4_general_ci		Nem	Nincs		
5	director_name	varchar(65)	utf8mb4_general_ci		Nem	Nincs		
6	email	varchar(100)	utf8mb4_general_ci		Nem	Nincs		
7	address	varchar(255)	utf8mb4_general_ci		Nem	Nincs		
8	active	tinyint(1)			Nem	1		
9	deleted	tinyint(1)			Nem	0		

- id: a cég azonosítója, a tábla elsődleges kulcsa
- name: a cég megnevezése
- company_register_number: a cég cégjegyzékszáma
- tax_number: a cég adószáma
- director_name: a cégigazgató neve
- email: a cég kapcsolattartási email címe
- address: a cég székhelyének címe
- active: a cég aktíválva van-e
- deleted: törölt-e az adott adat

Tábla létrehozása:

```
CREATE TABLE 'company' ( 'id' int(11) NOT NULL AUTO_INCREMENT, 'name'
varchar(65) NOT NULL, 'company_register_number' varchar(12) NOT NULL,
'tax_number' varchar(13) NOT NULL, 'director_name' varchar(65) NOT
NULL, 'email' varchar(100) NOT NULL, 'address' varchar(255) NOT NULL,
'active' tinyint(1) NOT NULL DEFAULT 1, 'deleted' tinyint(1) NOT NULL
DEFAULT 0, PRIMARY KEY ('id'), UNIQUE KEY 'name' ('name'), UNIQUE KEY
'company_register_number' ('company_register_number'), UNIQUE KEY
'tax_number' ('tax_number') ) ENGINE=InnoDB AUTO_INCREMENT=5 DEFAULT
CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „doctors” tábla

Ez a tábla tartalmazza a doktorok adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 	int(11)			Nem	Nincs		
2	slug 	varchar(30)	utf8mb4_general_ci		Igen	NULL		
3	birthdate	date			Igen	NULL		
4	birthdate_visible	tinyint(1)			Nem	0		
5	introduction	text	utf8mb4_general_ci		Igen	NULL		

- user_id: az orvos egyedi azonosítója, a tábla elsődleges kulcsa és másodlagos kulcsa is a users táblához
- slug: a böngészőben megjelenő cím
- birthdate: az orvos születési dátuma
- birthdate_visible: az orvos születési dátuma megjelenjen-e a bemutatkozó oldalon
- introduction: az orvos bemutatkozása

Tábla létrehozása:

```
CREATE TABLE 'doctors' ( 'user_id' int(11) NOT NULL, 'slug'
varchar(30) DEFAULT NULL, 'birthdate' date DEFAULT NULL,
'birthdate_visible' tinyint(1) NOT NULL DEFAULT 0,
'introduction' text DEFAULT NULL, PRIMARY KEY ('user_id'),
UNIQUE KEY 'slug' ('slug'), CONSTRAINT 'doctors_ibfk_1' FOREIGN
KEY ('user_id') REFERENCES 'users' ('id'), CONSTRAINT
'doctors_ibfk_2' FOREIGN KEY ('user_id') REFERENCES 'users' ('id') )
ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Az „educations” tábla

Ez a tábla tartalmazza az orvosok végzettségeit.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id	int(11)			Nem	Nincs		AUTO_INCREMENT
2	name	varchar(100)	utf8mb4_general_ci		Nem	Nincs		

- id: a tanulmány azonosítója, a tábla elsődleges kulcsa
- name: a tanulmány megnevezése

Tábla létrehozása:

```
CREATE TABLE 'educations' ( 'id' int(11) NOT NULL AUTO_INCREMENT, 'name'
varchar(100) NOT NULL, PRIMARY KEY ('id')) ENGINE=InnoDB AUTO_INCREMENT=5
DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Az „endpoints” tábla

Ez a tábla tartalmazza a végpontokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id	int(11)			Nem	Nincs		AUTO_INCREMENT
2	endpoint	varchar(100)	utf8mb4_general_ci		Nem	Nincs		
3	parent_id	int(11)			Igen	0		
4	position	int(11)			Nem	Nincs		
5	name	varchar(100)	utf8mb4_general_ci		Nem	Nincs		
6	description	text	utf8mb4_general_ci		Nem	Nincs		
7	controller	varchar(30)	utf8mb4_general_ci		Nem	Nincs		
8	dynamic_source	varchar(20)	utf8mb4_general_ci		Igen	NULL		
9	menu_item	tinyint(1)			Nem	1		
10	active	tinyint(1)			Nem	1		
11	deleted	tinyint(1)			Nem	0		

- id: a végpont azonosítója, a tábla elsődleges kulcsa
- endpoint: a végpont elérése
- parent_id: a végpont szülője
- position: a végpont elhelyezkedése a felsorolásban
- name: a végpont megnevezése
- controller: a végpont vezérlője
- dynamic_source: dinamikus forrás
- menu_item: a végpont a menüben megjelenik-e
- active: a végpont aktív-e
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'endpoints'( 'id' int(11) NOT NULL AUTO_INCREMENT,
'endpoint' varchar(100) NOT NULL, 'parent_id' int(11) DEFAULT 0,
'position' int(11) NOT NULL, 'name' varchar(100) NOT NULL,
'description' text NOT NULL, 'controller' varchar(30) NOT NULL,
'dynamic_source' varchar(20) DEFAULT NULL, 'menu_item' tinyint(1) NOT
NULL DEFAULT 1, 'active' tinyint(1) NOT NULL DEFAULT 1, 'deleted'
tinyint(1) NOT NULL DEFAULT 0, PRIMARY KEY ('id') ) ENGINE=InnoDB
AUTO_INCREMENT=18 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „genders” tábla

Ez a tábla tartalmazza a nemeket.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	gender_name	varchar(50)	utf8mb4_general_ci		Nem	Nincs		

- id: a nem egyedi azonosítója, a tábla elsődleges kulcsa
- gender_name: a nem neve

Tábla létrehozása:

```
CREATE TABLE 'genders' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'gender_name' varchar(50) NOT NULL, PRIMARY KEY ('id') )
ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

A „holidays” tábla

Ez a tábla tartalmazza a szabadságok adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	doctor_id 	int(11)			Nem	Nincs		
3	start_date	date			Nem	Nincs		
4	end_date	date			Nem	Nincs		
5	note	text	utf8mb4_general_ci		Nem	Nincs		
6	approved	tinyint(1)			Nem	0		

- id: a szabadság azonosítója, a tábla elsődleges kulcsa
- doctor_id: a szabadságon lévő orvos azonosítója, idegen kulcs
- start_date: a szabadság kezdeti napja
- end_date: a szabadság záró napja
- note: a szabadsághoz tartozó megjegyzés
- approved: a szabadságot jóváhagyták-e

Tábla létrehozása:

```
CREATE TABLE `holidays` ( `id` int(11) NOT NULL AUTO_INCREMENT,
`doctor_id` int(11) NOT NULL, `start_date` date NOT NULL,
`end_date` date NOT NULL, `note` text NOT NULL, `approved`
tinyint(1) NOT NULL DEFAULT 0, PRIMARY KEY (`id`), KEY
`doctor_id` (`doctor_id`), CONSTRAINT `holidays_ibfk_1` FOREIGN
KEY (`doctor_id`) REFERENCES `doctors` (`user_id`)) ENGINE
=InnoDB DEFAULT CHARSET=utf8mb COLLATE=utf8mb4_general_ci;
```

A „hospitals” tábla

Ez a tábla tartalmazza a kórházak adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 🔑	int(11)			Nem	Nincs		AUTO_INCREMENT
2	company_id 🔑	int(11)			Nem	Nincs		
3	name 🔑	varchar(65)	utf8mb4_general_ci		Nem	Nincs		
4	address	varchar(255)	utf8mb4_general_ci		Nem	Nincs		
5	open_hours	longtext	utf8mb4_bin		Igen	NULL		
6	description	text	utf8mb4_general_ci		Igen	NULL		
7	email	varchar(100)	utf8mb4_general_ci		Nem	Nincs		
8	slug 🔑	varchar(50)	utf8mb4_general_ci		Nem	Nincs		
9	active	tinyint(1)			Nem	1		
10	deleted	tinyint(1)			Nem	0		

- id: a kórház azonosítója, a tábla elsődleges kulcsa
- company_id: a kórházat üzemeltető cég azonosítója, idegen kulcs
- name: a kórház megnevezése, unique
- address: a kórház címe
- open_hours: a kórház nyitvatartási ideje
- description: a kórház rövid bemutatkozása
- email: a kórház kapcsolattartási email címe
- slug: a böngészőben megjelenő cím
- active: a kórház aktív állapotban van-e
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'hospitals' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'company_id' int(11) NOT NULL, 'name' varchar(65) NOT NULL, 'address'
varchar(255) NOT NULL, 'open_hours' longtext CHARACTER
SET utf8mb4 COLLATE utf8mb4_bin DEFAULT NULL CHECK
(json_valid('open_hours')), 'description' text DEFAULT NULL,
'email' varchar(100) NOT NULL, 'slug' varchar(50) NOT NULL,
'active' tinyint(1) NOT NULL DEFAULT 1, 'deleted' tinyint(1) NOT NULL
DEFAULT 0, PRIMARY KEY ('id'), UNIQUE KEY 'name' ('name'), UNIQUE KEY
'slug' ('slug'), KEY 'company_id' ('company_id'), CONSTRAINT
'hospitals_ibfk_1' FOREIGN KEY ('company_id') REFERENCES 'company'
('id') ) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

Az „images” tábla

Ez a tábla tartalmazza a képeket.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	image	longtext	utf8mb4_general_ci		Nem	Nincs		
3	type	varchar(50)	utf8mb4_general_ci		Nem	Nincs		
4	owner_type	varchar(30)	utf8mb4_general_ci		Nem	Nincs		
5	owner_id	int(11)			Nem	Nincs		
6	visible	tinyint(1)			Nem	1		
7	position	int(11)			Igen	NULL		
8	title	varchar(20)	utf8mb4_general_ci		Nem	Nincs		
9	alt	varchar(20)	utf8mb4_general_ci		Nem	Nincs		
10	deleted	tinyint(1)			Nem	0		

- id: a kép azonosítója, a tábla elsődleges kulcsa
- image: a kép base64 formátumban eltárolva
- type: a kép típusa
- owner_type: a kép tulajának típusa
- owner_id: a kép tulajának azonosítója
- visible: a kép láthatósága
- position: a kép sorrendben való helyének száma
- title: a kép címe
- alt: a képet helyettesítő szöveg
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'images' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'image' longtext NOT NULL, 'type' varchar(50) NOT NULL,
'owner_type' varchar(30) NOT NULL, 'owner_id' int(11) NOT NULL,
'visible' tinyint(1) NOT NULL DEFAULT 1, 'position' int(11)
DEFAULT NULL, 'title' varchar(20) NOT NULL, 'alt' varchar(20)
NOT NULL, 'deleted' tinyint(1) NOT NULL DEFAULT 0, PRIMARY KEY
('id') ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE = utf8mb4
_general_ci;
```

A „languages” tábla

Ez a tábla tartalmazza a nyelvek adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	language	varchar(50)	utf8mb4_general_ci		Nem	Nincs		

- id: a nyelv azonosítója, a tábla elsődleges kulcsa
- language: a nyelv megnevezése

Tábla létrehozása:

```
CREATE TABLE 'languages' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'language' varchar(50) NOT NULL, PRIMARY KEY ('id') )
ENGINE=InnoDB AUTO_INCREMENT=10 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

A „link_tokens” tábla

Ez a tábla tartalmazza a link tokenek adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	token	varchar(64)	utf8mb4_general_ci		Nem	Nincs		
3	secret_key	varchar(150)	utf8mb4_general_ci		Nem	Nincs		
4	used	tinyint(1)			Nem	0		
5	created_at	timestamp			Nem	current_timestamp()		
6	deleted	tinyint(1)			Nem	0		

- id: a link token azonosítója, a tábla elsődleges kulcsa
- name: maga a link token
- secret_key: a tokenhez tartozó titkos kulcs
- used: felhasznált-e az adott token
- created_at: mikor lett létrehozva az adott token
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'link_tokens' ( 'id' int(11) NOT NULL
AUTO_INCREMENT, 'token' varchar(64) NOT NULL, 'secret_key'
varchar(150) NOT NULL, 'used' tinyint(1) NOT NULL DEFAULT 0,
'created_at' timestamp NOT NULL DEFAULT current_timestamp(),
'deleted' tinyint(1) NOT NULL DEFAULT 0, PRIMARY KEY ('id') )
ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE =utf8mb4 _
general_ci;
```

A „phonenummer” tábla

Ez a tábla tartalmazza a telefonszámokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	phone_number	varchar(11)	utf8mb4_general_ci		Nem	Nincs		
3	type	varchar(50)	utf8mb4_general_ci		Nem	Nincs		
4	public	tinyint(1)			Nem	1		
5	deleted	tinyint(1)			Nem	0		

- id: a telefonszám azonosítója, a tábla elsődleges kulcsa
- phone_number: a telefonszám
- type: a telefonszám típusa
- public: publikus-e a telefonszám
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'phonenummer' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'phone_number' varchar(11) NOT NULL, 'type' varchar(50) NOT NULL, 'public'
tinyint(1) NOT NULL DEFAULT 1, 'deleted' tinyint(1) NOT NULL DEFAULT 0,
PRIMARY KEY ('id') )
ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „reservations” tábla

Ez a tábla tartalmazza a foglalások adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	datetime	datetime			Nem	Nincs		
3	reserved_at	timestamp			Nem	current_timestamp()		
4	user_id 	int(11)			Nem	Nincs		
5	hospital_id 	int(11)			Nem	Nincs		
6	doctor_id 	int(11)			Igen	NULL		
7	service_id 	int(11)			Nem	Nincs		
8	fulfilled	tinyint(1)			Nem	0		
9	type	varchar(20)	utf8mb4_general_ci		Nem	Nincs		
10	deleted	tinyint(1)			Nem	0		
11	price	int(11)			Nem	Nincs		

- id: a foglalás azonosítója, a tábla elsődleges kulcsa
- datetime: a foglalás időpontja
- reserved_at: a foglalás elküldésének időpontja
- user_id: a foglalást létrehozó felhasználó, idegen kulcs
- hospital_id: a foglalás kórházhoz rendelése, idegen kulcs
- doctor_id: a foglalás orvoshoz rendelése, idegen kulcs
- service_id: a foglalásban igénybe vett szolgáltatás azonosítója, idegen kulcs
- fulfilled: a foglalás megerősítésre került-e
- type: a foglalást milyen típusú felhasználó végezte el
- deleted: törölt-e az adat
- price: a foglalás végösszege

Tábla létrehozása:

```
CREATE TABLE 'reservations' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'datetime' datetime NOT NULL, 'reserved_at' timestamp NOT NULL
DEFAULT current_timestamp(), 'user_id' int(11) NOT NULL,
'hospital_id' int(11) NOT NULL, 'doctor_id' int(11) DEFAULT NULL,
'service_id' int(11) NOT NULL, 'fulfilled' tinyint(1) NOT NULL
DEFAULT 0, 'type' varchar(20) NOT NULL, 'deleted' tinyint(1) NOT NULL
DEFAULT 0, 'price' int(11) NOT
NULL, PRIMARY KEY ('id'), KEY 'hospital_id' ('hospital_id'), KEY
'service_id' ('service_id'), KEY 'user_id' ('user_id'), KEY
'doctor_id' ('doctor_id'), CONSTRAINT 'reservations_ibfk_1' FOREIGN
KEY ('hospital_id') REFERENCES 'hospitals' ('id'),
CONSTRAINT 'reservations_ibfk_2' FOREIGN KEY ('service_id')
REFERENCES 'services' ('id'), CONSTRAINT 'reservations_ibfk_3'
FOREIGN KEY ('user_id') REFERENCES 'users' ('id'), CONSTRAINT
'reservations_ibfk_4' FOREIGN KEY ('doctor_id') REFERENCES 'doctors'
('user_id') ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

A „roles” tábla

Ez a tábla tartalmazza a jogosultságokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	name	varchar(20)	utf8mb4_general_ci		Nem	Nincs		
3	description	varchar(255)	utf8mb4_general_ci		Nem	Nincs		
4	tags	longtext	utf8mb4_bin		Nem	Nincs		
5	deleted	tinyint(1)			Nem	0		

- id: a jogosultság azonosítója, a tábla elsődleges kulcsa
- name: a jogosultság megnevezése
- description: a jogosultság leírása
- tags: a jogosultsághoz tartozó végpontjogosultság tárolására szolgál
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'roles' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'name' varchar(20) NOT NULL, 'description' varchar(255) NOT
NULL, 'tags' longtext CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT
NULL, 'deleted' tinyint(1) NOT NULL DEFAULT 0, PRIMARY KEY
('id') ) ENGINE=InnoDB AUTO_INCREMENT=7 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

A „services tábla”

Ez a tábla tartalmazza a szolgáltatásokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	name	varchar(50)	utf8mb4_general_ci		Nem	Nincs		
3	description	text	utf8mb4_general_ci		Nem	Nincs		
4	slug	varchar(50)	utf8mb4_general_ci		Nem	Nincs		
5	deleted	tinyint(1)			Nem	0		

- id: a szolgáltatás azonosítója, a tábla elsődleges kulcsa
- name: a szolgáltatás megnevezése
- description: a szolgáltatás általános leírása
- slug: a böngészőben megjelenő cím
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'services' ( 'id' int(11) NOT NULL AUTO_INCREMENT,  
'name' varchar(50) NOT NULL, 'description' text NOT NULL, 'slug'  
varchar(50) NOT NULL, 'deleted' tinyint(1) NOT NULL DEFAULT 0,  
PRIMARY KEY ('id') ) ENGINE=InnoDB AUTO_INCREMENT=14 DEFAULT  
CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „tokens” tábla

Ez a tábla tartalmazza a tokenek adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	user_id 	int(11)			Igen	NULL		
3	access_token 	varchar(64)	utf8mb4_general_ci		Nem	Nincs		
4	expires_at	datetime			Nem	Nincs		

- id: a token azonosítója, a tábla elsődleges kulcsa
- user_id: a felhasználó azonosítója, idegen kulcs
- access_token: az apihoz tartozó token
- expires_at: idő, amikor lejár az adott token

Tábla létrehozása:

```
CREATE TABLE 'tokens' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'user_id' int(11) DEFAULT NULL, 'access_token' varchar(64) NOT
NULL, 'expires_at' datetime NOT NULL, PRIMARY KEY ('id'), UNIQUE
KEY 'access_token' ('access_token'), KEY 'user_id' ('user_id'),
CONSTRAINT 'tokens_ibfk_1' FOREIGN KEY ('user_id') REFERENCES
'users' ('id') ) ENGINE=InnoDB AUTO_INCREMENT=21 DEFAULT
CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „users” tábla

Ez a tábla tartalmazza a felhasználókat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	name	varchar(65)	utf8mb4_general_ci		Igen	NULL		
3	email 	varchar(100)	utf8mb4_general_ci		Nem	Nincs		
4	password	varchar(64)	utf8mb4_general_ci		Igen	NULL		
5	role_id 	int(11)			Nem	Nincs		
6	deleted	tinyint(1)			Nem	0		

- id: a felhasználó azonosítója, a tábla elsődleges kulcsa
- name: a felhasználó teljes neve
- email: a felhasználó email címe, unique
- password: a felhasználó jelszava
- role_id: kapcsolat a roles táblával, idegen kulcs
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'users' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'name' varchar(65) DEFAULT NULL, 'email' varchar(100) NOT NULL,
'password' varchar(64) DEFAULT NULL, 'role_id' int(11) NOT NULL,
'deleted' tinyint(1) NOT NULL DEFAULT 0, PRIMARY KEY ('id'),
UNIQUE KEY 'email' ('email'), KEY 'role_id' ('role_id'),
CONSTRAINT 'users_ibfk_1' FOREIGN KEY ('role_id') REFERENCES
'roles' ('id') ) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT
CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „user_data” tábla

Ez a tábla tartalmazza a felhasználók adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 🔑	int(11)			Nem	Nincs		
2	gender_id 🔑	int(11)			Igen	NULL		
3	phone	varchar(11)	utf8mb4_general_ci		Igen	NULL		
4	TAJ 🔑	varchar(9)	utf8mb4_general_ci		Igen	NULL		
5	address	varchar(255)	utf8mb4_general_ci		Igen	NULL		
6	birthdate	date			Nem	Nincs		
7	hasTB	tinyint(1)			Igen	NULL		

- user_id: a felhasználó azonosítója, a tábla elsődleges kulcsa és idegen kulcs is
- gender_id: a felhasználó neme, idegen kulcs
- phone: a felhasználó telefonszáma
- TAJ: a felhasználó tajszáma
- address: a felhasználó címe
- birthdate: a felhasználó születési dátuma
- hasTB: rendelkezik-e a felhasználó társadalombiztosítással

Tábla létrehozása:

```
CREATE TABLE user_data ( user_id int(11) NOT NULL, gender_id int(11) DEFAULT NULL, phone varchar(11) DEFAULT NULL, TAJ varchar(9) DEFAULT NULL, address varchar(255) DEFAULT NULL, birthdate date NOT NULL, hasTB tinyint(1) DEFAULT NULL, PRIMARY KEY (user_id), KEY gender_id (gender_id), CONSTRAINT user_data_ibfk_1 FOREIGN KEY (user_id) REFERENCES users (id), CONSTRAINT user_data_ibfk_2 FOREIGN KEY (gender_id) REFERENCES genders (id) ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „user_logs” tábla

Ez a tábla tartalmazza a felhasználók eseményeit.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 🔑	int(11)			Nem	Nincs		AUTO_INCREMENT
2	user_id 🔑	int(11)			Igen	NULL		
3	timestamp	timestamp			Nem	current_timestamp()		ON UPDATE CURRENT_TIMESTAMP()
4	action	varchar(255)	utf8mb4_general_ci		Igen	NULL		
5	ip_address	varchar(45)	utf8mb4_general_ci		Igen	NULL		
6	device_info	text	utf8mb4_general_ci		Igen	NULL		
7	affected_data_id	int(11)			Igen	NULL		
8	description	longtext	utf8mb4_bin		Igen	NULL		

- id: a végpont azonosítója, a tábla elsődleges kulcsa
- user_id: a felhasználó azonosítója, idegen kulcs

- timestamp: a művelet időpontja
- action: maga a művelet
- ip_address: a műveletet elvégző eszköz ip címe
- device_info: a műveletet elvégző eszköz információi
- affected_data_id: a művelet által megváltoztatott adatok információi
- description: leírás a műveletről, JSON

Tábla létrehozása:

```
CREATE TABLE 'user_logs' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'userid' int(11) DEFAULT NULL, 'timestamp' timestamp NOT NULL
DEFAULT current_timestamp() ON UPDATE current_timestamp(), 'action'
varchar(255) DEFAULT NULL, 'ip_address' varchar(45)
DEFAULT NULL, 'device_info' text DEFAULT NULL,
'affected_data_id' int(11) DEFAULT NULL, 'description' longtext
CHARACTER SET utf8mb4 COLLATE utf8mb4_bin DEFAULT NULL CHECK
(json_valid('description')), PRIMARY KEY ('id'), KEY 'userid'
('userid'), CONSTRAINT 'user_logs_ibfk_1' FOREIGN KEY
('userid') REFERENCES 'users' ('id') ) ENGINE=InnoDB
AUTO_INCREMENT=521 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

A „worktimes” tábla

Ez a tábla tartalmazza a munkaidőket.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
2	doctor_id 	int(11)			Nem	Nincs		
3	hospital_id 	int(11)			Nem	Nincs		
4	service_id 	int(11)			Nem	Nincs		
5	date	date			Nem	Nincs		
6	from_time	time			Nem	Nincs		
7	to_time	time			Nem	Nincs		
8	deleted	tinyint(1)			Nem	0		

- id: a munkaidő azonosítója, a tábla elsődleges kulcsa
- doctor_id: az orvos azonosítója, idegen kulcs
- hospital_id: a kórház azonosítója, idegen kulcs
- service_id: a szolgáltatás azonosítója, idegen kulcs
- date: a munkaidő dátuma

- from_time: a munkaidő kezdete
- to_time: a munkaidő vége
- deleted: törölt-e az adat

Tábla létrehozása:

```
CREATE TABLE 'worktimes' ( 'id' int(11) NOT NULL AUTO_INCREMENT,
'doctor_id' int(11) NOT NULL, 'hospital_id' int(11) NOT NULL,
'service_id' int(11) NOT NULL, 'date' date NOT NULL, 'from_time' time
NOT NULL, 'to_time' time NOT NULL, 'deleted' tinyint(1) NOT
NULL DEFAULT 0, PRIMARY KEY ('id'), KEY
'fk_foreign_key_doctor_id' ('doctor_id'), KEY
'fk_foreign_key_hospital_id' ('hospital_id'), KEY 'service_id'
('service_id'), CONSTRAINT 'fk_foreign_key_doctor_id' FOREIGN
KEY ('doctor_id') REFERENCES 'doctors' ('user_id'), CONSTRAINT
'fk_foreign_key_hospital_id' FOREIGN KEY ('hospital_id')
REFERENCES 'hospitals' ('id'), CONSTRAINT 'worktimes_ibfk_1'
FOREIGN KEY ('service_id') REFERENCES 'services' ('id') )
ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „zipcodes” tábla

Ez a tábla tartalmazza az irányítószámokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	zip	varchar(4)	utf8mb4_general_ci		Nem	Nincs		
2	name	varchar(50)	utf8mb4_general_ci		Nem	Nincs		

- zip: az irányítószám
- name: az irányítószámhoz tartozó település

Tábla létrehozása:

```
CREATE TABLE 'zip_codes' ( 'zip' varchar(4) NOT NULL, 'name'
varchar(50) NOT NULL ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

A „company_phonenumber” kapcsolótábla

Ez a tábla kapcsolja össze a cégeket a telefonszámokkal.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	company_id	int(11)			Nem	Nincs		
2	phonenumber_id	int(11)			Nem	Nincs		

- company_id: a cég azonosítója, idegen kulcs

- phonenumber_id: a telefonszám azonosítója, idegen kulcs

Tábla létrehozása:

```
CREATE TABLE 'company_phonenumber' ( 'company_id' int(11) NOT
NULL, 'onenumber_id' int(11) NOT NULL, KEY 'onenumber_id'
('onenumber_id'), KEY 'company_id' ('company_id') USING BTREE,
CONSTRAINT 'company_phonenumber_ibfk_1' FOREIGN KEY
('company_id') REFERENCES 'company' ('id'), CONSTRAINT
'company_phonenumber_ibfk_2' FOREIGN KEY ('onenumber_id')
REFERENCES 'onenumber' ('id') ) ENGINE=InnoDB DEFAULT
CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „company_service” kapcsolótábla

Ez a tábla kapcsolja össze a cégeket a szolgáltatásokkal.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	company_id 	int(11)			Nem	Nincs		
2	service_id 	int(11)			Nem	Nincs		
3	price	int(11)			Nem	Nincs		
4	duration	int(11)			Nem	Nincs		
5	description	text	utf8mb4_general_ci		Nem	Nincs		
6	deleted	tinyint(1)			Igen	0		

- company_id: a cég azonosítója, idegen kulcs
- service_id: a szolgáltatás azonosítója, idegen kulcs
- price: a szolgáltatás ára
- duration: a szolgáltatás időtartama
- description: a szolgáltatás leírása
- deleted: törölt-e az adott adat

Tábla létrehozása:

```
CREATE TABLE 'company_service' ( 'company_id' int(11) NOT NULL,
'service_id' int(11) NOT NULL, 'price' int(11) NOT NULL,
'duration' int(11) NOT NULL, 'description' text NOT NULL, 'deleted'
tinyint(1) DEFAULT 0, KEY 'service_id' ('service_id'),
KEY 'company_id' ('company_id'), CONSTRAINT
'company_service_ibfk_1' FOREIGN KEY ('company_id') REFERENCES
'company' ('id'), CONSTRAINT 'company_service_ibfk_2' FOREIGN KEY
('service_id') REFERENCES 'services' ('id'), CONSTRAINT
'company_service_ibfk_3' FOREIGN KEY ('company_id') REFERENCES
'company' ('id'), CONSTRAINT 'company_service_ibfk_4' FOREIGN
KEY ('company_id') REFERENCES 'company' ('id') ) ENGINE=InnoDB
DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „doctor_company” kapcsolótábla

Ez a tábla kapcsolja össze az orvosokat a cégekkel.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 	int(11)			Nem	Nincs		
2	company_id 	int(11)			Nem	Nincs		

- user_id: az orvos felhasználó azonosítója, idegen kulcs
- company_id: a szakma azonosítója, idegen kulcs

Tábla létrehozása:

```
CREATE TABLE 'doctor_company' ( 'user_id' int(11) NOT NULL,
'company_id' int(11) NOT NULL, KEY 'company_id' ('company_id'),
KEY 'user_id' ('user_id') USING BTREE, CONSTRAINT
'doctor_company_ibfk_1' FOREIGN KEY ('user_id') REFERENCES
'doctors' ('user_id'), CONSTRAINT 'doctor_company_ibfk_2'
FOREIGN KEY ('company_id') REFERENCES 'company' ('id') )
ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „doctor_education” kapcsolótábla

Ez a tábla kapcsolja össze az orvosokat az tanulmányokkal.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 	int(11)			Nem	Nincs		
2	education_id 	int(11)			Nem	Nincs		

- doctor_id: az orvos azonosítója, idegen kulcs
- education_id: a tanulmány azonosítója, idegen kulcs

Tábla létrehozása:

```
CREATE TABLE 'doctor_education' ( 'user_id' int(11) NOT NULL,  
'education_id' int(11) NOT NULL, KEY 'education_id'  
( 'education_id' ), KEY 'user_id' ( 'user_id' ) USING BTREE,  
CONSTRAINT 'doctor_education_ibfk_1' FOREIGN KEY ( 'user_id' )  
REFERENCES 'doctors' ( 'user_id' ), CONSTRAINT  
'doctor_education_ibfk_2' FOREIGN KEY ( 'education_id' ) REFERENCES  
'educations' ( 'id' ) ) ENGINE=InnoDB DEFAULT  
CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „doctor_language” kapcsolótábla

Ez a tábla kapcsolja össze az orvosokat a nyelvekkel.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 	int(11)			Nem	Nincs		
2	language_id 	int(11)			Nem	Nincs		

- user_id: az orvos azonosítója, idegen kulcs
- language_id: a nyelv azonosítója, idegen kulcs

Tábla létrehozása:

```
CREATE TABLE 'doctor_language' ( 'user_id' int(11) NOT NULL,  
'language_id' int(11) NOT NULL, KEY 'language_id'  
( 'language_id' ), KEY 'user_id' ( 'user_id' ) USING BTREE,  
CONSTRAINT 'doctor_language_ibfk_1' FOREIGN KEY ( 'user_id' )  
REFERENCES 'doctors' ( 'user_id' ), CONSTRAINT  
'doctor_language_ibfk_2' FOREIGN KEY ( 'language_id' ) REFERENCES  
'languages' ( 'id' ) ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;
```

A „doctor_phonenumber” kapcsolótábla

Ez a tábla kapcsolja össze az orvosokat a telefonszámokkal.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 	int(11)			Nem	Nincs		
2	phonenumber_id 	int(11)			Nem	Nincs		

- user_id: az orvos azonosítója, idegen kulcs
- phonenumber_id: a telefonszám azonosítója, idegen kulcs

Tábla létrehozása:

```
CREATE TABLE 'doctor_phonenumber' ( 'user_id' int(11) NOT NULL,
'phonenumber_id' int(11) NOT NULL, KEY 'phonenumber_id'
('phonenumber_id'), KEY 'doctor_id' ('user_id'), CONSTRAINT
'doctor_phonenumber_ibfk_1' FOREIGN KEY ('user_id') REFERENCES
'doctors' ('user_id'), CONSTRAINT 'doctor_phonenumber_ibfk_2'
FOREIGN KEY ('phonenumber_id') REFERENCES 'phonenumber' ('id') )
ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „doctor_service” kapcsolótábla

Ez a tábla kapcsolja össze az orvosokat a szolgáltatásokkal.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 	int(11)			Nem	Nincs		
2	service_id 	int(11)			Nem	Nincs		
3	color	varchar(9)	utf8mb4_general_ci		Igen	NULL		

- user_id: a szolgáltatás azonosítója, idegen kulcs
- service_id: a szolgáltatás azonosítója, idegen kulcs
- color: az orvos naptárához tartozó szín

Tábla létrehozása:

```
CREATE TABLE 'doctor_service' ( 'user_id' int(11) NOT NULL,
'service_id' int(11) NOT NULL, 'color' varchar(9) DEFAULT NULL,
KEY 'service_id' ('service_id'), KEY 'user_id' ('user_id') USING
BTREE, CONSTRAINT 'doctor_service_ibfk_1' FOREIGN KEY
('user_id') REFERENCES 'doctors' ('user_id'), CONSTRAINT
'doctor_service_ibfk_2' FOREIGN KEY ('service_id') REFERENCES
'services' ('id')) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
```

A „hospital_phonenumber” kapcsolótábla

Ez a tábla kapcsolja össze a kórházakat a telefonszámokkal.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	hospital_id 	int(11)			Nem	Nincs		
2	phonenumber_id 	int(11)			Nem	Nincs		

- hospital_id: a kórház azonosítója, idegen kulcs
- phonenumber_id: a telefonszám azonosítója, idegen kulcs

Tábla létrehozása

```
CREATE TABLE 'hospital_phonenumber' ( 'hospital_id' int(11) NOT NULL, 'phonenumber_id' int(11) NOT NULL, KEY 'phonenumber_id' ('phonenumber_id'), KEY 'hospital_id' ('hospital_id') USING BTREE, CONSTRAINT 'hospital_phonenumber_ibfk_1' FOREIGN KEY ('hospital_id') REFERENCES 'hospitals' ('id'), CONSTRAINT 'hospital_phonenumber_ibfk_2' FOREIGN KEY ('phonenumber_id') REFERENCES 'phonenumber' ('id') ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „hospital_service” kapcsolótábla

Ez a tábla kapcsolja össze a kórházakat a szolgáltatásokkal.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	hospital_id	int(11)			Nem	Nincs		
2	service_id	int(11)			Nem	Nincs		
3	description	text	utf8mb4_general_ci		Nem	Nincs		
4	price	int(11)			Nem	Nincs		
5	duration	int(11)			Nem	Nincs		
6	active	tinyint(1)			Nem	1		

- hospital_id: a kórház azonosítója, idegen kulcs
- service_id: a szolgáltatás azonosítója, idegen kulcs
- description: a kórházban érvényes leírás a kezeléshez
- price: a kórházban érvényes ár
- duration: a kórházban érvényes időtartam
- active: a kórházban érvényes-e az adott szolgáltatás

Tábla létrehozása:

```
CREATE TABLE 'hospital_service' ( 'hospital_id' int(11) NOT NULL, 'service_id' int(11) NOT NULL, 'description' text NOT NULL, 'price' int(11) NOT NULL, 'duration' int(11) NOT NULL, 'active' tinyint(1) NOT NULL DEFAULT 1, KEY 'service_id' ('service_id'), KEY 'hospital_id' ('hospital_id') USING BTREE, CONSTRAINT 'hospital_service_ibfk_1' FOREIGN KEY ('hospital_id') REFERENCES 'hospitals' ('id'), CONSTRAINT 'hospital_service_ibfk_2' FOREIGN KEY ('service_id') REFERENCES 'services' ('id') ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```


A „user_company” kapcsolótábla

Ez a tábla kapcsolja össze a felhasználókat a cégekkel.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 	int(11)			Igen	NULL		
2	company_id 	int(11)			Igen	NULL		

- user_id: a felhasználó azonosítója, idegen kulcs
- company_id: a cég azonosítója, idegen kulcs

Tábla létrehozása:

```
CREATE TABLE 'user_company' ( 'user_id' int(11) DEFAULT NULL,  
'company_id' int(11) DEFAULT NULL, KEY 'user_id' ('user_id'),  
KEY 'company_id' ('company_id'), CONSTRAINT  
'user_company_ibfk_1' FOREIGN KEY ('user_id') REFERENCES 'users'  
( 'id' ), CONSTRAINT 'user_company_ibfk_2' FOREIGN KEY  
( 'company_id' ) REFERENCES 'company' ( 'id' ) ) ENGINE=InnoDB  
DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

A „user_hospital” kapcsolótábla

Ez a tábla kapcsolja össze a felhasználókat a kórházakkal.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	user_id 	int(11)			Igen	NULL		
2	hospital_id 	int(11)			Igen	NULL		

- user_id: a felhasználó azonosítója, idegen kulcs
- hospital_id: a kórház azonosítója, idegen kulcs

Tábla létrehozása:

```
CREATE TABLE 'user_hospital' ( 'user_id' int(11) DEFAULT NULL,  
'hospital_id' int(11) DEFAULT NULL, KEY 'user_id' ('user_id'),  
KEY 'hospital_id' ('hospital_id'), CONSTRAINT  
'user_hospital_ibfk_1' FOREIGN KEY ('user_id') REFERENCES  
'users' ( 'id' ), CONSTRAINT 'user_hospital_ibfk_2' FOREIGN KEY  
( 'hospital_id' ) REFERENCES 'hospitals' ( 'id' ) ) ENGINE=InnoDB  
DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

4.4 Jogosultságok, jogosultsági szintek

Öt fő jogosultsági szint van. Web felhasználó, Orvos/Doktor, Kórház moderátor, Cég moderátor és admin felhasználó. Ezek a felhasználók külön hozzáférési jogosultságuk van a különböző felületeken.

Nem regisztrált felhasználó

A még nem regisztrált felhasználó hozzáfér a weboldal legtöbb funkciójához, de szolgáltatást nem tud foglalni. Regisztráció után Web felhasználóvá válik.

Web felhasználó

A webes felhasználónak a weboldalhoz van jogosultsága, ahova bejelentkezhet és foglalhat és lemondhat szolgáltatásokat. Ezek mellett a saját adatait tudja módosítani.

Orvos/Doktor

Az orvos felhasználó kizárólag az asztali alkalmazáshoz fér hozzá, ahol az orvosi felülethez tartozó adatokat tekintheti meg. Itt lehetősége van személyes adatainak módosítására, például új jelszó beállítására, születési dátumának vagy telefonszámának frissítésére. Ezen kívül hozzáfér saját statisztikáihoz, valamint egy webes naptárfelülethez, ahol beállíthatja munkaidejét és szabadságát. A naptáron belül a már teljesített szolgáltatásokat is lezárhatja.

Kórház moderátor

A kórház moderátor kizárólag az asztali alkalmazáson keresztül fér hozzá a számára kijelölt kórház felületéhez, amelyeket megtekinthet és módosíthat. Feladatai közé tartozik az orvosok és a kórházi szolgáltatások hozzárendelése és kezelése, amelyeket a fenntartó cég által biztosított orvosok és szolgáltatások közül választhat ki. Emellett frissítheti a kórház nyitvatartási idejét, elérhetőségeit, leírását és egyéb kapcsolódó információkat. Lehetősége van továbbá a kórház és a hozzá tartozó orvosok statisztikáinak megtekintésére. A moderátor ezen felül a felületen keresztül módosíthatja saját bejelentkezési jelszavát is.

Cég moderátor

A cég moderátor kizárólag az asztali alkalmazáson keresztül fér hozzá az általa kezelt cég adataihoz, amelyeket megtekinthet, módosíthat, törölhet, valamint új adatokat adhat hozzá. Feladatai közé tartozik a céghez tartozó felhasználók regisztrálása, beleértve az orvosokat, a kórház moderátorokat és további cég moderátorokat. Emellett kórházakat rendelhet a céghez, amelyeket elérhet és szükség szerint módosíthat a kórház felületén keresztül. Ez a felület megegyezik a kórház moderátor felületével, kivéve, hogy a cég moderátor nem fér hozzá a kórház moderátor jelszávának módosításához. Ezen kívül megtekintheti a cég, valamint a hozzá tartozó kórházak és orvosok statisztikáit. A moderátor saját bejelentkezési jelszavát is módosíthatja a felületen keresztül.

Admin

Az admin felhasználó kizárólag az asztali alkalmazáson keresztül fér hozzá, ahol új felhasználókat regisztrálhat és törölhet, akik hozzáférhetnek az asztali alkalmazáshoz. A web felhasználók adatait módosíthatja (jelszó kivételével), illetve a felhasználókat törölheti. Emellett cégeket és kórházakat regisztrálhat, és hozzáférhet azok felületeihez, ahol módosításokat végezhet, mintha cég- vagy kórház felhasználó lenne. Az admin ezen kívül beléphet az orvosi felületre is, ahol a naptár és a jelszómódosításon kívül minden más funkcióhoz hozzáfér. További adatokat rögzíthet és törölhet, amelyek az alárendelt felhasználók számára elérhetők lesznek, például új szolgáltatások vagy nyelvek hozzáadása. Végül, az admin hozzáférést kap az összes statisztikai adat megtekintéséhez is.

4.5 Mobil Applikáció fejlesztői dokumentáció

4.5.1 Bevezetés

Ebben a bekezdésben alapvető információkat tudhat meg a CareCompass mobil applikációról.

- Alkalmazás neve: CareCompass
- Verziószáma: 1.0
- Támogatott operációs rendszerek: Android, iOS
- Fejlesztéshez használt platform: Kotlin Multiplatform (Android, iOS)
- Fejlesztői környezet: Android Studio
- Célközönség: Fejlesztők, üzemeltetők

Az applikáció fő funkciója az információszolgáltatás és kisebb betekintés a CareCompass működésébe

4.5.2 Fejlesztési környezet és eszközök

Ebben a bekezdésben a használt fejlesztési környezet verziójáról, illetve a program függőségeiről lesz szó.

- Használt programozási nyelvek: Kotlin
- Fejlesztői eszközök és azok verziói:
 - Android Studio Ladybug Feature Drop | 2024.2.2
 - Kotlin Multiplatform 0.8.3
 - Kotlin 1.9.20
 - Gradle 8.10.2
- Kötelezően telepítendő csomagok és függőségek:
 - Navigation-runtime-ktx 2.8.5
 - Voyager bottom sheet navigator 1.1.0-beta02
 - Voyager hilt 1.1.0-beta02
 - Voyager kodein 1.1.0-beta02
 - Voyager koin 1.1.0-beta02
 - Voyager livedata 1.1.0-beta02
 - Voyager navigator 1.1.0-beta02

- Voyager rxjava 1.1.0-beta02
- Voyager screenmodel 1.1.0-beta02
- Voyager tab-navigator 1.1.0-beta02
- Voyager transitions 1.1.0-beta02

4.5.3 Rendszerarchitektúra

Ebben a bekezdésben a főbb modulokról, komponensekről és azok működéséről lesz szó.

App function

```
@Composable
@Preview fun
App() {
    MaterialTheme {
        Navigator(HomeScreen) {
            CurrentScreen()
        }
    }
}
```

Fő function-je az alkalmazásnak. Ez indítja el az adott képernyő lejátszását.

ImageSliderWithText function

Ennek a function-nak a feladata, hogy a második oldal alján található slider képeit és annak szövegeit kezelje. Található benne egy lista változó, amiben a képek és a hozzájuk tartozó szöveg van eltárolva. Maga a tartalom egy oszlopban van tárolva a könnyű formázás miatt. Ezen belül van egy HorizontalPager, aminek feladata, hogy a felhasználó görgetése alapján változtassa a képeket.

```
@Composable val slides =
    listOf(
        SlideData(Res.drawable.babi_levente, "Kotlin developer,
        founder"),
        SlideData(Res.drawable.nyikonyuk_gergo, "FullStack developer,
        founder"),
        SlideData(Res.drawable.szlonkai_marcell, ".NET developer,
        founder")
    )
```

Ez a lista változó tartalmazza a megjelenítéshez szükséges képeket és ahhoz a szövegeket.

```
Column( modifier =
    Modifier.fillMaxSize(),
    horizontalAlignment = Alignment.CenterHorizontally
) {
    HorizontalPager( state =
        pagerState,
        modifier = Modifier.fillMaxWidth(),
        contentPadding = PaddingValues(horizontal = 64.dp),
        pageSpacing = 16.dp
    ) { page ->
        Box( modifier = Modifier.fillMaxWidth(),
            contentAlignment = Alignment.Center
        ) {

            Card( shape = RoundedCornerShape(16.dp),
                elevation = 4.dp, modifier = Modifier
                    .fillMaxWidth(0.8f)
                    .height(300.dp)
            ) {
                Image( painter
                    =
                    painterResource(slides[page].imageRes),
                    contentDescription = "Slider Image $page",
                    modifier = Modifier.fillMaxSize(), contentScale =
                    ContentScale.Crop
                )
            }
        }
    }
}
```

Ez a kódrészlet jeleníti meg a képeket, az aktuális PagerState alapján.

HomeScreen definíció

Ez a fő képernyő, amivel találkozik először a felhasználó. Itt szintén oszlopban található a tartalom, amiben található kettő szöveg és egy kép. Az oldal alján még van egy gomb, erre a kotlinban található alapvető gombot használtam és azt formáztam. A gomb fő szerepe, hogy az onclick segítségével voyager navigation definíció alapján átvigye a felhasználót a második képernyőre.

```
object HomeScreen : Screen {  
    @Composable  
    override fun Content() { val navigator =  
        LocalNavigator.current  
        Column( modifier =  
            Modifier  
                .fillMaxSize(), verticalArrangement =  
                Arrangement.Top,  
                horizontalAlignment = Alignment.CenterHorizontally  
        ) {  
            Image( painter = painterResource(Res.drawable.main_pic),  
                contentDescription = "Demo Kép", modifier = Modifier  
                    .fillMaxWidth() .padding(0.dp)  
                    .height(500.dp), contentScale =  
                    ContentScale.Crop,  
            )  
        }  
    }  
}
```

Maga a tartalom oszlopban, illetve a navigator változó mely a második oldalra való navigáció miatt fontos. Főoldalon megjelenő kép kódja.

```

Button(
    onClick = { navigator?.push(SecondScreen) }, colors =
    ButtonDefaults.buttonColors( backgroundColor = Color(red =
    31, green = 57, blue =
        92),
        contentColor = Color.White
    ),
    shape = RoundedCornerShape(13.dp), modifier =
    Modifier
        .size(width = 350.dp, height = 50.dp)
) {
    Text("Tovább!") }

```

A gomb, amely átvizsi a felhasználót a második oldalra, a kotlinban található buttont használja és a Voyagerben található navigator?.push-al működik.

SecondScreen definíció

Ezen az oldalon található fontos információk, illetve az alkotókról készült slider is. A definíció elején van egy navigator változó, ami a navigátor pillanatnyi elérését tartalmazza. Található még itt egy slides nevű lista típusú változó, ami tartalmazza a képeket és azokhoz tartozó szöveget, mely a sliderhez fontos. Van még itt egy pagerState változó, ami a slider pillanatnyi állapotát tárolja, azaz, hogy a felhasználó épp melyik alkotót nézi. A tartalom itt is egy oszlopban található, amiben itt extraként be lett állítva, hogy görgethető legyen az oldal a verticalScroll segítségével. Ebben az oszlopban egy dobozba rendezve található a „Rólunk” tartalom és a felirat mellett található vissza nyíl. A nyílhoz IconButton-t használtam, aminek az onclick eseményével vissza tudja vinni a felhasználót a főoldalra. Maga a nyíl a kotlinban megtalálható Icons.Default.ArrowBack. Az olyan elválasztó vonalak és azokon található címek dobozban vannak, viszont az alattuk lévő szöveg/leírás már egy új oszlopban van a könnyű formázhatóság miatt. Az alapítóink rész szintén dobozban található, viszont itt a text részben van egy olyan kiegészítés, hogy a slides lista text részét a pagerState.currentPage alapján kiírja. Ezután egy egyszerű Spacer után meghívja a program az ImageSliderWithText-et átadva neki a jelenlegi pagerState-t.


```

val pagerState = rememberPagerState(pageCount = { slides.size })

Column ( modifier =
    Modifier
        .fillMaxWidth()
        .verticalScroll(rememberScrollState())
        .fillMaxHeight(), verticalArrangement =
Arrangement.Top, horizontalAlignment =
Alignment.CenterHorizontally
){
    Spacer(modifier = Modifier.height(20.dp))
    Box( modifier = Modifier.fillMaxWidth()
    ){
        Text( text = "Rólunk", textAlign =
            TextAlign.Center, style =
            MaterialTheme.typography.h4, fontWeight =
            FontWeight.Bold, modifier =
            Modifier.align(Alignment.Center)
        )
        IconButton( onClick = {
            navigator?.push(HomeScreen) }, modifier =
            Modifier
                .align(Alignment.CenterStart)
                .padding(end = 16.dp)
        ){
            Icon( imageVector = Icons.Default.ArrowBack,
                contentDescription = "Vissza", tint = Color(red
                = 31, green = 57, blue = 92)
            )
        }
    }
}

```

A tartalom szintén oszlopban van, illetve a "Rólunk" felirat dobozban, ami mellett van egy nyíl, amivel visszalépni az előző oldalra. Az egész definíció elején a pagerState változó.

4.5.4 Platform specifikus fejlesztés

Közös kódbázis (`commonMain`)

A legtöbb kód ebben lett írva azon belül is az `App.kt`-ban. Tekintve, hogy az alkalmazás Kotlin Multiplatformban lett fejlesztve, így a dizájn és a UI is megosztott. A projekt elkészítéséhez a Kmp Wizard-ot használtam. Ami jelentősen megkönnyítette a projekt átláthatóságát és tervezését.

Android kódbázis (`androidMain`)

Mivel a KMP lehetőséget ad az osztott UI-ra, így nem kellett külön Android specifikus kódrészt készíteni.

iOS kódbázis (`iosMain`)

iOS-nél szintén ez a helyzet, az osztott UI miatt teljesen ugyanúgy működik az alkalmazás iOS rendszeren is mint Androidon.

4.5.5 Függőségek és harmadik féltől származó könyvtárak

Ezen függőségek és könyvtárak listája:

- `Navigation-runtime-ktx – v2.8.5`
- `Voyager Bottom Sheet Navigator – v1.1.0-beta02`
- `Voyager Hilt – v1.1.0-beta02`
- `Voyager Kodein – v1.1.0-beta02`
- `Voyager Koin – v1.1.0-beta02`
- `Voyager LiveData – v1.1.0-beta02`
- `Voyager Navigator – v1.1.0-beta02`
- `Voyager RxJava – v1.1.0-beta02`
- `Voyager ScreenModel – v1.1.0-beta02`
- `Voyager Tab Navigator – v1.1.0-beta02`
- `Voyager Transitions – v1.1.0-beta02`

Ezeket felül még további könyvtárak:

CommonMain:

- `implementation(compose.runtime)`

- `implementation(compose.foundation)`
- `implementation(compose.material)`
- `implementation(compose.ui)`
- `implementation(compose.components.resources)`
- `implementation(compose.components.uiToolingPreview)` •
`implementation(libs.androidx.lifecycle.viewmodel)`
- `implementation(libs.androidx.lifecycle.runtime.compose)`

AndroidMain:

- `implementation(compose.preview)`
- `implementation(libs.androidx.activity.compose)`

Ezeknek a könyvtáraknak, illetve függőségeknek az implementációja a `build.gradle.kts` fájlban található meg. A Voyager feladata a több oldal közti navigáció. Ennek frissítését az Android Studioból is meg lehet tenni, illetve a Voyager weboldalán is mindig megtalálható annak legfrissebb verziója. A maradék könyvtár, mint például a `compose.ui`, olyan kiegészítők, amik a formázást és a tartalmak kezelését könnyítik. Importálásra került még a `mavenCentral` és a `google repository` ami a Voyager működéséhez szükséges.

4.5.6 Tesztelés

Az applikációt egységteszteléssel teszteltem. Minden `function`-t külön teszteltem, különböző adatokkal. Mivel az alkalmazásban leginkább beégetett adatok vannak, így leginkább a képernyők közti kommunikációra kellett tesztelni. Ezen felül több külső embert is felkértem az alkalmazás tesztelésére, akiknek a fő feladata az volt, hogy a lehető legjobban szimulálják az alkalmazás működését, mint egyszerű felhasználók.

4.5.7 Telepítés és verziókezelés

Alkalmazás telepítése és publikálása

Az alkalmazás telepítéséhez lehet használni az Android Studio beépített vezeték nélküli vagy vezetékes telepítőjét, illetve szintén az Android Studio-val le is lehet buidelni az alkalmazást majd feltölteni azt akár Google Play Store-ba akár App Store-ba. Ennek módja a következő:

Google Play Store

1. Először is regisztrálni kell a Google Play Console-n, melynek egyszeri díja 25\$.
2. Ezután fel kell töltenünk az Android Studio által létrehozott AAB fájlt, hiszen a Play Store már nem támogatja az APK kiterjesztést.
3. Következőnek digitálisan alá kell írni az applikációt.
4. Majd létrehozni a Play Store-ban megjelenő oldalt.
5. Végül publikálni és megvárni, amíg azt jóváhagyja a Google.

App Store

1. Itt legelőször szükségünk lesz egy Apple fejlesztői fiókra, majd csatlakozni az Apple Developer Programhoz, ami évente 99\$.
2. Ezután szintén alá kell írni az alkalmazást és azt feltölteni.
3. Majd az App Store Connect-en keresztül elkészíteni az alkalmazás oldalát.
4. Végül publikálni és megvárni, amíg azt jóváhagyja az Apple.

Verziókezelés

Az alkalmazás több verziója a GitHubon érhető el. A GitHub-ot használtam verziókezelésre, itt megtalálható az alkalmazás legelső és jelenlegi verziója is. Napi szinten volt frissítve az alkalmazás, így az visszaállítható napra pontosan bármilyen állapotra. Amellett, hogy folyamatosan fel volt töltve GitHubra, minden egyes feltöltésnél pontosan le volt írva, hogy aznap épp milyen folyamatok haladtak, mik változtak.

4.6 Asztali alkalmazás fejlesztői dokumentáció

4.6.1 Bevezetés

Ebben a bekezdésben az asztalai alkalmazás alapvető információkat tudhat meg többet.

- Célközönség: Orvosok és üzemeltetők
- Szoftverkövetelmények
 - Windows 10 vagy újabb
 - .Net Framework 4.7.2
 - Külső könyvtárak: Newtonsoft.Json

4.6.2 Rendszerarchitectúra

Általános áttekintés:

- Felhasználói felület (UI): Windows Forms alapú, témakezeléssel
- Üzleti logika: Api-val kommunikáló osztályok, adatfeldolgozás
- Adatréteg: Külső REST API (localhost/mesterremek)

Főbb modulok és osztályok:

- Program.cs: Ez indítja a programot és hívja meg a Login.cs fület
- Bejelentkezés.cs: Bejelentkezési adatok és token adatok tárolás
- ApiClient.cs: API lekérdezések és kérések kezelése
- Main.cs: Menük meghívása a felhasználók szerepköre szerint
- ThemeManager.cs: Dinamikus témaválasztás kezelése
- Menük (4db van): Megfelelő formok meghívása
- További adatmegjelenítő formok: adatok betöltése és kezelése

Kapcsolódás az API-hoz:

- Rest API használata, egyedi Token alapú hitelesítéssel
- Minden kérés tartalmazza a token a fejlécben

4.6.3 Rendszer indulása

A Program.cs felel a program elindításáért, ez a fájl először is megnézi, hogy a program már fut-e, hogy ne induljon el újra. Erre használjuk a Mutex-et.

```
bool isRestart =  
Environment.GetCommandLineArgs().Contains("-restart");  
bool ell;  
Mutex m = new Mutex(true, "CareCompassKey", out ell);  
if (!ell && !isRestart)  
{ MessageBox.Show("Már elindult a program");  
  return;  
}
```

Ha nem fut már a program akkor meghívja a témaválasztót a ThemeManager.cs-t ami automatikusan beállítja a témát. Majd indítja a Bejelentkezés.cs bejelentkezés felületet.

```
ThemeManager.InitializeTheme();  
Application.Run(new Bejelentkezés());
```

4.6.4 Adatmodell és API Integráció

A Bejelentkezés.cs-ben található statikus osztályok tárolják a felhasználó adatait és a hitelesítési tokenet. GlobalData osztály Ez a statikus osztály tartalmazza a bejelentkezésnél kapott adatokat, amelyek a formokon az API lekérdezésekhez szükségesek.

```
public static class GlobalData {  
    public static string RoleIdentifier {get; set;}  
    public static string Username {get; set;}  
    public static string Email {get; set;}  
    public static int UserId {get; set;}  
    public static int HospitalId {get; set;}  
    public static int CompanyId {get; set;}  
    public static string Institution_name {get; set;}  
}
```

Token osztály Ez a statikus osztály tárolja a bejelentkezésnél kapott token adatokat.

```
public static class Token {  
    public static string Access_token {get; set;}  
    public static DateTime ExpiresAt {get; set;}  
    public static int timezone_type {get; set;}  
    public static string timezone {get; set;}  
}
```

Az ApiCleint.cs tartalmazza az API-val történő kommunikációt.

Token frissítése akkor történik, ha a Token.ExpiresAt-ben lévő idő lejár. A RefreshTokenAsync metódus itt új tokenet kér a felhasználó számára egy post kéréssel melynek tartalmaznia kell a felhasználó id-jét (user_id).

```

public static async Task RefreshTokenAsync()
{
    ...code
    var refreshData = new { user_id = GlobalData.UserId };
    var request = new HttpRequestMessage(HttpMethod.Post,
        $"{BaseUrl}/api/v1/api.php/auth/refreshToken")
    {
        Content = new
            StringContent(JsonConvert.SerializeObject(refreshData),
        };
    request.Headers.Authorization = new
        System.Net.Http.Headers.AuthenticationHeaderValue("Bearer",
        Token.Access_token);

    var response = await new HttpClient().SendAsync(request);
    if (response.IsSuccessStatusCode)
    { var responseData =
        JsonConvert.DeserializeObject<dynamic>(await
        response.Content.ReadAsStringAsync());
        Token.Access_token =
        responseData.data.token.access_token;
        Token.ExpiresAt =
        DateTime.Parse(responseData.data.token.expiresAt.date.
        ToString());
    } code...
}

```

4.6.4.1 API Kommunikáció és adatok kezelése

Minden formon ahol API kommunikáció történik meg kell hívni a form elején az ApiClient-et.

```
using static CareCompass.ApiClient;
```

FetchData (GET kérések) Ez az API get kéréseit kezeli és figyeli, hogy a token lejárt-e már mivel, ha lejárt akkor újat kér és újra elküli a kérést. Ezek után kezeli a további hibákat (ha vannak) sikeres kérésnél meg továbbítja az adatokat, hogy a szükséges adatok feltöltésére kerüljenek (pl: combobox-ba)

FetchData kódrészlet

```
public static async Task<dynamic> FetchData(string endpoint)
{ try { using (var client = new
    HttpClient())
    {
        // Token érvényességének ellenőrzése
        if (Token.ExpiresAt <= DateTime.Now)
            await RefreshTokenAsync();

        client.DefaultRequestHeaders.Authorization = new
        System.Net.Http.Headers.AuthenticationHeaderValue("Bearer",
        Token.Access_token);

        var response = await
        client.GetAsync($"{BaseUrl}{endpoint}");
        var result = await response.Content.ReadAsStringAsync();
        var responseData =
        JsonConvert.DeserializeObject<dynamic>(result);

        if (responseData.success == false) throw new
            Exception(responseData.error.message);

        return responseData.data;
    }
}
catch (Exception ex)
{ MessageBox.Show($"HIBA: {ex.Message}");
    return null;
}
}
```

Gyakorlatban a végpont megadása után még egy ellenőrzésen átesik majd a szükséges adatok különböző mezőkbe feltöltésére kerülnek. Legjobb példa a Doctor_Profil.cs mivel sok különböző inputmezőt tartalmaz, amely feltöltésre kerül. Ez elején meghívjuk az ApiClient.cs-ből a FetchData metódust.


```

var data = await
FetchData($" /api/v1/api.php/doctors/GetDoctor?doctor_id=
{GlobalData.UserId}"); if (data == null || data.doctor
== null)
    { Debug.WriteLine("Nincsenek adatok.");
      return;
    }

```

Ezek után a Textboxok és az egyedi CheckBox kerülnek feltöltésre

```

Lb_OrvosNev.Text = data.doctor.doctor_name;
Lb_Szuldatum.Text = data.doctor.doctor_birthdate;
dTP_Szuldatum.Value = data.doctor.doctor_birthdate;
TB_Bemutakozas.Text = data.doctor.doctor_introduction;
CB_SzuldatumLathato.Checked = data.doctor.doctor_birthdate_visible;

```

Ezek után a profilkép kerül feltöltésre. Ha nem rendelkezik az orvos profilképpel akkor egy placeholder kép kerül feltöltésre a PictureBox-ba. Itt még a kép adatai eltárolásra kerülnek egy esetleges kép módosításra amikor ezek az adatok is szükségesek lesznek a módosításnál.

"Kép esetén a base64 kód konvertálása"

```

try{ var imageBytes1 =
    Convert.FromBase64String(Profilkep); using
    (MemoryStream ms = new MemoryStream(imageBytes1))
    { ms.Position = 0;
      MemoryStream persistentStream = new MemoryStream();
      ms.CopyTo(persistentStream);
      persistentStream.Position = 0;
      CPb_OrvosKep.Image = Image.FromStream(persistentStream, true);
    }
}
catch (Exception ex)
{
    Debug.WriteLine("Hiba a kép betöltése során: " + ex.Message);
    CPb_OrvosKep.Image = null;
}

```

SendRequestAsync (Post, Delete) Az API post és delete kéréseit kezeli. Ez is a Bejelentkezés.cs-ben található és onnan kerül meghívásra a formokon. Ha a token lejárt új tokenet kér majd írja elvégzi a post vagy delete kérést.

```
public static async Task SendRequestAsync<T>(HttpMethod method,
string endpoint, T data)
{ try { using (var client = new
    HttpClient())
    { if (Token.ExpiresAt <= DateTime.Now)
        await RefreshTokenAsync();

        client.DefaultRequestHeaders.Authorization = new
        System.Net.Http.Headers.AuthenticationHeaderValue("Bearer",
        Token.Access_token);

        var request = new HttpRequestMessage(method,
        $"{BaseUrl}{endpoint}")
        {
            Content = new
                StringContent(JsonConvert.SerializeObject(data),
                Encoding.UTF8, "application/json"),
        };

        var response = await client.SendAsync(request);
        if (!response.IsSuccessStatusCode)
        { var error = await response.Content.ReadAsStringAsync();
            throw new Exception(error); }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"HIBA: {ex.Message}");
    }
}
```

Gyakorlatban használható új adat hozzáadásánál, adatok módosításánál és adatok törlésénél. Adat hozzáadásánál és módosításánál Post-ot használunk míg a törléshez Delete-t. A Doctor_Profil.cs-ben megtalálható mind a 3.

Telefonszám törlése

```
private async Task DeleteDoctorLanguage(int doctor_id,
int language_id) { var data = new { doctor_id,
language_id }; await SendRequestAsync(HttpMethod.Delete,
"/api/v1/api.php/doctors/DeleteLanguage", data);
}
```

Telefonszám hozzáadása

```
private async Task AddDoctorLanguage(int doctor_id, int language_id){
var data = new { doctor_id, language_id };
await SendRequestAsync(HttpMethod.Post,
"/api/v1/api.php/doctors/AddLanguage", data);
}
```

Telefonszám aktív státuszának módosítása

```
private async Task ChangePhonenumberActiveState(int doctor_id, int
phonenumber_id) {
var data = new { doctor_id, phonenumber_id };
await SendRequestAsync(HttpMethod.Post,
"/api/v1/api.php/doctors/ChangePhonenumberActiveState", data);
}
```

4.6.5 Témakezelés

Főbb funkciói

- Dinamikus Light/Dark mód váltás
- Gombok, panelek, chartok automatikus színezése
- Ikonok betöltése témától függően
- Indításkor a Windows theme-je alapján választ a Light és a Dark közül

4.6.5.1 Alapvető metódusok

A témakezeléssel kapcsolatos kódok a ThemeManager.cs-ben található (néhány esetben néhány elem designje a saját form-ján található).

SetTheme

Ez a metódus felel a téma beállításáért

```
public static void SetTheme(string theme)
{
    CurrentTheme = theme;
    ApplyThemeToAllForms();
}
```

A téma beállítását minden vezérlőre be kell állítani (pl: panel, button, datagridview). Ezt az ApplyTheme metóduson keresztül állítjuk be.

Téma alkalmazása a Form vezérlőkre

```
public static void ApplyTheme(Control control)
{ if (control is Form form)
    { if (form.Name == "Doctor_Menu" || form.Name == "Hospital_Menu"
        || form.Name == "Company_Menu" || form.Name == "Admin_Menu")
        { form.BackColor = CurrentTheme == "Light" ?
            Color.FromArgb(179, 185, 196) : Color.FromArgb(22, 26, 29);
          form.ForeColor = CurrentTheme == "Light" ? Color.Black :
            Color.White; } else { form.BackColor =
            CurrentTheme == "Light" ?
            Color.FromArgb(241, 242, 244) : Color.FromArgb(44, 51, 58);
          form.ForeColor = CurrentTheme == "Light" ? Color.Black :
            Color.White;
        }
    }
}
// További vezérlők (Panel, Label, DataGridView stb.)
```

A menü gombok ikonokkal vannak ellátva melyek ugyan úgy a témától (theme) függenek. Ezek külön mappákban vannak eltárolva a színük szerint.

Menü ikonok beállítása

```
private static void SetButtonIcon(Button button, string baseName)
{
    string rootDirectory = Path.GetFullPath
        (Path.Combine(AppDomain.CurrentDomain.BaseDirectory, @"..\..\"));
    string iconFolderPath = Path.Combine(rootDirectory, "Images",
        "Icons");
    string iconFileName = CurrentTheme == "Light" ? "Black" :
        "White"; string imageFileName = CurrentTheme == "Light" ?
        $"{baseName}_Black.png" : $"{baseName}_White.png"; string
    imagePath = Path.Combine(iconFolderPath, iconFileName,
        imageFileName);
    if (File.Exists(imagePath))
    {
        button.Image = Image.FromFile(imagePath);
    }
    else
    {
        Debug.WriteLine($"Icon file not found: {imagePath}");
    }
}
```

4.6.6 Felület váltás

Az admin és a cég jogosultságú felhasználó beléphet az alattuk álló felhasználók felületére a saját felületükről (admin: összes felhasználó, cég: az alárendelt kórházak felületére). Ezek többnyire datagridview-ekben elhelyezett gombokból lehet belépni. Aloldalról már nem lehet még egy aloldalt megnyitni (admin->cég->kórház ez le van tiltva). A main.cs-en el van helyezve egy aloldal változó (IsSubPage).

main.cs aloldal változója

```
public bool IsSubPage { get; set; } = false;
```

Az aloldal megnyitásához az összes olyan adatot továbbítani kell, ami szükséges lesz az adott felhasználó lekérdezéseihez és módosításaihoz. A formváltásnál figyelni kell, hogy melyik formon van a felhasználó, hogy a lekérdezéseknél a változóknál ne ütközzön a felhasználó problémába.

Admin_Company.cs: Cég felület megnyitása

```
private void OpenCompany(dynamic companyID, dynamic companyName)
{
    string username = GlobalData.UserName;
    int userid = GlobalData.UserId;
    Bejelentkezés.GlobalData.CompanyId = companyID;
    Bejelentkezés.GlobalData.RoleIdentifier = "company";
    Bejelentkezés.GlobalData.UserName = "company"; Main mainForm = new
    Main(); mainForm.role_identifier = "company"; mainForm.Text =
    "Care_Compass_-_" + companyName; //Care Compass + a cég neve
    mainForm.IsSubPage = true; mainForm.Show(); mainForm.Activated += (s,
    args) =>
    {
        Debug.WriteLine("Main_form_aktív,_RoleIdentifier=_company");
        Bejelentkezés.GlobalData.RoleIdentifier = "company";
        Bejelentkezés.GlobalData.UserName = companyName;
        Bejelentkezés.GlobalData.CompanyId = companyID;
    };
    // Figyeljük, mikor vesztí el a fókusz a Main form
    mainForm.Deactivate += (s, args) =>
    {
        Debug.WriteLine("Main_form elvesztette_a_fókuszt,_RoleIdentifier=_ad
        min");
        Bejelentkezés.GlobalData.RoleIdentifier = "admin";
        Bejelentkezés.GlobalData.UserName = username;
        Bejelentkezés.GlobalData.UserId = userid;
    };
    // Amikor a Main form bezárul Form1 bezárul
    mainForm.FormClosed += (s, args) =>
    {
        Bejelentkezés.GlobalData.RoleIdentifier = "admin"; // Visszaállítás
        Bejelentkezés.GlobalData.UserName = username;
        Bejelentkezés.GlobalData.UserId = userid;
        this.Close();
    };
};
```

4.6.7 Hibakezelés, input ellenőrzés

4.6.7.1 Hibakezelés

Try-catch használata az API hívásoknál Hibák és válaszok naplózása Debug-okkal.

"Bejelentkezés.cs token frissítés hibakezelés"

```
public static async Task RefreshTokenAsync()
{ try { using (HttpClient client = new
    HttpClient())
    { code...
    }
  }
  catch (Exception ex)
  { throw new Exception($"Hiba történt a refreshTokenAsync
    függvényben: {ex.Message}");
  }
}
```

4.6.7.2 Input ellenőrzés

A módosításnál és új adat bevitelénél figyelünk a megfelelő formátumra, mint például az email legyen megfelelő formátumú vagy a cég adószáma.

Email ellenőrzés

```
private bool IsGoodEmail(string email)
{
    return Regex.IsMatch (email,
        @"^([\w-\.]*)@([\d]{1,3}\. [\d]{1,3}\. [\d]{1,3}\. )|
        (([\w-]+\.)+)([a-zA-Z]{2,4}|[\d]{1,3})(\?)$");
}
```

Adószám ellenőrzés

```
private bool IsValidTaxNumber(string taxNumber)
{ string elso8szam = taxNumber.Substring(0, 8);

    int[] multipliers = { 9, 7, 3, 1, 9, 7, 3
    }; int sum = 0; for (int i = 0; i < 7; i++)
```

```
{ sum += (elso8szam[i] - '0') * multipliers[i];  
}  
int calculatedCheckDigit = (10 - (sum % 10)) % 10;  
int actualCheckDigit = elso8szam[7] - '0';  
  
return calculatedCheckDigit == actualCheckDigit;  
}
```

4.6.8 Tesztelési terv

A tesztelést a program írása közben hajtottuk végre, vagyis integrációs tesztet végeztünk. Itt kitértünk az összes általunk feltételezett hibás lehetőségre és töréspontokkal vizsgáltuk az egyes szekciók milyen eredményeket adnak vissza.

Mivel az alkalmazás törzsét az API adja így annak a tesztelése adja így az API-t részben az applikáción teszteltük. Az applikáció befejezése után tesztre kiadtuk olyan külső személyeknek, akik nem vettek részt a fejlesztésben (családtagok). Hiba esetén megvizsgáltuk és javítottuk a hiba forrását.

4.7 Weboldal fejlesztői dokumentáció

4.7.1 Bevezetés

Ez a dokumentáció bemutatja a CareCompass weboldal fejlesztését. Célja, hogy segítse a fejlesztőket a projekt átlátásában és továbbfejlesztésében.

4.7.2 Rendszer áttekintése

A weboldal célja, hogy a látogatók információkat kapjanak a különböző kórházakról, orvosokról és az elérhető szolgáltatásokról. Az adatokat egy REST API-n keresztül kérjük le, és PHP segítségével jelenítjük meg a felhasználók számára.

4.7.3 Architektúra

A weboldal az alábbi fő komponensekből áll:

1. Felhasználói felület (Frontend)
 - Statikus HTML, PHP, CSS és JavaScript biztosítja a megjelenést.

- AJAX hívások és dinamikus tartalom megjelenítése.

2. Backend (PHP)

- API-k meghívása és az adatok feldolgozása.
- Felhasználói kérések kezelése.

3. API kapcsolat

- Az adatokat a `ApiAccessGet ()` függvény hívja le a megfelelő végpontokról.

4.7.4 Frontend és backend kapcsolata

A webdoldal alapvetően szétbontható frontend és backend részekre. A backendet kiegészíti még az API integráció, aminek a célja, hogy az adatokat összegyűjtse és feldolgozza. Az alábbiakban különböző hívásokat mutatunk be, a megfelelő kódrészletekkel együtt.

API Végpont	Leírás
<code>/api/v1/api.php/GetHospital</code>	Kórház adatainak lekérdezése
<code>/api/v1/api.php/auth/login</code>	Bejelentkeztetés
<code>/api/v1/api.php/appointments/Cancel</code>	Regisztráció

Példa API hívásra PHP-ban:

GET

- `/api/v1/api.php/GetHospital?hospital_id=`

```
$url = $GLOBALS['API_DEFAULT_URL'] .  
      "/hospitals/GetHospital?hospital_id=" . $hospital_id;  
$data = ApiAccessGet($url);
```

Ez a kód az adott kórház adatait kéri le és tárolja a `$data` változóban az `ApiAccessGet(url)` függvény segítségével.

Kapott adatok feldolgozása (példa):

```
<div class="image-title-section">
  <?php if($hospital_data["images"]["cover_image"]['image'] !== null) { ?>
    "
      alt="<?php_echo($hospital_data["images"]["cover_image"]["image_alt"]);_>"
      title="<?php_echo($hospital_data["images"]["cover_image"]["image_title"]);_>"
    ?>">
  <?php } else { ?>
    
    ?>/assets/static/hospital_wide_placeholder.jpg"
    alt="helyettesítő_kép" title="Helyettesítő_kép">
  <?php }?>

  <h1><?php_echo($hospital_data["hospital_name"]); ?></h1>
  <?php if($company_data["images"]["logo"]['image'] !== null) { ?>
    "
    class="logo"
    alt="<?php_echo($company_data["images"]["logo"]["image_alt"]);_>"
    title="<?php_echo($company_data["images"]["logo"]["image_title"]);_>"
  <?php } else { ?>
    
    ?>/assets/static/logo_placeholder.png" class="logo"
    alt="helyettesítő_kép" title="Helyettesítő_kép">
  <?php }?>
</div>
```

A bemutatott példában a visszakapott adatokat alapján betöltjük a kórház bannerét a megfelelő képpel, névvel és céglogóval a frontend megjelenítéshez.

POST

- /api/v1/api.php/auth/login:

```
// Bejelentkezés kezelése
$("#login").bind("submit", async(event) => {
    event.preventDefault();

    const email = $("#login-email").val();
    const password =
        $("#login-password").val();

    try { var requestURL =
        `${BASE_URL}/apiaccess.php`;

        const response = await
            fetch(requestURL, { method:
                'POST', headers: {
                    'X-Requested-With': 'XMLHttpRequest'
                },
                body: JSON.stringify({ "url": `${API_URL}/auth/login`,
                    "data": {
                        "email": email, "password": password }, "setSession":
                        "login" })
            });
        const data = await response.json();

        if (data.success) {
            AddMessage("success", data.message);

            Redirect();
        } else { throw
            Error(data.error.message); }

    } catch (error) {
        AddMessage("error",
            error.message); }
    });
```

A fenti kódban szerepel a bejelentkezést végző javascript kódrészlet. A kód elején beolvassuk a felhasználó által megadott email címet és jelszót, majd ezt egy fetch metódus segítségével továbbítjuk POST kérésként a szervernek, ami feldolgozza azt és

visszatér helyes vagy helytelen válasszal. Ennek függvényében hiba/siker üzenetet jelenítünk meg a felhasználó számára, aki ebből tudja a következő teendőjét.

DELETE

- /api/v1/api.php/appointments/Cancel:

```
async function DeleteReservation(reservation_id) { try
{ var requestURL = `${BASE_URL}/apiaccess.php`;

    const response = await fetch(requestURL, {
        method: 'DELETE', headers: {
            'X-Requested-With': 'XMLHttpRequest'
        }, body: JSON.stringify({
            "url":
            `${API_URL}/appointments/CancelReservation`, "data":
            { "reservation_id": reservation_id },
            "includeUserId": true })
    });

    const data = await response.json();

    if (data.success) {
        AddMessage("success", data.message);

        Redirect("../profil");
    } else {
        throw Error(data.error.message);
    }

} catch (error) {
    AddMessage("error", error.message);
}
}
```

A javascript kódrészlet segítségével a felhasználó lefoglalt időpontja törölhető. Az oldal betöltéskor rögzíti a 'reservation_id' változó értékét, majd a gombnyomást követően a fetch metódussal DELETE kérést küldünk a szervernek, ami feldolgozza azt és válasszal

tér vissza felhasználónak. Ezt a választ meg is jelenítjük, ami lehet sikeres vagy sikertelen. Sikeres törlés esetén átirányítjuk a profil oldalra.

4.7.5 Útvonalkezelés az oldalon

A weboldal egy központi fájlra van átirányítva minden esetben és a böngészőben megjelenő link ez alapján tölt be különböző megjelenítési sablonokat.

Az átirányítást végző kódrészlet (htaccess):

```
RewriteEngine On
RewriteBase /mesterremek/

# Allow access to everything in the "assets" folder
RewriteCond %{REQUEST_URI} ^/mesterremek/assets/ [NC]
RewriteRule ^ - [L]

# Allow access to everything in the "extra" folder
RewriteCond %{REQUEST_URI} ^/mesterremek/extra/ [NC]
RewriteRule ^ - [L]

# Allow access to the "apiaccess.php" file in the root directory
RewriteCond %{REQUEST_URI} ^/mesterremek/apiaccess.php$ [NC]
RewriteRule ^ - [L]

# Redirect everything else to index.php
RewriteRule ^ index.php [L]
```

Ez a htaccess kódrészlet minden kérést átirányít az index.php fájlra, kivéve néhány elengedhetetlen útvonalat, mint például a képek, stíluslapok, szkriptek mappáit és az API hívás működéséért felelős apiaccess.php fájlhoz engedett hozzáférést.

Irányválasztó (router) működése:

```

<?php

// kapcsolódás az adatbázishoz require_once("config.php");
require_once ("database.php");
$db = new Database($GLOBALS["database_host"],
$GLOBALS["database_name"], $GLOBALS["database_user"],
$GLOBALS["database_password"]);

$conn = $db->connect();
// Aktuálisan kért útvonal előkészítése, főleg részletek leszedése
$requestUri = trim(parse_url($_SERVER['REQUEST_URI'],
PHP_URL_PATH), '/');

$basePath = "mesterremek"; // Adjust to project base path
$requestPath = str_replace($basePath . "/", "",
$requestUri);
if($basePath == $requestPath){
    $requestPath = "/";
}
// Útvonal kérés az adatbázisban

$stmt = $conn->prepare("SELECT * FROM endpoints WHERE endpoint = ? AND active = 1 AND
deleted = 0 LIMIT 1"); $stmt->execute([$requestPath]);
$route = $stmt->fetch(PDO::FETCH_ASSOC);
if (!$route) {

    // Ha nincs pontos egyezés, keresés paraméter alapján $stmt = $conn->prepare("SELECT *
FROM endpoints WHERE endpoint LIKE '%:%' AND active = 1 AND deleted = 0"); $stmt-
>execute();
    $dynamicRoutes = $stmt->fetchAll(PDO::FETCH_ASSOC);
    foreach ($dynamicRoutes as $dynamicRoute) {

        $pattern = preg_replace('/:\\w+/', '([^\s/]+)', $dynamicRoute['endpoint']); //
paraméter helyettesítése Regex-el
        $pattern = str_replace('/', '\\/', $pattern); if (preg_match('/^' . $pattern . '$/',
$requestPath,
        $matches)) {

            $route = $dynamicRoute;

            array_shift($matches); // Teljes egyezés törlése

            $params = $matches; // Paraméterek kinyerése

        }

    }
}
if ($route) {

    // Egyezés esetén betöltjük a vezérlőt
    $controllerPath = __DIR__ . '/controllers/' .
    $route['controller'];

    if (file_exists($controllerPath)) { require_once $controllerPath;
    } else {

        http_response_code(500);

    }
} else {

    // Nincs ilyen útvonal http_response_code(404);
} ?>

```

Az útvonalválasztó lekérdezi az adatbázisból a lehetséges útvonalakat, majd ezt követően feldolgozza azt. Amennyiben talál ilyen útvonalat megpróbálja betölteni az adott oldalt az URL-ben található adatok alapján. Hiba vagy nem talált elérési útvonal esetén megfelelő oldalra irányítja a felhasználót, ahol tájékoztatást kap a hibáról.

4.7.6 Weboldal konfigurálása

A weboldal konfigurálásához 3 elengedhetetlen lépést kell végrehajtanunk. Amennyiben nem a fő domainen futtatjuk, abban az esetben a .htaccess fájlban módosítanunk kell a RewriteBase-t

```
RewriteBase {útvonal}
```

A gyökérmappában található config.php fájlban a konfigurációs paramétereket személyre kell szabnunk:

```
$GLOBALS['API_DEFAULT_URL'] = {api.php elérési útvonala};
$GLOBALS['DEFAULT_URL'] = {főoldal elérési útvonala};
$GLOBALS['DEFAULT_PATH'] = realpath(dirname(__FILE__));

// adatbázis adatok
$GLOBALS["database_host"] = {adatbázis szerver};
$GLOBALS["database_name"] = {adatbázis neve};
$GLOBALS["database_user"] = {felhasználónév};
$GLOBALS["database_password"] = {jelszó};

// email
$GLOBALS['email_username'] = {email cím};
$GLOBALS['email_password'] = {jelszó}; $GLOBALS['email_port'] = {port};
$GLOBALS['email_host'] = {mail szerver};
```

A fenti két konfigurációs módosítást kiegészítő harmadik, egyben utolsó teendő a javascript konfigurációs fájl módosítása. A fájl elérési útvonala: </assets/scripts/config.js>

```
// Alap url létrehozása
const BASE_URL = {főoldal elérési útvonala};
//API elérésének URL címe
const API_URL = {api.php elérési útvonala};
```

A fenti kódokban kapcsos zárójellel jelölt változók kitöltése kötelező.

4.7.7 Hibakezelés

Az oldalon a hibakezelést egyedül a szerver oldali API biztosítja, amely hibával tér vissza és ezt javascript kód jelenti meg üzenetben a felhasználó számára.

Hibaüzenetek megjelenítése:

```
var showTime = 3000; var waitTime
= 1000; var messageShowing =
false;
function ShowMessage(){
  if(messageShowing){ return;
  }
  if(sessionStorage.getItem("messages")){ messages =
    JSON.parse(sessionStorage.getItem("messages")); // ü zenetek
  } else { messages =
    [];
  }
  if(messages.length > 0){
    // üzenet felöltése
    $("#msg").removeClass("error");
    $("#msg").removeClass("warning");
    $("#msg").removeClass("success");
    $("#msg").addClass(messages[0][0]); $("#msg-
    text").text(messages[0][1]);
    // üzenet megjelenítése messageShowing =
    true;
    $("#msg-container").fadeIn(150); setTimeout(() => { //
    üzenet eltávolítása
      $("#msg-container").fadeOut(150); //
      üzenet törlése a listából messages.shift(-
      1);
      sessionStorage.setItem("messages",
      JSON.stringify(messages)); messageShowing =
      false;
      // hogyha még van üzenet megjelenítjük
      if(messages.length > 0){ setTimeout(() => {
        ShowMessage();
      }, waitTime);
      }
    }, showTime);
  } }
```

A javascript kód a különböző üzenetek megjelenítéséért felel, amelyeket a sessionStorage tárol el, így oldalak közt is könnyen átvihető, akár több is.

Beolvassa a sessionStorage tartalmát, majd innen a legrégebbivel kezdve megjeleníti azokat sorrendben, ahogy beérkeztek.

3 típusa a hiba, figyelmeztetés és siker, amelyek különböző színnel - piros, sárga, zöld - jelennek meg a felhasználó számára a hibát egyértelműsítve.

4.7.8 API hozzáférés a weboldal számára

A weboldal számára az API biztonságos és megbízható elérést az apiaccess.php biztosítja. Itt lehetőségünk van php, valamint AJAX kérések teljesítésére.

Token érvényességének ellenőrzése Az API lekérdezésekhez minden esetben szükségünk van token használatára, amelyet a SESSION-ben tárolunk és az alábbi függvény az érvényesség ellenőrzéséért és a megújításért felel.

```

function CheckApiAccessToken(){ // visszatérés a tokennel
//token ellenőrzése, ha nincs, lekérése
if(isset($_SESSION['token']) && $_SESSION['token'] != null &&
strtotime($_SESSION['token']['expiresAt']['date']) > strtotime('now')){

    $token = $_SESSION['token']['access_token'];
} else { // token lekérése if(isset($_SESSION['token']) &&
    $_SESSION['token']['level'] == 'public') ||
    !isset($_SESSION['logged_in']) ||
    !isset($_SESSION['user_id']) || $_SESSION['logged_in'] == false){ // új nyilvános token
    Lekérézése

        // cURL beállítások
        $ch = curl_init($GLOBALS['API_DEFAULT_URL'] .
        '/public/public-token');
        curl_setopt($ch, CURLOPT_RETURNTRANSFER, true); curl_setopt($ch, CURLOPT_HTTPHEADER,
        [
            "Content-Type: application/json" ]);
        $response = curl_exec($ch); if (curl_errno($ch)) { echo
        'Error: ' . curl_error($ch);
        }
        curl_close($ch);
        // Válasz feldolgozása
        $data = json_decode($response, true);
        $_SESSION['token'] = $data['data'];
        $token = $_SESSION['token']['access_token'];
    } else { // privát token frissítése
        $sendData = [
            "user_id" => $_SESSION['user_id']
        ];
        // cURL beállítások
        $ch = curl_init($GLOBALS['API_DEFAULT_URL'] .
        '/auth/refreshToken');
        curl_setopt($ch, CURLOPT_RETURNTRANSFER, true); curl_setopt($ch, CURLOPT_POST, true);
        // POST kérés curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($sendData));
        curl_setopt($ch, CURLOPT_HTTPHEADER, [ "Authorization: Bearer " .
            $_SESSION['token']['access_token'],
            "Content-Type: application/json" ]);
        $response = curl_exec($ch); if (curl_errno($ch)) { echo
        'Error: ' . curl_error($ch);
        }
        curl_close($ch);
        // Válasz feldolgozása
        $data = json_decode($response, true);
        $_SESSION['token'] = $data['data']['token'];
        $token = $_SESSION['token']['access_token'];
    } }
    return $token;
}

```

A kódrészlet első felében megnézzük a helyileg elmentett token lejáratát, amennyiben érvényes visszaadjuk a már mentett adatot. Ellenkező esetben az API felé intézett kéréssel új hozzáférési tokent kapunk, majd ezt adjuk vissza a kéréshez.

4.8 API dokumentáció

4.8.1 Bevezetés

Ez a dokumentáció bemutatja a CareCompass API fejlesztését. Célja, hogy segítse a fejlesztőket a projekt átlátásában és továbbfejlesztésében.

4.8.2 api.php és .htaccess működése

Az API egy központi fájl, az api.php segítségével működik, melyre minden kérést az alábbi .htaccess fájl irányít át.

```
RewriteEngine On
RewriteCond %{HTTP:Authorization} ^(.*)
RewriteRule .* - [e=HTTP_AUTHORIZATION:%1]
# Közvetlen hozzáférés tiltása minden PHP fájlhoz
<FilesMatch "\.php$">
    Order allow,deny
    Deny from all
</FilesMatch>
# Csak az api.php fájl elérhető kívülről
<Files "api.php">
    Allow from all
</Files>
# Minden v1 API kérést az api.php fájlhoz irányítunk, az api.php
# fájl megadása nélkül
RewriteRule ^api/v1/api/(.*)$ api.php?resource=$1 [L,QSA]
# Minden v1 API kérést az api.php fájlhoz irányítunk
# RewriteRule ^api/v1/(.*)$ api.php [L,QSA]
```

A fenti htaccess kód letilt minden direkt php kiterjesztésű fájllelérést, kivéve az api.php fájlhoz, valamint minden kérést ide irányít automatikusan.

Az api.php tartalma:

```
<?php
// error_reporting(E_ALL);
// ini_set('display_errors', 1);
require_once '../..../config.php';
// Adatbázis osztály meghívása require_once
'./database/database.php';
// Response osztály meghívása require_once
"./classes/Response.php";
// Logger osztály meghívása require_once
"./classes/Logger.php";
// Token osztály meghívása require_once
"./classes/Token.php";
// Mailer osztály meghívása require_once
"./classes/Mailer.php";
// Naptáresemény generátor meghívása require_once
"./classes/ICS.php";
// Függvények betöltése require_once
"./functions.php";
```

A fájl elején betöltésre kerülnek a szükséges osztályok és egyéb fájlok. Ezután létrejön az adatbázis kapcsolat és az eseménynapló frissítéséért felelős objektum is.

```

// Új adatbázis kapcsolat létrehozása
$db = new Database($GLOBALS['database_host'], $GLOBALS['database_name'],
$GLOBALS['database_user'], $GLOBALS['database_password']);
$conn = $db->connect();

// Új naplózó létrehozása
$logger = new Logger($conn);

// Az útvonal kinyerése
$requestUri = $_SERVER['REQUEST_URI'];
$requestUri = str_replace("/mesterremek", "", $requestUri); //kiszedi a link azon részét, amely rámutat a
györkérmappára, amiben a program elhelyezkedik
// var_dump($requestUri);
$uriSegments = explode('/', trim($requestUri, '/')); // backslash kicserélése / jelre
// Ellenőrizzük, hogy a megfelelő útvonalon vagyunk if ($uriSegments[0] === 'api' && $uriSegments[1]
=== 'v1' && $uriSegments[2] === 'api.php') {
    $resource = $uriSegments[3] ?? null; // Erőforrás
    $method = $_SERVER['REQUEST_METHOD']; // Lekérdezzük a HTTP metódust
    switch ($resource) { case 'auth': // publikusan elérhető $function = $uriSegments[4]
        ?? null; require_once './endpoints/auth.php'; // Az autentikációs logika break;
        ...
    case 'contact': // nyilvános végpont
        // szerveren futó javascript hívhat meg, ellenőrzés
        if($_SERVER['SERVER_ADDR'] === $_SERVER['REMOTE_ADDR'] &&
            $_SERVER['REQUEST_METHOD'] == "POST"){
            // üzenet tartalmi ellenőrzése sendContactFormEmail(file_get_contents("php://input"),
            $conn); // adatok kinyerése
        } else {
            Response::error(403, "Nincs megfelelő jogosultsága");
        }
        break;
        ...
    default:
        Response::error(404, "Invalid endpoint"); break;
    }
} else {
    Response::error(404, "API version not specified");
}
}

```

A link feldarabolását követően feldolgozzuk a kérést és a megfelelő logikai végpontot hívjuk meg hozzá, ahol további műveletek, lekérdezések segítségével választ küldünk a kérésre.

Egyetlen kivételes végpont a 'contact' névvel rendelkező kapcsolati oldal, mely kirázólag a szerverről érhető el, hogy külső támadás ellen védett legyen.

Ezen kívül lehetőségünk van publikus token igénylésére is, mellyel a további funkciók érhetőek el.

4.8.3 API logikai vezérlők

Az alábbi táblázatban található az összes logikai vezérlő átfogó leírása:

Vezérlő	Leírás
auth.php	Felhasználó autentikációja, regisztrációja, tokenműveletek
statistics.php	Statisztikák lekérdezése
doctors.php	Orvosokhoz tartozó adatok lekérdezése, módosítása, hozzáadása, törlése
hospitals.php	Kórházakhoz tartozó adatok lekérdezése, módosítása, hozzáadása, törlése
companies.php	Cégekhez tartozó adatok lekérdezése, módosítása, hozzáadása, törlése
users.php	Felhasználókhoz tartozó adatok lekérdezése, módosítása, hozzáadása, törlése
services.php	Szolgáltatásokhoz tartozó adatok lekérdezése, módosítása, hozzáadása, törlése
appointments.php	Időpontokhoz tartozó adatok lekérdezése, módosítása, hozzáadása, törlése
data.php	Egyéb adatok lekérdezése, módosítása, hozzáadása, törlése

calendar.php	Naptárhoz szükséges adatok lekérdezése, módosítása, hozzáadása, törlése
public_token.php	Publikus hozzáférésű token lekérdezése

4.8.4 API token bemutatása

A rendszer egyedi token-alapú autentikációt használ a biztonságos hozzáférés érdekében. Az API végpontjaihoz való hozzáféréshez egy hozzáférési token (access token) szükséges, amelyet a rendszer generál és adatbázisban tárol. Ez a token nem JWT, hanem egy biztonságos, véletlenszerűen generált hexadecimális string, amely meghatározott időn belül lejár.

Token generálása

```
public static function generateToken($db, $user_id, $validTime
= "+10 minute", $length = 32) {
    $accessToken = bin2hex(random_bytes($length));
    $expiresAt = new DateTime($validTime); // Token élettartama
    $stmt = $db->prepare("INSERT INTO tokens (user_id, access_token,
expires_at) VALUES (?, ?, ?)");
    $stmt->execute([$user_id, $accessToken, $expiresAt->format('Y-m-d
H:i:s')]);
    if($user_id != null){
        //log frissítése
        $logger = new Logger($db);
        $logger->LogUserAction($user_id, "New token requested",
$_SERVER['REMOTE_ADDR'], isset($_SERVER['REMOTE_HOST']) ?
$_SERVER['REMOTE_HOST'] : null, null, ["success" => false]);
    }
    return array("access_token" => $accessToken, "expiresAt" =>
$expiresAt, "level" => "private");
}
```

- A token véletlenszerűen generált bináris adat, amelyet hexadecimálissá alakítunk (bin2hex(random_bytes(length))).
- Alapértelmezett hossza: 32 byte (64 karakter hosszú string).
- A lejáratí idő dinamikusan állítható, alapértelmezés szerint 10 perc.
- Az új token az adatbázisban tárolódik, és az API minden kérés során ezt ellenőrzi.

Token tárolása

- A generált token az **tokens** táblában tárolódik, amely tartalmazza:
 - **user_id** (a felhasználó azonosítója)
 - **access_token** (a generált token)
 - **expires_at** (lejáratási idő)

Token ellenőrzése

```
public static function CheckToken($db, $accessToken,
$endpoint){
// lejárati idő lekérdezése a token páros alapján
$stmt = $db->prepare("SELECT 'tokens'.'.id',
'tokens'.'.user_id', 'tokens'.'.expires_at' FROM 'tokens' WHERE
'tokens'.'.access_token' = ?");
$stmt->execute([$accessToken]);
$resp = $stmt->fetch(PDO::FETCH_ASSOC);
$token_expiration = $resp['expires_at'];
$user_id = $resp['user_id'];
//log frissítése
$logger = new Logger($db);
// végpontok alapértéke
$apiEndpoints = array();
// token létezésének ellenőrzése if
($resp == false) {
    $logger->LogUserAction($user_id, "Usage of invalid token",
    $_SERVER['REMOTE_ADDR'], isset($_SERVER['REMOTE_HOST']) ?
    $_SERVER['REMOTE_HOST'] : null, null, ["success" => false]);
    // return ["success" => false, "status_code" => 401,
    "message" => "Érvénytelen token"];
    Response::error(401, "Érvénytelen token", ["token_detail"
    => false]);
}
// lejárati idő ellenőrzése
$expiresAt = new DateTime($token_expiration); if ($expiresAt <
new DateTime()) {
    $logger->LogUserAction($user_id, "Usage of expired token",
    $_SERVER['REMOTE_ADDR'], isset($_SERVER['REMOTE_HOST']) ?
    $_SERVER['REMOTE_HOST'] : null, null, ["success" => false]);
    // return ["success" => false, "status_code" => 401,
    "message" => "Lejárt token"];
    Response::error(401, "Lejárt token", ["token_detail" => false]);
}
if($user_id != null){
    // Ide kerül a jogosultságellenőrzés logikája
    $stmt = $db->prepare("SELECT 'roles'.'.tags' FROM 'roles'
    INNER JOIN 'users' ON 'roles'.'.id' = 'users'.'.role_id' WHERE 'users'.'.id' = ?");
    $stmt->execute([$user_id]);
    $resp = $stmt->fetch(PDO::FETCH_ASSOC);
    // jogosultság nem létezik if($resp == false)
    {
        $logger->LogUserAction($user_id, "No role found",
        $_SERVER['REMOTE_ADDR'], isset($_SERVER['REMOTE_HOST']) ?
        $_SERVER['REMOTE_HOST'] : null, null, ["success" => false]);
        // return ["success" => false, "status_code" => 401,
        "message" => "Nincs ilyen jogosultság"];
        Response::error(401, "Nincs ilyen jogosultság");
    }
    $apiEndpoints = json_decode($resp['tags'])->api_endpoints;
}
```

```

// végpont lekérdezés adatbázisból
$stmt = $db->prepare("SELECT 'api_endpoints'.'.id',
'api_endpoints'.'.public' FROM 'api_endpoints' WHERE 'api_endpoints'.'.name' = ? AND
'api_endpoints'.'.deleted' = ?");
$stmt->execute([$endpoint, false]);
$endpointResp = $stmt->fetch(PDO::FETCH_ASSOC);
if($endpointResp == false) {
    $logger->LogUserAction($user_id, "No endpoint found",
    $_SERVER['REMOTE_ADDR'], isset($_SERVER['REMOTE_HOST']) ?
    $_SERVER['REMOTE_HOST'] : null, null, ["success" => false,
    "endpoint" => $endpoint]);
    // return ["success" => false, "status_code" => 401,
    "message" => "Nincs ilyen végpont"];
    Response::error(401, "Nincs ilyen végpont", ["endpoint" =>
    $endpoint]); }
$endpoint_id = $endpointResp['id'];
$endpoint_public = $endpointResp['public'];
if($endpoint_public || $apiEndpoints == "ALL" || in_array($endpoint_id,
$apiEndpoints)){
    // $logger->LogUserAction($user_id, "Usage of token",
    $_SERVER['REMOTE_ADDR'], isset($_SERVER['REMOTE_HOST']) ? $_SERVER['REMOTE_HOST'] :
    null, null, ["success" => true,
    "endpoint" => $endpoint]);
    return ["success" => true, "user_id" => $user_id, "message"
    => "Jóváhagyva"];
    // Response::success(["access_token" => $accessToken, "expiresAt" => $expiresAt,
    "success" => true], "Access grantend.");
} else{
    $logger->LogUserAction($user_id, "Usage of insufficient
    token", $_SERVER['REMOTE_ADDR'], isset($_SERVER['REMOTE_HOST']) ?
    $_SERVER['REMOTE_HOST'] : null, null, ["success" => false, "endpoint" =>
    $endpoint]);
    // return ["success" => false, "status_code" => 403, "message" => "Nincs
    megfelelő jogosultsága"];
    Response::error(403, "Nincs megfelelő jogosultsága",
    ['user_id' => $user_id, 'token' => $accessToken]);
}
}

```

- Minden API kérés során a rendszer ellenőrzi, hogy:
 - A token létezik az adatbázisban.
 - A token nem járt le (**expires_at** mező ellenőrzése).
 - A token rendelkezik a megfelelő jogosultságokkal

A token validálása a Authorization fejléccel történik:

```
Authorization: Bearer {access_token}
```

Token naplózása

- Minden új token létrehozásakor log entry készül, amely naplózza:
 - A felhasználó azonosítóját (**user_id**)
 - Az IP-címet (**REMOTE_ADDR**)

Biztonsági megoldások

- Egy token csak egy adott időszakig érvényes, utána új kérés szükséges.
- Minden API végpont token-alapú hitelesítést igényel, kivéve a nyilvános végpontokat.
- A tokenek adatbázisban tárolása lehetővé teszi a visszavonást, ha szükséges.

4.8.5 Email küldés API-val

Az API automatikusan email-t küld a felhasználóknak különböző esetekben.

Emailek:

- Sikeres regisztráció
- Email megerősítve
- Elfelejtett jelszó
- Felhasználói fiók frissítése
- Jelszó frissítése
- Új időpontfoglalás
- Időpont lemondása
- Orvost céghez rendelték
- Orvost kórházhoz rendelték
- Létrehoztak egy új céget az emaillel
- Létrehoztak egy új kórházat az emaillel
- Kapcsolat oldal értesítés

Példa email küldésére:

```

function newReservationEmail($email, $id, $date, $location,
$duration, $service){
    // sablon betöltése és adatok helyettesítése
    $template = file_get_contents($GLOBALS['DEFAULT_PATH'] .
'/api/v1/email_templates/newReservation.html');
    $template = str_replace('{{date}}', $date, $template);
    $template = str_replace('{{location}}', $location, $template);
    $template = str_replace('{{duration}}', $duration, $template);
    $template = str_replace('{{service}}', $service, $template);

    $template = str_replace('{{link}}', $GLOBALS['DEFAULT_URL'] .
"/foglalasok/" . $id, $template);
    // ICS esemény adatok
    $event_title = $service; $event_description =
    "";
    $event_location = $location;
    $start_time = $date; // YYYY-MM-DD HH:MM:SS formátum
    $end_time = date_add(date_create($date),
date_interval_create_from_date_string($duration . " minute"))-
>format("Y-m-d H:i");
    // ICS esemény létrehozása
    $ics = new ICSGenerator($event_title, $event_description,
    $event_location, $start_time, $end_time);
    $ics_content = $ics->generateICS();
    // Mentés fájlba
    $filename = "reservation_" . time() . ".ics";
    file_put_contents($filename, $ics_content);
    $resp = Mailer::sendMail($email, "Sikeres időpontfoglalás!",
    $template, $filename);
    if($resp){ return
        true;
    } else {
        return $resp;
    }
}

```

A függvény elején behelyettesítjük a sablont a helyes adatokkal, majd előkészítjük a naptáresemény fájl létrehozásához szükséges adatokat.

Létrehozzuk a naptáreseményt, majd a PHPMailer segítségével elküldjük az emailt.

4.8.6 Adatvalidálás

Az szervernek küldött adatokat minden esetben validálni kell. A biztonságot a `'htmlspecialchars()'` és **előkészített lekérdezések** segítségével tudjuk garantálni, valamint a főbb adatokat külön validálási eljárásnak tesszük ki.

A lenti példában a jelszó ellenőrzését mutatjuk be.

```
function ValidatePassword($password) {
    // jelszó ellenőrzése karakterszámmra és karakterfajtára
    if(strlen($password) < 8){
        Response::error(400, "A jelszónak legalább 8 karakter hosszúnak kell lennie!", ['errorFields' => array("password")]);
    } else if(!preg_match('/[A-Z]/', $password)){
        Response::error(400, "A jelszónak tartalmaznia kell nagybetűt!", ['errorFields' => array("password")]);
    } else if(!preg_match('/[a-z]/', $password)){
        Response::error(400, "A jelszónak tartalmaznia kell kisbetűt!", ['errorFields' => array("password")]);
    } else if(!preg_match('/[0-9]/', $password)){
        Response::error(400, "A jelszónak tartalmaznia kell számot!", ['errorFields' => array("password")]);
    } else if(!preg_match('/^[A-Za-z0-9]/', $password)) {
        Response::error(400, "A jelszónak tartalmaznia kell különleges karaktert!", ['errorFields' => array("password")]);
    }
}
```

A függvény ellenőrzi, hogy a jelszó megfelelő hosszúságú-e, valamint tartalmaz-e minden szükséges karaktertípust.

4.8.7 API tesztelése

API Tesztelésének Folyamata

Az API tesztelését strukturáltan és végpontonként végeztük. Minden végpont esetében figyelembe vettük a lehetséges kimeneteket, és törekedtünk arra, hogy lefedjük az összes eshetőséget.

A tesztelési folyamat az alábbiak szerint zajlott: GET kérések tesztelése

Minden GET típusú végpont esetében ellenőriztük:

- Mit ad vissza a rendszer, ha létező erőforrást kérünk le (pl. meglévő felhasználó azonosító alapján).
- Mit ad vissza, ha nem létező azonosítóval történik a lekérdezés (pl. hibakód, hibaüzenet).

Szabad időpontok lekérdezése

- Leírás: Adott orvoshoz, szolgáltatáshoz és kórházhoz tartozó szabad időpontok lekérdezését végző végpont. A megadott bemeneti paraméterek alapján visszatér a szabad időpontokkal.
- URL: </api/v1/api.php/appointments/GetFreeAppointmentTimes>
- Metódus: **GET**
- Paraméterek:
 - **service_id**: Kötelező, int
 - **hospital_id**: Kötelező, int
 - **doctor_id**: Kötelező, int
 - **from**: Kötelező, date
 - **to**: Kötelező, date
- Válasz (200 - OK):

```
{
  "success": true,
  "data": {
    "free_times": {
      "2025-04-05": [
        "08:00", "08:40",
        "09:20",
        "10:00"
      ]
    }
  },
  "meta": {
    "response_timestamp": 1743786079
  },
  "message": "Sikeres kérés!"
}
```

- Válasz (404 - Nincs találat):

```
{
  "success": false,
  "error": {
    "code": 404,
    "message": "Nincs szabad időpont a megadott intervallumban!",
    "details": []
  }
}
```

Adott kórház lekérdezése

- Leírás: Adott kórház teljes adatlapjának lekérdezése
- URL: </api/v1/api.php/hospitals/GetHospital>
- Metódus: GET
- Paraméterek: – **hospital_id**: Kötelező, int
- Válasz (200 - OK):

```

{
  "success": true,
  "data": {
    "hospital": {
      "hospital_id": 3,
      "hospital_name": "Kórház 1",
      "company_id": 2,
      "company_name": "Cég 1 Rt.",
      "hospital_address": "1127 Budapest, Ferenc körút  
34-78",
      "hospital_open_hours": "{\n\"0\": [\n\"6:00\",  
\n\"22:00\"], \n\"1\": [\n\"6:00\", \n\"22:00\"], \n\"2\": [\n\"6:00\",  
\n\"22:00\"], \n\"3\": [\n\"6:00\", \n\"22:00\"], \n\"4\": [\n\"6:00\",  
\n\"22:00\"], \n\"5\": [\n\"6:00\", \n\"20:00\"], \n\"6\": [\n\"8:00\",  
\n\"20:00\"]}",
      "hospital_description": "Ez egy szuper kórház, ahol  
kezelések vannak",
      "hospital_email": "korhaz1@gmail.com",
      "phonenumbers": [],
      "images": {
        "cover_image": {
          "image_id": null,
          "image_title": null,
          "image_alt": null,
          "image_visible": null,
          "image": null
        },
        "profile_image": {
          "image_id": 2,
          "image_title": "Kórház 1 Profilkép",
          "image_alt": "korhaz 1 profilkep",
          "image_visible": 1, "image":
            "data:image/gif;base64,R0lGODlhAQABAAAAACw="
        }
      }
    }
  },
  "meta": {
    "response_timestamp": 1743868434
  },
  "message": "Sikeres kérés!"
}

```

- Válasz (404 - Nincs találat):


```
{
  "success": false,
  "error": {
    "code": 404,
    "message": "Nincs ilyen kórház!",
    "details": []
  }
}
```

POST kérések tesztelése

A POST végpontokat úgy teszteltük, hogy:

- Megvizsgáltuk, hogyan működik az adatok helyes formátumban történő beküldésekor, és megfelelően létrejön-e az új erőforrás.
- Teszteltük a hibás vagy hiányos adatok beküldésének esetét is, hogy lássuk, megfelelő hibakezelést alkalmaz-e az API (pl. validációs hiba, 400-as válasz).

Bejelentkezés

- Leírás: Ez a végpont felel a felhasználó hitelesítésért, valamint a hitelesítést követően az összes szükséges adat biztosításához a sikeres és biztonságos bejelentkeztetéshez.
- URL: `/api/v1/api.php/auth/login`
- Metódus: **POST**
- Paraméterek:
 - **email**: Kötelező, string – **password**: Kötelező, string
- Válasz (200 - OK):

```

{
  "success": true,
  "data": {
    "user_id": 1,
    "user_name": "Példa Béla",
    "user_email": "peldabela@gmail.com",
    "role_identifier": "basic",
    "web_login": "1",
    "token": {
      "access_token": "db38b39e7a76a2c778da85e98
7875ef1099fd701cf9d18f68...",
      "expiresAt": {
        "date": "2025-04-02 21:28:56.429260",
        "timezone_type": 3,
        "timezone": "Europe/Berlin"
      },
      "level": "private"
    }
  },
  "meta": {
    "response_timestamp": 1743621536
  },
  "message": "Sikeres azonosítás!"
}

```

Nyelv hozzáadása orvoshoz

- Leírás: Korábban hozzá nem adott nyelv hozzárendelése az orvoshoz
- URL: </api/v1/api.php/doctors/AddLanguage>
- Metódus: **POST**
- Paraméterek:
 - **doctor_id**: Kötelező, int
 - **language_id**: Kötelező, int
- Válasz (200 - OK):

```

{
  "success": true,
  "data": [],
  "meta": {
    "response_timestamp": 1743866439
  },
  "message": "Sikeres kérés!"
}

```

- Válasz (404 - Nincs találat):

```
{
  "success": false,
  "error": {
    "code": 404,
    "message": "Nincs ilyen nyelv!",
    "details": []
  }
}
```

4.8.8 DELETE kérések tesztelése

Itt azt vizsgáltuk meg, hogy:

- Mi történik, ha egy létező erőforrást próbálunk törölni – sikeres törlés és helyes válaszkód.
- Hogyan reagál az API, ha egy nem létező azonosítót adunk meg – például megfelelő hibakód és üzenet visszaadása.

Orvos törlése

- Leírás: Orvos eltávolítása a cégtől.
- URL: `/api/v1/api.php/companies/RemoveDoctor`
- Metódus: **DELETE**
- Paraméterek:
 - `company_id`: Kötelező, int
 - `doctor_id`: Kötelező, int
- Válasz (200 - OK):

```
{
  "success": true,
  "data": [],
  "meta": {
    "response_timestamp": 1743858998
  },
  "message": "Sikeres kérés!"
}
```

Felhasználó törlése

- Leírás: Felhasználó törlése a rendszerből
- URL: `/api/v1/api.php/users/Delete`
- Metódus: `DELETE`
- Paraméterek:
 - `user_id`: (Kötelező, int) A törlendő felhasználó azonosítója.
- Válasz (200 - OK):

```
{
  "success": true,
  "data": [],
  "meta": {
    "response_timestamp": 1743881812
  },
  "message": "Sikeres kérés!"
}
```

5 Felhasználói dokumentációk

5.1 Telepítési útmutató

1. Projekthez tartozó fájlok letöltése

- Töltse le a GitHub-ról a következő fájlokat:
 - WinApp
 - WebApp
 - API
 - Adatbázis

2. XAMPP telepítése

Telepítse a XAMPP alkalmazást a hivatalos weboldaláról.

3. XAMPP konfiguráció beállítása

- Nyissa meg a XAMPP konfigurációs fájlt.
- Állítsa be, hogy az Apache szerver a 80-as portot használja.

4. Apache és MySQL modulok elindítása

- Indítsa el a XAMPP alkalmazást.
- Kapcsolja be az Apache és MySQL modulokat.

5. Adatbázis importálása

- Nyissa meg a böngésződben a következő címet: <http://localhost/dashboard/>.
- A navigációs menü jobb oldalán kattintson a phpMyAdmin gombra.
- A phpMyAdmin felületén válassza az Importálás opciót, majd töltsd be a legfrissebb SQL adatbázis fájlt.

6. WebApp másolása és átnevezése

- Keresse meg a XAMPP telepítési mappáját.
- Másolja a WebApp tartalmát a htdocs mappába.
- Nevezze át a másolt mappát mesterremek névre.

7. API telepítése

- A mesterremek mappán belül hozza létre egy új mappát api néven.
- Másolja ide a GitHub-ról letöltött API fájlokat.

8. Asztali alkalmazás indítása

- Nyissa meg a WinApp mappát.

- A Debug almappában található .exe fájl elindításával futtassa az alkalmazást.

Ezeket a lépéseket követve az alkalmazás sikeresen telepítésre kerül, és használatra kész

5.2 Weboldal felhasználói dokumentáció

5.2.1 Bevezetés

Rövid ismertetés a CareCompass weboldalról

A CareCompass weboldalának elsődleges célja, hogy felhasználóinkat hatékonyan összekösse a kórházak és egészségügyi intézmények által nyújtott szolgáltatásokkal. Platformunk lehetőséget biztosít a látogatók számára, hogy egyszerűen regisztráljanak, majd ezt követően könnyedén böngészhessenek a különböző kórházak és egészségügyi szolgáltatók kínálatában. A felhasználók a rendszer segítségével időpontot foglalhatnak a számukra szükséges egészségügyi ellátásokra, legyen szó szakorvosi konzultációkról, diagnosztikai vizsgálatokról vagy egyéb egészségügyi szolgáltatásokról. Célunk, hogy gyorsabbá, átláthatóbbá és kényelmesebbé tegyük a betegellátáshoz való hozzáférést, miközben egy felhasználóbarát és megbízható online felületet biztosítunk minden érdeklődő számára.

Célközönség meghatározása

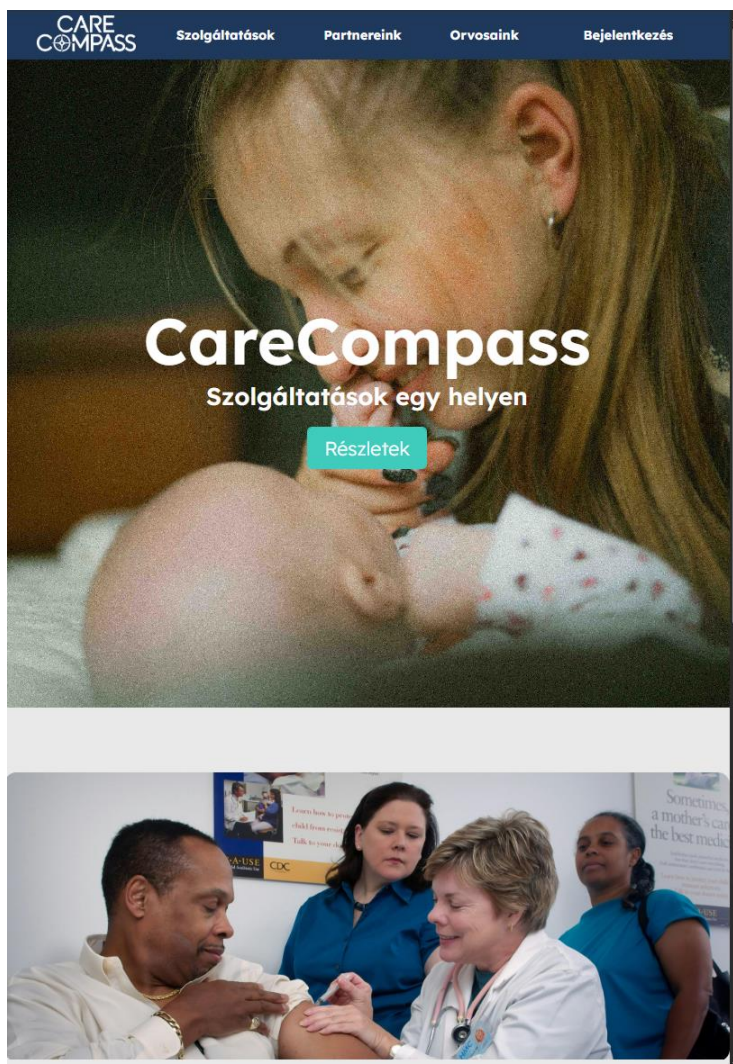
A CareCompass célközönsége elsősorban azok a felhasználók, akik gyorsan és kényelmesen szeretnék egészségügyi szolgáltatásokat igénybe venni Magyarországon található magánegészségügyi intézményekben.

Reszponzivitás

Az oldal teljes mértékben részponzív kialakítást kapott, így zökkenőmentes felhasználói élményt nyújt minden eszközön, legyen az mobiltelefon, táblagép vagy asztali számítógép. A részponzív működés biztosításához az alábbi technológiákat és elveket alkalmaztuk:

- **Rugalmas rácsszerkezet (grid):** A layout a CSS Grid és/vagy Flexbox segítségével lett kialakítva, így dinamikusan alkalmazkodik a különböző képernyőméretekhez.
- **Media query-k:** A CSS-ben media query-k segítségével különböző stílusokat definiáltunk a kis, közepes és nagy felbontású eszközökre.
- **Mobilbarát navigáció:** A menü mobil nézetben hamburger menüvé alakul, biztosítva a könnyű kezelhetőséget.
- **Reszponzív képek:** A képek automatikusan méreteződnek.
- **Touch-barát elemek:** Interaktív elemek (gombok, linkek) megfelelő méretűek és elhelyezkedésűek, hogy érintőképernyős eszközökön is könnyen használhatók legyenek.

Az oldal tesztelve lett különböző böngészőkön és eszközökön, hogy biztosítsuk a konzisztens megjelenést és működést.



Regisztráció

Név

Email

Jelszó

Telefonszám

TAJ szám

Lakcím

Nem

Születési dátum:

Rendelkezik TB-vel?

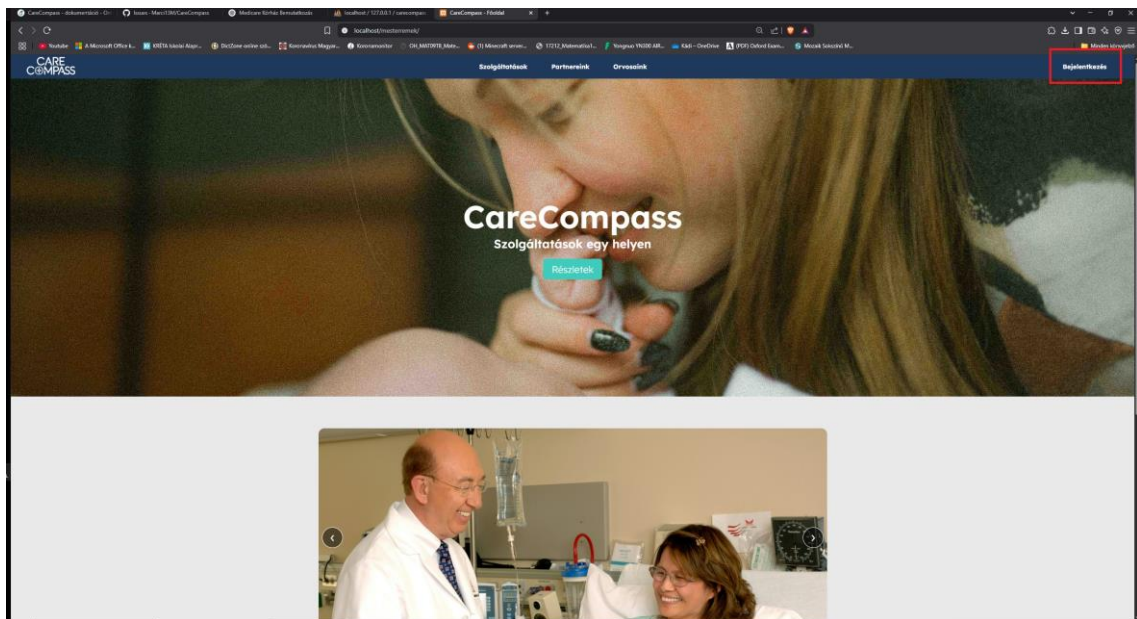
[Vissza a főoldalra](#)

Üdvözljük újra!

Jelentkezzen be adataival, hogy minden funkcionálitáshoz hozzáférést kapjon

5.2.2 Kezdeti lépések

Regisztráció



Az oldalunkra való regisztráció gyors és egyszerű. A főoldal jobb felső sarkában található "Bejelentkezés" gombra kattintva megjelenik a bejelentkezési felület. Itt, a jobb oldalon található "Regisztráció" gombra kattintva elérhetővé válik a regisztrációs űrlap, ahol néhány alapvető adat megadása után a felhasználók könnyedén létrehozhatják fiókjukat. A regisztráció során a felhasználóknak ki kell tölteniük az űrlapon megadott adatmezőket. Ennek megkönnyítésére egy beépített segítség áll rendelkezésre: amint a felhasználó egy beviteli mezőbe kattint, az oldal automatikusan megjeleníti alatta az adott mezőhöz tartozó formátumi követelményeket. Ez biztosítja, hogy a szükséges információkat megfelelő módon adják meg, elkerülve a hibás kitöltést. Található itt még egy "vissza a főoldalra" gomb, amely elnavigálja a felhasználót a főoldalra.

A regisztrációt követően minden felhasználónknak aktiválnia kell a fiókját. Ehhez egy megerősítő e-mailt küldünk a megadott e-mail címre, amely tartalmaz egy aktiválási linket. A fiók véglegesítéséhez a felhasználónak csak rá kell kattintania erre a linkre.

Bejelentkezés

Regisztráció

Gulyás Levente

ccgulyaslevente@gmail.com

0615486578

TAJ szám

2162 Örbottyán, Petőfi u. 32

Férfi

Születési dátum: 06/20/1997

Rendelkezik TB-vel? Nem

REGISZTRÁCIÓ

Vissza a főoldalra

Üdvözljük újra!

Jelentkezzen be adataival, hogy minden funkcióhoz hozzáférést kapjon

BEJELENTKEZÉS

Az oldalra való bejelentkezés nagyon hasonló, mint a regisztráció. Szintén az oldal jobb oldalán található "Bejelentkezés" gombra kell kattintani, majd ezután egyből a bejelentkezés felülete jelenik meg, melyhez a regisztrált email és jelszó páros szükséges. Található még itt egy "vissza a főoldalra" gomb, amely vissza navigálja a felhasználót a főoldalra.

Sikeres regisztráció!

Már csak 1 lépés maradt hátra. Kérjük erősítse meg email címét!

Megerősítés

Amennyiben Ön nem regisztrált, kérjük hagyja figyelmen kívül az emailt.

© CareCompass. Minden jog fenntartva.



Elfelejtett jelszó

Amennyiben elfelejtettük jelszavunkat, és emiatt nem tudunk belépni a weboldalra, e-mail címünk segítségével új jelszót igényelhetünk.

A folyamat elindításához egyszerűen kattintsunk a "Elfelejtette a jelszavát?" felíratra a bejelentkezési felületen. Ez egy új oldalra irányít minket, ahol meg kell adnunk a regisztrált e-mail címünket. Ezt követően kapunk egy e-mailt, amely tartalmaz egy "Új jelszó" gombot.

A gombra kattintva megnyílik egy oldal, ahol beállíthatjuk az új jelszavunkat. Ehhez mindössze kétszer meg kell adnunk az új jelszót, majd a "Jelszó megváltoztatása" gombbal véglegesíthetjük azt.

5.2.3 Alapvető funkciók

Főoldal

A weboldal főoldalán, a kezdőlap tetején található egy "Részletek" gomb. Ennek segítségével a felhasználók könnyedén hozzáférhetnek a különböző szolgáltatások részletes információihoz, ahol áttekinthetik a kínált lehetőségeket és további tudnivalókat. Ez alatt a szekció alatt található egy képnézegető, amely elsősorban szórakoztató célt szolgál. Ez a funkció lehetőséget ad a felhasználóknak, hogy a képek között böngészhessenek.

Navigáció

Az oldal tetején egy jól átlátható navigációs sáv található, amely segítségével a felhasználók könnyedén eligazodhatnak az oldalon.

- Bal oldalon helyezkedik el a CareCompass logó, amelyre kattintva a felhasználó bármelyik oldalról visszatérhet a főoldalra.
- Középen három fő menüpont kapott helyet, amelyek az alábbi tartalmakhoz vezetnek:
 - Szolgáltatások – Itt az összes szolgáltatás kerül felsorolásra, valamint egy "Összes" gomb is elérhető, amely a teljes "Szolgáltatások" oldalra irányítja a felhasználót.
 - Partnereink – Ebben a menüpontban található az az egészségügyi szolgáltatók, kórházak, amelyekkel a CareCompass együttműködik.

- Orvosaink – Itt az együttműködő intézményekhez tartozó orvosok bemutatkozása és szakmai leírása érhető el.

Ez a navigációs sáv biztosítja a gyors és egyszerű hozzáférést az oldal legfontosabb funkcióihoz.

Lábléc

Az oldal alján egy lábléc található, amely hasznos információkra mutató linkeket tartalmaz. A linkek oszlopba rendezve jelennek meg, hogy könnyen átláthatóak legyenek:



- CareCompass Kft. Ebben a szekcióban a CareCompass-szal kapcsolatos alapvető információk szerepelnek, például a címünk, e-mail címünk és telefonszámunk.
- Felhasználói fiók - Itt a felhasználók különböző fiókkezelési lehetőségeket érhetnek el, például átléphetnek a bejelentkezési oldalra, megtekinthetik adataikat egy külön oldalon, valamint hozzáférhetnek a foglalásaikhoz egy dedikált felületen.
- Információk - Ebben az oszlopban az oldal működésével és szabályzataival kapcsolatos lényeges tudnivalók találhatók. Ide tartozik az Általános Szerződési Feltételek, a "Rólunk" oldal, valamint az adatkezelési tájékoztató. Ezen kívül itt érhető el egy kapcsolati oldal, ahol a felhasználók közvetlenül üzenetet küldhetnek a CareCompass csapatának.

Ez a jól strukturált lábléc biztosítja, hogy a felhasználók könnyen megtalálják a számukra fontos információkat és gyorsan navigálhassanak az oldalon.

5.2.4 Haladó funkciók

Szolgáltatás foglalása orvos szerint

The screenshot displays the 'Foglalás áttekintése' (Appointment Overview) page. On the left, a sidebar contains the following information:

- Foglalás adatai**
 - Név: O Lajos
 - Email: o.lajos@gmail.com
 - Telefonszám: 06205896562
- Foglalás adatai**
 - Szolgáltatás: Gerincgyógyászat
 - Időpont: 2025-04-09 08:35
 - Orvos: Dr Teszt Elek
 - Helyszín: 1051 Budapest, Széchenyi tér 7
 - Ár: 95.000 Ft

At the bottom of the sidebar are two buttons: 'Mégse' (Cancel) and 'Foglalás megerősítése' (Confirm booking).

The main content area, titled 'Orvosaink' (Our Doctors), features a search bar and a list of doctors categorized by clinic:

- Medicover Zrt.**
 - Dr Csonka Milán (Hegtekintés)
 - Dr Nagy Lajos (Gerinc MR vizsgálat, Belső fül CT vizsgálat, Szülész-nőgyógyász vizsgálat, Hegtekintés)
 - Dr Szabó Gábor (Azonnali sürgős ellátás, Szív ultrahang, Hegtekintés)
 - Dr Teszt Elek (Gerinc MR vizsgálat, Nyugalmi EKG, Hegtekintés)
- Tritonlife Holding Zrt.**
 - Dr Csonka Milán (Hegtekintés)
 - Dr Kemény Géza (Urológiai vizsgálat, Hegtekintés)
 - Dr Kis István (Hegtekintés)
 - Dr Németh Patrik
 - Dr Teszt Elek

Az oldalon elérhető a sok adat megjelenítésénél egy szűrőszáv, ahol tudunk keresni a szövegben. Emelett helyet kapott egy "Szűrő" felirat is, amelyre kattintva megjelenik egy részletes szűrő és beállíthatjuk a kívánt paramétereket. A szűrt eredményeket a böngésző keresősávjában is elmenti a rendszer, így könnyen küldhetővé válnak a találatok.

The screenshot shows the 'Ágyéki gerinc MR vizsgálat' (Lumbar spine MR examination) page. It includes an illustration of a doctor and a patient in a consultation room. The text describes the procedure:

Az ágyéki vagy lumbális (lumbosacralis) gerincszakasz MR vizsgálatával meghatározhatók a gerincszakasz elváltozásai, a porckorong betegségei, a csontvelő és a csigolya környéki elváltozások. Segítségével ellenőrizhetők a mûtétek utáni hegek. MR vizsgálatot a lumbális szakaszon leggyakrabban porckorongsérves (discus hernia) és fejlődési rendellenességek gyanúja esetén végzünk.

Below the text, there is a search bar and a filter dropdown menu. The filter dropdown is open, showing options for 'Kórház' (Hospital) and 'Orvos' (Doctor). The search results show the following information:

- Szolgáltatás helyszíne:** 1134 Budapest, Váci út 29
- Intézmény:** Medicare Kórház és Diagnosztikai Központ
- Szolgáltatás ára:** 15.000 Ft

On the right, there is a calendar view titled 'Legkorábbi időpontok' (Earliest available times) showing a grid of dates and times for booking.

Miután a felhasználó kiválasztotta a számára megfelelő szolgáltatást, egy új oldalra kerül, ahol lehetősége van kiválasztani, hogy melyik orvos lássa el. Az orvos kiválasztása után megjelennek a szabad időpontok, amelyek közül kiválaszthatja a számára legmegfelelőbbet.

Ha az elsőként megjelenített 12 időpont között nem talál számára megfelelő lehetőséget, a "Továbbiak" gombra kattintva megtekintheti az adott orvos összes elérhető időpontját. Ez biztosítja, hogy a felhasználó a számára legideálisabb időpontot foglalhassa le.

Ezután megjelenik egy felugró ablak, melyben át tudjuk tekinteni foglalásunk fontos adatait, majd, ha mindent rendben találtunk meg is tudjuk foglalásunk erősíteni. Megerősítés után természetesen profilunk "adataim" menüpontjában még le tudjuk mondani foglalásunk.

Szolgáltatás foglalása kórház szerint

The screenshot shows the website of TritonLife Medical Center Hegyalja. The header includes navigation links: Szolgáltatások, Partnereink, and Orvosaink. Below the header is a banner image of the center's building and logo. The main content area is titled 'Bemutakozás' and contains a short description of the center's services. Below this is a table titled 'Nyitvatartás' (Opening Hours) showing the center's hours for each day of the week. Further down is a section titled 'Elérhetőségek' (Contact Information) with an email address. At the bottom, there is a section for 'Szolgáltatások' (Services) with a red box highlighting the 'Krónikus migrén kezelése' (Chronic migraine treatment) service, which has a 'Megtekintés' (View) button. Other services listed include 'Szív ultrahang' (Heart ultrasound), 'Szülész-nőgyógyász vizsgálat' (Gynecological examination), and 'Urológiai vizsgálat' (Urological examination), each with its own 'Megtekintés' button.

Nyitvatartás	
Hétfő	8:00 - 20:00
Kedd	8:00 - 20:00
Szerda	8:00 - 20:00
Csütörtök	8:00 - 20:00
Péntek	8:00 - 20:00
Szombat	8:00 - 12:00
Vasárnap	8:00 - 12:00

Elérhetőségek	
• ccinfo@robertkorhaz.hu	

Szolgáltatások	
Krónikus migrén kezelése	Megtekintés
Szív ultrahang	Megtekintés
Szülész-nőgyógyász vizsgálat	Megtekintés
Urológiai vizsgálat	Megtekintés

A Partnereink menüpontban a felhasználók megtalálhatják a különböző egészségügyi szolgáltató cégeket, köztük a kórházak listáját is. Miután kiválasztották a számukra megfelelő kórházat, egy új oldal nyílik meg, ahol a kórház részletes leírása és az ott elérhető szolgáltatások jelennek meg.

Ha a felhasználó itt kiválaszt egy adott szolgáltatást, akkor az előzőekben említett foglalási oldalra kerül. Innen a korábban ismertetett lépések követésével – orvos és időpont kiválasztásával – egyszerűen és gyorsan véglegesítheti foglalását.

Adataim - menüpont

Adataim	
Név	O Lajos
Jelszó	Jelszó megváltoztatása
Email	o.lajos@gmail.com
Telefonszám	06205896562
Lakcím	2133 Sződliget, Határ út 1
Születési dátum	1999-06-10
Nem	Férfi
TAJ szám	122203352
Társadalombiztosítás	Igen
Szerkesztés	

Foglalásaim		
Legutóbbi foglalás ideje 2025-04-08 21:44		
Következő időpontom 2025-04-09 08:35		
Gerinc MR vizsgálat	2025-04-25 10:00:00	Megtekintés
Dr Nagy Lajos		
Gerinc MR vizsgálat	2025-04-18 09:00:00	Megtekintés
Dr Nagy Lajos		
Gerincgyógyászat	2025-04-09 09:00:00	Megtekintés
Dr Tóth Lajos		
Gerincgyógyászat	2025-04-09 08:35:00	Megtekintés
Dr Teszt Elek		
Gerinc MR vizsgálat	2025-04-07 08:00:00	Megtekintés
Dr Teszt Elek		

Bejelentkezés után az oldal jobb felső sarkában, ahol korábban a "Bejelentkezés" gomb volt, most a felhasználó neve jelenik meg. Ha az egérkurzort erre a névre visszük, egy legördülő menü jelenik meg, amely két lehetőséget kínál:

- "Adataim" – Erre kattintva egy új oldalra kerülünk, ahol lehetőség van:
 - Foglalásaink megtekintésére és lemondására
 - Felhasználói adataink módosítására, beleértve:
 - * Nevünk
 - * Telefonszámunk
 - * Lakcímünk
 - * Nemünk
 - * TB jogviszonyunkkal kapcsolatos rendelkezés
 - * Születési dátumunk
 - * TAJ-számunk
- "Kijelentkezés" – Ezzel a lehetőséggel a felhasználó kiléphet fiókjából, és visszatérhet az oldal bejelentkezés előtti állapotába.

5.3 Asztali alkalmazás felhasználói dokumentáció

5.3.1 Bevezetés

Rövid ismertetés a CareCompass asztali alkalmazásról

A CareCompass asztali alkalmazás célja, hogy összekapcsolja a szolgáltatókat a felhasználókkal. Az alkalmazás lehetőséget biztosít a szolgáltatások hozzáadására, módosítására és törlésére, továbbá ezek a szolgáltatások cégekhez, kórházakhoz és orvosokhoz rendelhetők. Az orvosok számára elérhető funkció, hogy megtekinthetik a hozzájuk foglalt időpontokat. Az alkalmazás különböző felhasználói jogosultságokat biztosít, így elérhető adminisztrátoroknak, céges felhasználóknak, kórházi felhasználóknak és orvosoknak.

5.3.2 Kezdeti lépések

5.3.2.1 Rendszerkövetelmények

Minimális rendszerkövetelmények

- Operációs rendszer: Windows 10
- Processzor: Intel i3 2.0 GHz vagy jobb
- Memória: 4 GB RAM
- Tárhely: 500 MB szabad hely
- Képernyőfelbontás: 1280x720
- Szoftver előfeltételek: .NET Framework 4.7.2 vagy újabb
- Internetkapcsolat: Szükséges

Ajánlott rendszerkövetelmények

- Operációs rendszer: Windows 10
- Processzor: Intel i5 3.0 GHz vagy jobb
- Memória: 8 GB RAM
- Tárhely: 1 GB szabad hely
- Képernyőfelbontás: 1920x1080
- Szoftver előfeltételek: .NET Framework 4.7.2 vagy újabb
- Internetkapcsolat: Szükséges

Az alkalmazás telepítése előtt néhány előzetes lépést kell elvégezni. Az alábbiakban részletesen bemutatjuk a telepítés menetét:

5.3.3 Haladó funkciók

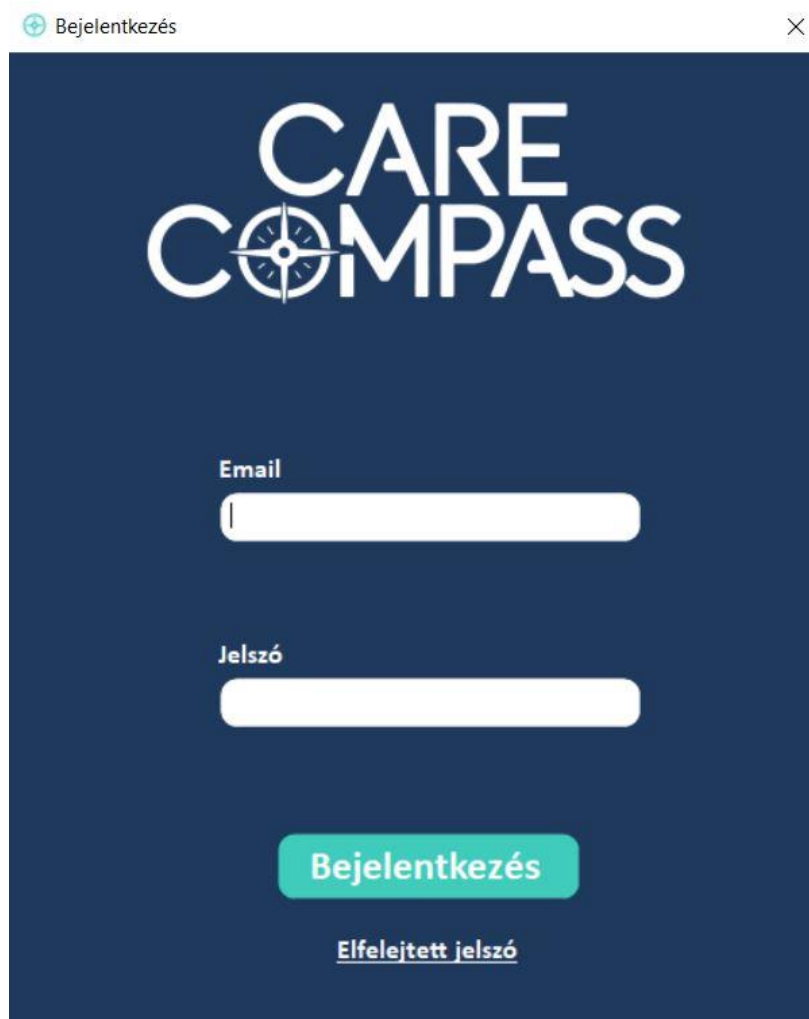
5.3.3.1 Bejelentkezés

Amint a felhasználó elindítja a programot, be kell jelentkeznie profiljával. Itt lehetősége van új jelszót igényelni is az „elfelejtett jelszó” gomb segítségével. Ezen gomb megnyomása után



The screenshot shows a web form titled 'Új jelszó igénylése' (New password request) under the 'CARE COMPASS' logo. Below the title, it says 'Adja meg email címét:' (Enter your email address:). There is a text input field containing 'pelda@pelda.hu'. Below the input field is a green button labeled 'Igénylés' (Request). At the bottom, there is a link that says 'Vissza a főoldalra' (Back to the main page).

megnyílik egy ablak, ahol beírhatja a felhasználó azt az email címet melyhez tartozó jelszavát elfelejtette. Az „igénylés” gomb megnyomása után kap egy email a felhasználó, melyben található egy link, ahol új jelszót tud beállítani.



The screenshot shows a login window titled 'Bejelentkezés' (Login) with a dark blue background. At the top is the 'CARE COMPASS' logo. Below the logo are two white input fields: the first is labeled 'Email' and the second is labeled 'Jelszó' (Password). Below the input fields is a green button labeled 'Bejelentkezés'. At the bottom, there is a link labeled 'Elfelejtett jelszó' (Forgot password).

5.3.3.2 Orvos felhasználói felület

Az alkalmazásban megtalálható menüpontok és azok leírása.

Profil

Care Compass - Dr Nagy Lajos

CARE COMPASS

- Profil
- Naptár
- Statisztika

Theme ☐

Kijelentkezés

Dr Nagy Lajos módosítások



Kép módosítása

Születésnap

1980.05.12

Látható ☒

Nyelv	Törlés
Angol	Törlés
Orosz	Törlés

Hozzáadás

Szakterület	Törlés
Belső fül CT vizsgálat	Törlés
Gerinc MR vizsgálat	Törlés
Szülész-nőgyógyás...	Törlés

Hozzáadás

Dr Nagy Lajos profilja

Módosítások mentése

Bemutakozás

Több mint tíz éve dolgozom orvosként, és ezalatt az idő alatt számos tapasztalatot szereztem a betegek ellátásában. Munkám során mindig a figyelmes, empátikus hozzáállást tartom szem előtt. Fontosnak tartom, hogy a pácienseim biztonságban érezzék magukat, és minden kérdésükre választ kapjanak. Rendszeresen részt veszek szakmai továbbképzéseken, hogy naprakész tudással segíthessem a hozzám fordulókat. Célom, hogy minden beteg egyénre szabott, átgondolt ellátásban részesüljön.

Végzettség	Törlés
Semmelweis Orvostudományi Egyet...	Törlés

Hozzáadás

Telefonszám	Látható	Törlés
0619877655	☒	Törlés

Hozzáadás

Ezen az oldalon lehetősége van az orvosnak módosítani az adatait, melyek a profilján megjelennek. Ilyen például a beszélt nyelvek, profilkép, bemutatkozás, szakterület, végzettség és telefonszám. Ezen módosítások pontos folyamata:

- Profilkép módosítás
 - A profilkép módosításhoz meg kell nyomnunk a "kép módosítása" gombot, majd a felugró fájlkezelő ablakban ki kell választanunk a kívánt profilképet.
- Születésnap és annak láthatóságának beállítása
 - A születésnap láthatóságának beállításához mindössze, csak annyit kell tennünk, hogy születési dátum mellett "Látható" felirat található gombot megnyomjuk egyszer.

A születésnap változtatásához a meglévő születésnapra való kattintás után lehet kiválasztani az új születésnapot.

 - Születésnap módosítás után meg kell nyomnunk a "módosítás mentése" gombot, hogy elmentsük azt.

- Bemutakozás módosítása/megírása
 - A bemutatkozás felirat alatt található, szöveg mezőbe tudjuk beírni a bemutatkozást, illetve, ha még nem adtunk meg bemutatkozást akkor itt tudjuk megírni.
 - Bemutakozás módosítás után meg kell nyomnunk a "módosítás mentése" gombot, hogy elmentsük azt.
- Beszélt nyelv hozzáadása, törlése
 - A nyelv felirat mellett található üres beviteli mezőbe kattintva tudjuk kiválasztani a programban található nyelvek közül a számunkra megfelelőt. A nyelv törléséhez csak egyszerűen az adott nyelv mellett található "Törlés" gombot kell lenyomnunk.
- Végzettség hozzáadása, törlése
 - Az oktatás felirat mellett található üres beviteli mezőbe kattintva tudjuk kiválasztani a programban található végzettségek közül a számunkra megfelelőt. A végzettség törléséhez csak egyszerűen az adott végzettség mellett található "Törlés" gombot kell lenyomnunk.
- Szakterület hozzáadása, törlése
 - A szakterület felirat mellett található üres beviteli mezőbe kattintva tudjuk kiválasztani a programban található szolgáltatások közül a számunkra megfelelőt. A szakterület törléséhez csak egyszerűen az adott szakterület mellett található "Törlés" gombot kell lenyomnunk.
- Telefonszám hozzáadása, törlése
 - A telefonszám felirat mellett található üres beviteli mezőbe kell beírnunk a telefonszámot, majd a "Hozzáadás" gomb segítségével felvinni azt a rendszerbe.
- Módosítás mentése gomb

Ezt akkor nyomjuk meg ha változtatás történt a születésnapnál vagy a bemutatkozásnál változtatás történt mivel ez a gomb mind a kettő adatot menti egyszerre.

Található meg a programban a felső sorban egy "Név módosítások, pl.: Dr Teszt Elek módosítások. Ebben a menüpontban jelszót tudunk változtatni, miután megadtuk egyszer a régi jelszavunk majd kétszer az új jelszót.

Naptár



Ebben a menüpont tudja az orvos megtekinteni a naptárát, melyet a program a böngészőben nyit meg. Itt tudja ellenőrizni az orvos a hozzá foglalt időpontokat, illetve itt tud szabadságot beírni és törölni. Mindezek mellett itt tudja az orvos megadni és törölni az adott szolgáltatáshoz a munkaidejét. Ennek a módja a következő:

1. Nyissuk meg a naptárat az alkalmazásból.

- A böngészőben lévő ablakban a jobb oldalon található szolgáltatások melletti plusz gomb segítségével tudunk megadni munkaidőt, a megfelelő adatok felvételével.

- Hasonlóan a szabadság felirat melletti plusz gombbal tudjuk beállítani szabadságunk.

- Egy adott szolgáltatás már beírt időpontját akkor lehet törölni, ha még nincs időpont foglalva ahhoz a szolgáltatáshoz. Ha nincs foglalás akkor rákattintásnál a felugró ablakon ahol a munkaidő adatai szerepelnek lehet törölni a munkaidőt.

Munkaidő áttekintése

Kórház
Medicare Szülészeti

Szolgáltatás Szülész-nőgyógyász vizsgálat	Dátum 2025-04-24
Kezdés 08:00:00	Végzés 16:00:00

Bezár
Munkaidő törlése

- Lehetősége van az orvosnak a hozzá foglalt időpontot is megtekinteni. A foglalt időpont egy fehér csíkként jelenik meg az adott napon, munkaidőben.

Időpont áttekintése

Név
O Lajos

Szolgáltatás Gerinc MR vizsgálat	Dátum 2025-04-25
Kezdet 10:00	Vég 10:20

Kórház
Medicare Kórház és Diagnosztikai Központ

Bezár
Időpont lezárása

- A szabadság törlése akkor lehetséges, ha a szabadság még nem kezdődött. Ha még nem kezdődött el a szabadság akkor rákattintásnál a felugró ablakon lehet törölni a szabadságot.

Statisztika



Ebben a menüpontban az orvoshoz tartozó statisztikák érhetőek el. Itt a bal oldalon beállíthatunk egyedi szűrést, de lehetőség van alapvetően napi, heti, havi statisztikák kiírására.

5.3.3.3 Kórház moderátor felhasználó műveletek

Profil

Ezen az oldalon tudjuk a kórház adatait módosítani, hozzáadni, mint például profilkép, borítókép, cím, bemutatkozás, nyitvatartás, telefonszám.

- Profilkép hozzáadás/módosítás

- A profilkép módosításhoz meg kell nyomnunk a "kép módosítása" gombot, majd a felugró fájlkezelő ablakban ki kell választanunk a kívánt profilképet.
- Cím módosítás
 - A cím módosításához be kell írunk a "Cím" felirat alatti szöveges mezőbe a megfelelő adatokat.
 - Cég módosítás után meg kell nyomnunk a "módosítás mentése" gombot, hogy elmentsük azt.
- Bemutakozás hozzáadás/módosítás
 - A bemutatkozás módosításához be kell írunk a "Bemutakozás" felirat alatti szöveges mezőbe a bemutatkozást vagy módosítani azt.
 - Bemutakozás módosítás után meg kell nyomnunk a "módosítás mentése" gombot, hogy elmentsük azt.
- Borítókép hozzáadás/módosítás
 - A borító kép módosításhoz meg kell nyomnunk a "borító módosítása" gombot, majd a felugró fájlkezelő ablakban ki kell választanunk a kívánt borító képet.
- Nyitvatartás hozzáadása/módosítása
 - A nyitvatartás hozzáadásához a "Nyitvatartás" felirat alatti mezőkbe napok szerint tudjuk megadni a nyitás és a zárás időpontját. Ennek formátuma például 7:00, amennyiben nem írunk be semmit, a program azt a napot zártnak tekinti.
 - Nyitvatartás módosítás után meg kell nyomnunk a "módosítás mentése" gombot, hogy elmentsük azt.
- Telefonszám hozzáadása, törlése
 - A telefonszám felirat mellett található üres beviteli mezőbe kell beírunk a telefonszámot, majd a "Hozzáadás" gomb segítségével felvinni azt a rendszerbe.
- Módosítás mentése gomb

Ezt akkor nyomjuk meg ha változtatás történt a Címnél, bemutatkozásnál vagy a nyitvatartásnál változtatás történt mivel ez a gomb az összezs adatot menti egyszerre.

Található meg a programban a felső sorban egy "Név módosítások, pl.: Dr Teszt Elek módosítások. Ebben a menüpontban jelszót tudunk változtatni, miután megadtuk egyszer a régi jelszavunk majd kétszer az új jelszót.

Alkalmazottak

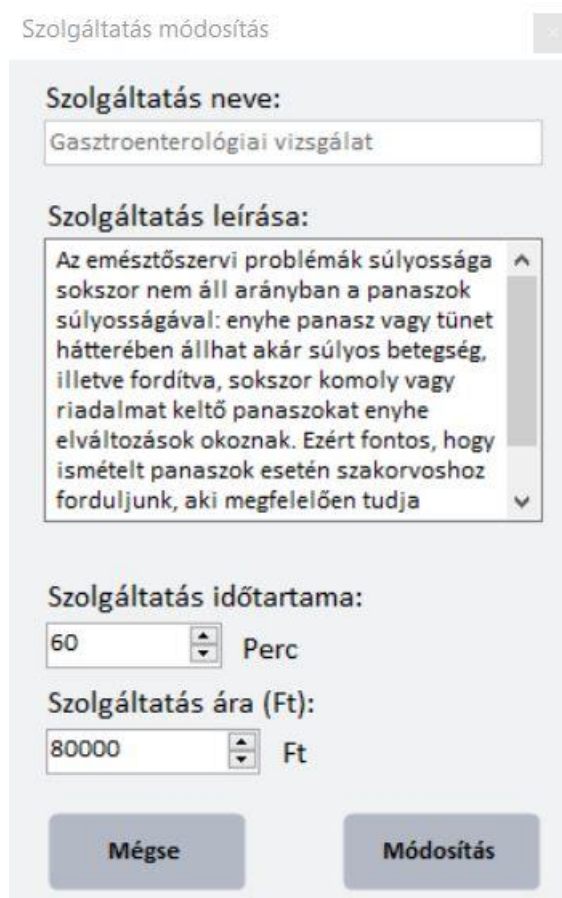
Ezen az oldalon lehetőségünk van ellenőrizni a kórházhoz tartozó alkalmazottakat, mind a moderátorokat, mind az orvosokat. Lehetőségünk van továbbá orvost hozzáadni, az oldal jobb oldalán található legördülő menüvel mely az "Orvos hozzáadása" felirat alatt található. Miután kiválasztottuk a kívánt orvost, egyszerűen csak kattintsunk rá, a "Hozzáadás" gombra. Ezután megjelenik a listában is az adott orvos.

Szolgáltatások

Itt van lehetőségünk kórházunkhoz szolgáltatást hozzárendelni, illetve már meglévőt módosítani. A szolgáltatás hozzáadásának a módja:

- Az oldal alján található az "Új szolgáltatás hozzáadása" felirat alatt tudjuk kiválasztani mely szolgáltatást szeretnénk hozzárendelni a kórházhoz.
- A szolgáltatás kiválasztása után meg tudjuk adni az adott szolgáltatás időtartamát, árát, és annak leírását majd a "Hozzáadás" gomb lenyomásával hozzá is rendeltük a szolgáltatást a kórházhoz.

Szolgáltatás módosításának módja:



Szolgáltatás módosítás

Szolgáltatás neve:
Gasztroenterológiai vizsgálat

Szolgáltatás leírása:
Az emésztőszervi problémák súlyossága sokszor nem áll arányban a panaszok súlyosságával: enyhe panasz vagy tünet háttérében állhat akár súlyos betegség, illetve fordítva, sokszor komoly vagy riadalmat keltő panaszokat enyhe elváltozások okoznak. Ezért fontos, hogy ismételt panaszok esetén szakorvoshoz forduljunk, aki megfelelően tudja

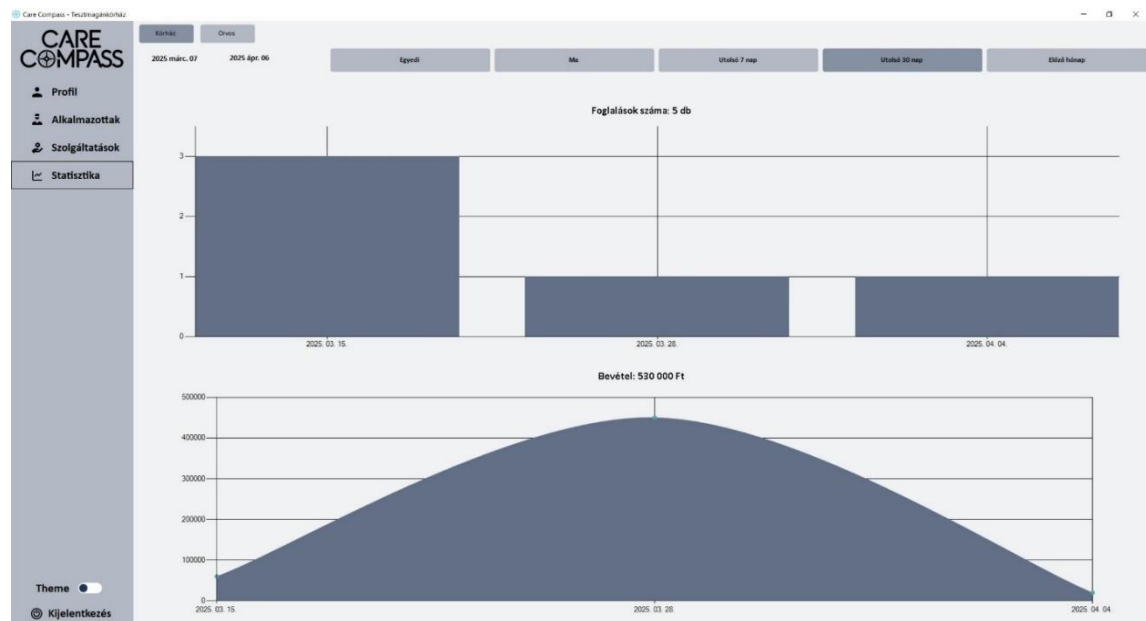
Szolgáltatás időtartama:
60 Perc

Szolgáltatás ára (Ft):
80000 Ft

Mégse Módosítás

- A felsorolásban az adott szolgáltatás jobb oldalán található "módosítás" gombra kell kattintanunk.
- Miután ezt megtettük felugrik egy ablak, ahol a szolgáltatáshoz tartozó bármely adatot módosítani tudjuk.
- Majd, ha végeztünk módosításainkkal, csak nyomjuk meg a "Módosítás" gombot a felugró ablakban. Amennyiben meggondoltuk magunkat és mégsem szeretnénk a módosításokat menteni, csak kattintsunk a "Mégse" gombra.

Statisztika



Ebben a menüpontban a kórházhoz tartozó statisztikák érhetőek el. Itt a bal oldalon beállíthatunk egyedi szűrést, de lehetőség van alapvetően napi, heti, havi statisztikák kiírására. Olyan statisztikák érhetünk el, mint például a foglалások száma, vagy épp a bevételünk mértéke. Lehetőségünk van még a bal felső sarokban egy legördülő menüben orvost kiválasztani, így megtekinthető az adott orvos statisztikája is.

5.3.3.4 Cég moderátor felhasználó műveletek

Profil

The screenshot displays the 'Profil' (Profile) section of the Care Compass application for user 'Boross Lajos'. The page shows the profile of 'Medicover Zrt.' with various fields for editing:

- Telefonszám**: A table with columns 'Telefonszám', 'Látható', and 'Törlés'. The first row shows the number '0614653131' with a checked 'Látható' box and a 'Törlés' button.
- Kórházak**: A table with columns 'Név' and 'Cím'. The first row shows 'Medicare Kórház és Diagnosztikai Központ' with the address '1134 Budapest, '.
- Cím**: Fields for 'Irányítószám' (1134), 'Település' (Budapest), 'Utca' (Váci út), and 'Házzszám' (29).
- Cégjegyzékszám**: Field with value '01-10-042382'.
- Email**: Field with value 'cckapcsolat@medicare-gro'.

The 'Módosítások mentése' (Save changes) button is located at the bottom right. The left sidebar contains navigation links: Profil, Kórház, Alkalmazottak, Szolgáltatások, and Statisztika. The top bar shows filters for 'Kórház' and 'Orvos'.

Ezen az oldalon tudjuk a cég adatait módosítani, hozzáadni, mint például profilkép, cím, telefonszám.

- Profilkép hozzáadás/módosítás
 - A profilkép módosításhoz meg kell nyomnunk a "kép módosítása" gombot, majd a felugró fájlkezelő ablakban ki kell választanunk a kívánt profilképet.
- Cím módosítás
 - A cím módosításához be kell írunk a "Cím" felirat alatti szöveges mezőbe a megfelelő adatokat.
 - Cím módosítás után meg kell nyomnunk a "módosítás mentése" gombot, hogy elmentsük azt.
- Cég adatok módosítása.
 - Amelyik adatot módosítani szeretnénk az adott adat neve alatt lévő szöveges mezőbe tudjuk azt beírni.
 - Cég adatok módosítás után meg kell nyomnunk a "módosítás mentése" gombot, hogy elmentsük azt.
- Telefonszám hozzáadása, törlése
 - A telefonszám felirat mellett található üres beviteli mezőbe kell beírunk a telefonszámot, majd a "Hozzáadás" gomb segítségével felvinni azt a rendszerbe.
- Meg tudjuk tekinteni ezen az oldalon még a céghez tartozó kórházakat.
- Módosítás mentése gomb

Ezt akkor nyomjuk meg ha változtatás történt a címnél vagy a cég adatoknál változtatás történt mivel ez a gomb mind az összes adatot menti egyszerre.

Található meg a programban a felső sorban egy "Név módosítások, pl.: Dr Teszt Elek módosítások. Ebben a menüpontban jelszót tudunk változtatni, miután megadtuk egyszer a régi jelszavunk majd kétszer az új jelszót.

Kórház

Care Compass - Boross Lajos

CARE COMPASS

Profil

Kórház

Alkalmazottak

Szolgáltatások

Statisztika

Theme

Kijelentkezés

Kórházak

Név	Cím	Email	Aktív	Felület	Törlés
Medicare Kórház és Diagnosztikai Központ	1134 Budapest, Váci út 29	cckapcsolat@medicare-group.hu	☒	Belépés	Törlés
Medicare Ügyeleti Centrum (Urgent Care Center)	1134 Budapest, Váci út 29	cckapcsolat@medicare-group.hu	☒	Belépés	Törlés
Medicare Szülészeti	1134 Budapest, Váci út 29	cckapcsolat@medicare-group.hu	☒	Belépés	Törlés

Új kórház hozzáadása

Kórház neve:

Kórház címe:

Irányítószám

Település

Utca

Házzszám

Kórház Email címe:

Hozzáadás

Itt tudunk új kórházat felvenni a céghez, illetve megtekinteni az jelenleg cégünkhöz tartozó kórházakat.

- **Új kórház hozzáadása**

- Az oldal jobb oldalán található "Új kórház hozzáadása" felirat alatt, a megfelelő adatok megadásával majd a "Hozzáadás" gomb lenyomásával lehetőségünk van új kórházat felvenni cégünkbe.
- Ezen a felületen a felsorolt kórházak közt jobb oldalon található két gomb. Az "Aktív" jelölőgomb segítségével deaktiválni tudjuk a céget anélkül, hogy törölnénk azt. A "Belépés" gomb segítségével lehetőségünk van belépni az adott cég felületébe. Végül a "Törlés" gomb segítségével törölni tudjuk a kórházat a cégünkből.

Alkalmazottak

Care Compass - Boross Lajos

CARE COMPASS

- Profil
- Kórház
- Alkalmazottak**
- Szolgáltatások
- Statistika

Theme ☒

Kijelentkezés

Alkalmazottak

Orvosok

Név	Email	Törlés
Dr Teszt Elek	teszt.elek@gmail.com	Törlés
Dr Nagy Lajos	ccnagylajos@gmail.com	Törlés
Dr Szabó Gábor	ccszabogabor@gmail.com	Törlés
Dr Csonka Milán	cccsonkamilan@gmail.com	Törlés

Moderátorok

Név	Email	Hatáskör	Intézmény	Törlés
Boross Lajos	cccegmoderator@gmail.com	Cég	Medicover Zrt.	

Új alkalmazott hozzáadása

Alkalmazott munkaterülete:

Hozzáadás

Ezen az oldalon tudjuk ellenőrizni a céghez tartozó alkalmazottakat, illetve hozzáadni, törölni alkalmazottakat.

- Új alkalmazott hozzáadása
 - Az oldal jobb oldalán lévő "Új alkalmazott hozzáadása" felirat alatt található legördülő menüből ki kell választanunk az alkalmazott munkaterületét, majd az alatta lévő mezőbe be kell írunk a felhasználó email címét.
 - Ezután egyszerűen a "Hozzáadás" gomb segítségével fel tudjuk vinni az alkalmazottat a cégünkbe.
- Alkalmazott törlése
 - Két alkalmazott fajta törlése van lehetőségünk. A felső felsorolásban az orvosok közül tudunk törölni a jobb oldalon található "Törlés" gomb segítségével. Az alsó felsorolásban céghez tartozó kórház moderátorok törlésére van lehetőségünk, ugyanúgy, mint az orvosok esetében.

Szolgáltatások

Care Compass - Boross Lajos

CARE COMPASS

Profil

Kórház

Alkalmazottak

Szolgáltatások

Statistika

Theme

Kijelentkezés

Szolgáltatások

Név	Leírás	Időtartam	Ár	Módosítás	Törlés
Gerinc MR vizsgálat	Az ágyéki vagy lumbális (lumbosacralis) gerincszakasz MR vizsgálatával ...	20	15000	Módosítás	Törlés
Belső fül CT vizsgálat	Ezzel a CT vizsgálattal a belső fül finom csontszerkezetét és a hallójárató...	40	25000	Módosítás	Törlés
Azonnali sürgős ellátás	Akár már enyhe tünetek is komoly betegséget jelezhetnek. Ha időben or...	60	50000	Módosítás	Törlés
Szülész-nőgyógyász vizsgálat	Lehetőség szerint a 12. várandóssági hétig javasolt rá sort keríteni. Ez a vi...	120	85000	Módosítás	Törlés
Nyugalmi EKG	A nyugalmi EKG vizsgálat semmiféle előkészületet nem igényel a pácién...	40	50000	Módosítás	Törlés
Szív ultrahang	Szívünk mesterien kivitelezett pumpa, melyet apró elektromos automati...	5	36000	Módosítás	Törlés
Proktológia	A proktológia az általános sebészet speciális szakterülete, mely a végbél...	5	40000	Módosítás	Törlés

Új szolgáltatás hozzáadása

Gerincgyógyászat

Időtartam

5

Perc

Szolgáltatás ára

0

Ft

Szolgáltatás leírása

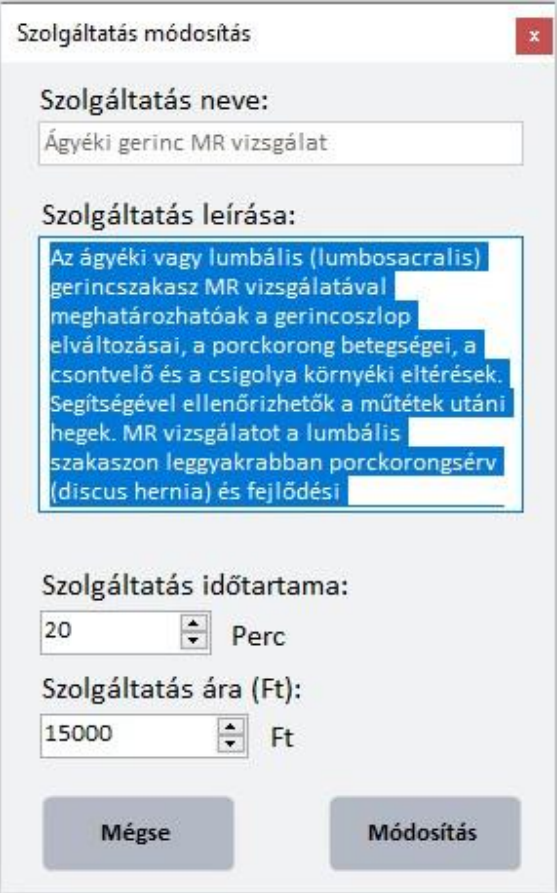
Az egyik leggyakoribb mozgásszervi panasz, amellyel reumatológushoz, ortopéd szakorvoshoz, vagy gerincgyógyászhoz (újabbban alternatív medicinához) fordulnak hazánkban a derékfájdalom, illetve a nyaki vagy háti panaszok, időnként végtagfájdalommal párosulva. A magyar lakosság majdnem 80 százaléka szenved életében legalább egyszer valamilyen gerinctáji fájdalomtól. Ezen a tüneteken túl a gerincoszlop egyéb kóros elváltozásai, komplex mozgásszervi eltérések, deformitások, korrekciók és a vértápláltsági problémák is a gerincgyógyászati ellátási körhöz tartoznak.

Hozzáadás

Itt van lehetőségünk a cégünkhöz szolgáltatást hozzárendelni, illetve már meglévőt módosítani. A szolgáltatás hozzáadásának a módja:

1. A szolgáltatások menüpontra kattintva, megjelenik egy oldal, ahol fel vannak sorolva az eddig hozzáadott szolgáltatások.
2. Ezen az oldalon az oldal alján található az "Új szolgáltatás hozzáadása" felirat alatt tudjuk kiválasztani mely szolgáltatást szeretnénk hozzárendelni a céghez.
3. A szolgáltatás kiválasztása után meg tudjuk adni az adott szolgáltatás időtartamát, árát, és annak leírását majd a "Hozzáadás" gomb lenyomásával hozzá is rendeltük a szolgáltatást a céghez.

Szolgáltatás módosításának módja:



Szolgáltatás módosítás

Szolgáltatás neve:
Ágyéki gerinc MR vizsgálat

Szolgáltatás leírása:
Az ágyéki vagy lumbális (lumbosacralis) gerincszakasz MR vizsgálatával meghatározhatóak a gerincoszlop elváltozásai, a porckorong betegségei, a csontvelő és a csigolya környéki eltérések. Segítségével ellenőrizhetők a műtétek utáni hegek. MR vizsgálatot a lumbális szakaszon leggyakrabban porckorongsérv (discus hernia) és fejlődési

Szolgáltatás időtartama:
20 Perc

Szolgáltatás ára (Ft):
15000 Ft

Mégse Módosítás

- A felsorolásban az adott szolgáltatás jobb oldalán található "módosítás" gombra kell kattintanunk.
- Miután ezt megtettük felugrik egy ablak, ahol a szolgáltatáshoz tartozó bármely adatot módosítani tudjuk.
- Majd, ha végeztünk módosításainkkal, csak nyomjuk meg a "Módosítás" gombot a felugró ablakban. Amennyiben meggondoltuk magunkat és mégsem szereténk a módosításokat menteni, csak kattintsunk a "Mégse" gombra.

Statisztika



Ebben a menüpontban a céghez tartozó statisztikák érhetőek el. Itt a bal oldalon beállíthatunk egyedi szűrést, de lehetőség van alapvetően napi, heti, havi statisztikák kiírására. Olyan statisztikák érhetünk el, mint például a foglалások száma, vagy épp a bevételünk mértéke. Lehetőségünk van még a bal felső sarokban egy legördülő menüben orvost vagy épp kórházat kiválasztani, így megtekinthető az adott orvos vagy kórház statisztikája is.

5.3.3.5 Admin felhasználói műveletek

Felhasználók

Moderátorok

Név	Email	Beosztás	Intézmény	Törítés
Boross Lajos	cccegmoderator@gmail.com	Cég	Medicover Zrt.	Törítés
Seres Lajos	cckorhazmoderator@gmail.com	Kórház	TritonLife Róbert Magánkórház	Törítés

Új alkalmazott hozzáadása

Alkalmazott email címe:

Alkalmazott munkaterülete:

Orvosok

Név	Email	Felület	Törítés
Dr Csonka Milán	ccsonkamilan@gmail.com	Belépés	Törítés
Dr Kemény Géza	cckemenygeza@gmail.com	Belépés	Törítés
Dr Kiss István	cckissistvan@gmail.com	Belépés	Törítés
Dr Nagy Lajos	ccnagyajos@gmail.com	Belépés	Törítés
Dr Németh Patrik	ccnemethpatrik@gmail.com	Belépés	Törítés
Dr Szabó Gábor	ccszabogabor@gmail.com	Belépés	Törítés
Dr Teszt Elek	teszt.elek@gmail.com	Belépés	Törítés
Dr Tóth Lajos	cctohtlajos@gmail.com	Belépés	Törítés

Admin

Név	Email	Törítés
Bábi Levente	babilevi13@gmail.com	Törítés
Admin Lajos	ccadmin@gmail.com	Törítés

Ezen az oldalon van lehetőségünk új felhasználót hozzáadni adott munkaterülethez, illetve megtekinteni a moderátor/orvos felhasználókat.

- Új felhasználó hozzáadása
 - Az oldal jobb oldalán lévő "Új alkalmazott hozzáadása" felirat alatt található legördölő menüből ki kell választanunk az alkalmazott munkaterületét, majd az alatta lévő mezőbe be kell írunk a felhasználó email címét.
 - Ezután egyszerűen a "Hozzáadás" gomb segítségével fel tudjuk vinni az alkalmazottat a rendszerbe.
- Alkalmazott törlése
 - Három alkalmazott fajta törlése van lehetőségünk. A felső felsorolásban a moderátorok közül tudunk törölni a jobb oldalon található "Törlés" gomb segítségével. Az alsó bal felsorolásban cég orvosok törlésére van lehetőségünk, ugyanúgy, mint a moderátorok esetében. Végül az admin felhasználókat az eddigi módon a jobb alsó sarokban található felsorolásból tudjuk törölni.
- Orvos felületre belépés
 - Az orvos táblázatban a "Belépés" gomb segítségével lehetőségünk van belépni az adott orvos felületére ahol adatmódosításra van lehetőség.

Található meg a programban a felső sorban egy "Név módosítások, pl.: Dr Teszt Elek módosítások. Ebben a menüpontban jelszót tudunk változtatni, miután megadtuk egyszer a régi jelszavunk majd kétszer az új jelszót.

Web felhasználók

Itt tudjuk ellenőrizni azon felhasználók listáját, akik a weben regisztráltak. Továbbá lehetőségünk van módosítani ezen felhasználók adatait, illetve törölni ezen felhasználók közül.

- Felhasználók ellenőrzése
 - Az oldalon található listában minden webes felhasználó megtalálható.

Care Compass - Admin Lajos

CARE COMPASS

Felhasználók

Web felhasználók

Cég

Kórház

Szolgáltatások

Egyéb adatok

Statistika

Theme ☐

Kijelentkezés

Web felhasználó módosítás és törlése

Felhasználónév	Email	Születésnap	Módosítás	Törlés
O Lajos	o.lajos@gmail.com	1999-06-10	Módosítás	Törlés
Nagy Sándor	szlonkaim@gmail.com	2000-04-12	Módosítás	Törlés

- Felhasználók módosítása

- A felsorolt felhasználók jobb oldalán található egy "Módosítás" gomb. Ezt megnyomva megnyílik egy új ablak melyben módosítani tudjuk a felhasználók által megadott összes webes adatot. Köztük a nevet, emailt, nemet, telefonszámot, taj számot, címet, születésnapot, TB rendelkezést.
- Módosítás után csak rá kell kattintanunk a jobb alsó sarokban található "Módosítás" gombra. Amennyiben mégsem szeretnénk módosítani az adatokat, a bal oldalon található "Mégsem" gombbal el tudjuk vetni a műveletet.

Webes felhasználó adatainak módosítása

Felhasználó neve:

O Lajos

Email:

o.lajos@gmail.com

Nem (gender):

Férfi

Telefonszám:

06205896562

Taj szám:

122203352

Cím:

Irányítószám:

2133

Település

Sződliiget

Utca:

Határ út

Házzszám:

1

Születésnap:

1999. június 10.,

Van-e TB (társadalom biztosítása)?

☒

Mégse

Módosítás

- Felhasználók törlése

- A felhasználó jobb oldalán található "Törlés" gomb lenyomásával van lehetőségünk törölni a kiválasztott felhasználót.

Cég

Care Compass - Admin Lajos

Cégek hozzáadása és törlése

Név	Email	Cím	Aktív	Felület	Törlés
Medicover Zrt.	cckapcsolat@medicare-group.hu	1134 Budapest, Váci út 29-31	☒	Belépés	Törlés
Tritonlife Holding Zrt.	ccinfo@tritonlife.hu	1134 Budapest, Róbert Károly körút. 64-66	☒	Belépés	Törlés
Emineo Kft.	ccemineo@emineo.hu	1016 Budapest, Hegyalja út 7	☒	Belépés	Törlés
Dc Rose Magánkórház Kft.	ccinfo@drrose.hu	1051 Budapest, Széchenyi tér 7-8	☒	Belépés	Törlés

Cég hozzáadása

Név:

Céglapjék szám:

Adószám:

Igazgató neve:

Email:

Irányítószám:

Település:

Utca:

Házszám:

Ezen az oldalon lehetőségünk van megnézni a cégeket, illetve hozzáadni cégeket, belépni a cégekbe.

- Cég hozzáadása

- Bejelentkezés után egyszerűen a bal oldalon található menü segítségével válasszuk ki a "Cég" menüpontot.
- Ezután az alkalmazás jobb oldalán a "Cég hozzáadása" felirat alatt a megfelelő adatok beírásával, majd a "Hozzáadás" gomb segítségével felvihetjük a céget a rendszerbe.
- Miután felvittük a céget, a cégek alatt megjelenik a felsorolásban.

- Cég funkciók

- Ezen a felületen a felsorolt cégek közt jobb oldalon található két gomb. Az "Aktív" jelölőgomb segítségével deaktiválni tudjuk a céget anélkül, hogy törölnénk azt. A "Belépés" gomb segítségével lehetőségünk van belépni az adott cég felületébe. Végül a "Törlés" gomb segítségével törölni tudjuk céget.

Kórház

Care Compass - Admin Lajos

CARE COMPASS

Felhasználók

Web felhasználók

Cég

Kórház

Szolgáltatások

Egyéb adatok

Statistika

Theme ☐

Kijelentkezés

Kórházak

Név	Email	Aktív	Felület	Törlés
Medicare Kórház és Diagnosztikai Központ	cckapcsolat@medicare-group.hu	☒	Belépés	Törlés
Medicare Ügyeleti Centrum (Urgent Care Center)	cckapcsolat@medicare-group.hu	☒	Belépés	Törlés
Medicare Szülészeti	cckapcsolat@medicare-group.hu	☒	Belépés	Törlés
TritonLife Róbert Magánkórház	ccinfo@robertkorhaz.hu	☒	Belépés	Törlés
TritonLife Medical Center Hegyalja	ccinfo@robertkorhaz.hu	☒	Belépés	Törlés
Emineo Magánkórház	ccemineo@emineo.hu	☒	Belépés	Törlés
Dr. Rose Magánkórház	ccinfo@drrose.hu	☒	Belépés	Törlés

Új kórház hozzáadása

Kórház neve:

Kórház címe:

Irányítószám:

Település:

Utca:

Házzszám:

Kórház Email címe:

Felügyelő cég:

Hozzáadás

Ezen az oldalon lehetőségünk van megnézni a kórházakat, illetve hozzáadni cégeket, belépni a kórházakba.

- Kórház hozzáadása
 - Bejelentkezés után egyszerűen a bal oldalon található menü segítségével válasszuk ki a "Kórház" menüpontot.
 - Ezután az alkalmazás jobb oldalán a "Új kórház hozzáadása" felirat alatt a megfelelő adatok beírásával, illetve a fenntartó cég kiválasztása után, a "Hozzáadás" gomb segítségével felvihetjük a kórházat a rendszerbe.
 - Miután felvittük a kórházat az megjelenik a felsorolásban.
- Kórház funkciók
 - Ezen a felületen a felsorolt kórházak közt jobb oldalon található két gomb. Az "Aktív" jelölőgomb segítségével deaktiválni tudjuk a céget anélkül, hogy törölnénk azt. A "Belépés" gomb segítségével lehetőségünk van belépni az adott kórház felületébe. Végül a "Törlés" gomb segítségével törölni tudjuk a kórházat.

Szolgáltatások

Ezen az oldalon tudunk új szolgáltatást hozzáadni a rendszerhez, illetve a meglévőket módosítani, törölni.

Care Compass - Admin Lajos

- Felhasználók
- Web felhasználók
- Cég
- Kórház
- Szolgáltatások**
- Egyéb adatok
- Statistika

Theme ☒

Kijelentkezés

Szolgáltatások

Név	Leírás	Módosítás	Törlés
Arcplasztika	Az arcplasztikai beavatkozások azok számára lehet ideális választás, akik fiat...	Módosítás	Törlés
Azonnali sürgős ellátás	Akár már enyhe tünetek is komoly betegséget jelezhetnek. Ha időben orvos...	Módosítás	Törlés
Belső fül CT vizsgálat	Ezzel a CT vizsgálattal a belső fül finom csontszerkezetét és a hallójáratokat l...	Módosítás	Törlés
Gastroenterológiai vizsgálat	Az emésztőszervi problémák súlyossága sokszor nem áll arányban a panasz...	Módosítás	Törlés
Gerinc MR vizsgálat	Az ágyéki vagy lumbális (lumbosacralis) gerincszakasz MR vizsgálatával meg...	Módosítás	Törlés
Gerincgyógyászat	Az egyik leggyakoribb mozgásszervi panasz, amellyel reumatológushoz, orto...	Módosítás	Törlés
Krónikus migrén kezelése	A krónikus migrén, a szem körüli vagy a nyakizmok akaratlan és ismétlődő iz...	Módosítás	Törlés
Nyugalmi EKG	A nyugalmi EKG vizsgálat semmiféle előkészületet nem igényel a páciens r...	Módosítás	Törlés
Proktológia	A proktológia az általános sebészet speciális szakterülete, mely a végbél be...	Módosítás	Törlés
Szív ultrahang	Szívünk mesterien kivitelezett pumpa, melyet apró elektromos automatika ...	Módosítás	Törlés
Szülés-nőgyógyász vizsgálat	Lehetőség szerint a 12. várandóssági héttig javasolt rá sort keríteni. Ez a vizsg...	Módosítás	Törlés
Testplasztika	Akár a kor, a várandósság vagy a testsúlyingadozás hagy nyomot a testen, az ...	Módosítás	Törlés
Urológiai vizsgálat	Az urológiai szakorvosi vizsgálat során lehetőség van a vese, húgyhólyag, hú...	Módosítás	Törlés

Szolgáltatás hozzáadása

Név:

Szolgáltatás leírása:

Hozzáadás

- Szolgáltatás hozzáadása
 - Bejelentkezés után egyszerűen a bal oldalon található menü segítségével válasszuk ki a "Szolgáltatások" menüpontot.
 - Ezután az alkalmazás jobb oldalán a "Szolgáltatás hozzáadása" felirat alatt a szolgáltatás nevének és leírásának beírásával, a "Hozzáadás" gomb segítségével felvihetjük a szolgáltatást a rendszerbe.
 - Miután felvittük a szolgáltatást az megjelenik a felsorolásban.
- Szolgáltatás módosítása

Szolgáltatás módosítása

Szolgáltatás neve:

Szolgáltatás leírása:

Akár már enyhe tünetek is komoly betegséget jelezhetnek. Ha időben orvoshoz fordulunk, a szakszerű segítség és a korai diagnosztika súlyos következményektől menthet meg. Enyhe mellkasi vagy hasi fájdalom, felső légúti tünetek, láz, felfázás, deréktáji panaszok, vagy akár háztartási balesetek esetén is forduljon hozzánk!

Mégse

Módosítás

- A felsorolásban az adott szolgáltatás jobb oldalán található "módosítás" gombra kell kattintanunk.
- Miután ezt megtettük felugrik egy ablak, ahol a szolgáltatáshoz tartozó bármely adatot módosítani tudjuk.

- Majd, ha végeztünk módosításainkkal, csak nyomjuk meg a "Módosítás" gombot a felugró ablakban. Amennyiben meggondoltuk magunkat és mégsem szereténk a módosításokat menteni, csak kattintsunk a "Mégse" gombra.
- Szolgáltatás törlése
 - A felsorolás jobb oldalán található "Törlés" gombra kell kattintanunk, ezzel tudjuk törölni a szolgáltatást a rendszerből.

Egyéb adatok

Care Compass - Admin Lajos

CARE COMPASS

- Felhasználók
- Web felhasználók
- Cég
- Kórház
- Szolgáltatások
- Egyéb adatok**
- Statisztika

Theme ☐

Kijelentkezés

Adatok feltöltése

Nyelv

Nyelv	Törlés
Angol	Törlés
Német	Törlés
Francia	Törlés
Orosz	Törlés
Ukrán	Törlés
Spanyol	Törlés
Arab	Törlés

Hozzáadás

Oktatás

Oktatás	Törlés
Semmelweis Orvostudományi Egyetem Általános Or...	Törlés
Debreceni Egyetem Általános Orvostudományi Kar Á...	Törlés
Pécsi Tudományegyetem Általános Orvostudományi ...	Törlés
Szegedi Tudományegyetem Szent-Györgyi Albert Or...	Törlés

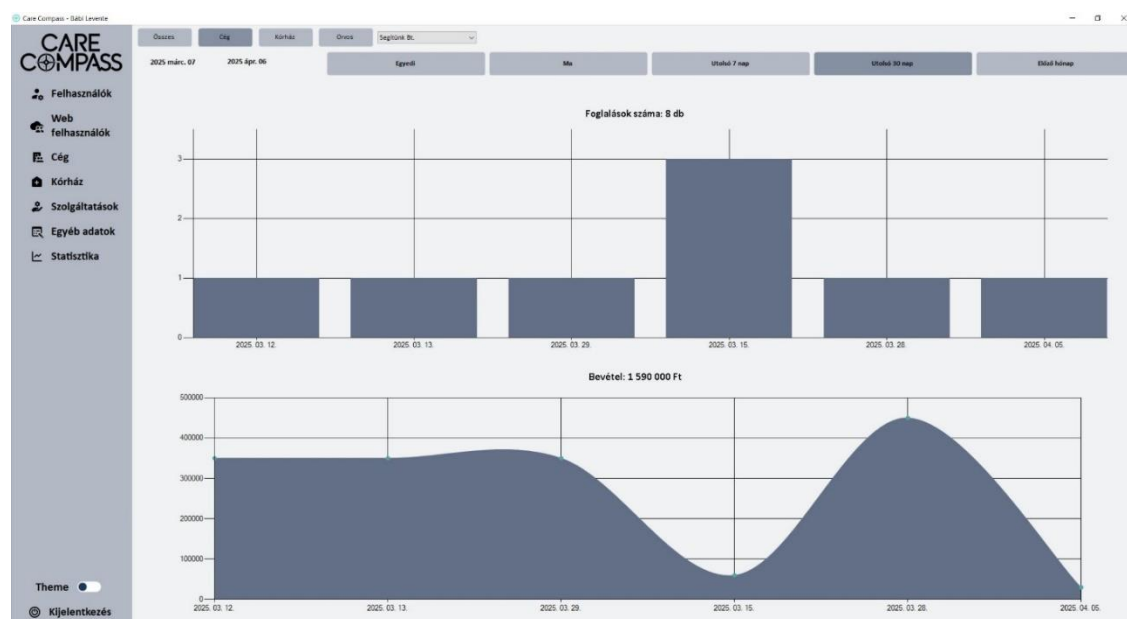
Hozzáadás

Ezen az oldalon tudunk új nyelveket, illetve új végzettségeket felvenni.

- Nyelv hozzáadása
 - Bejelentkezés után egyszerűen a bal oldalon található menü segítségével válasszuk ki a "Egyéb adatok" menüpontot.
 - Itt a "Nyelv" felirattól balra van egy beviteli mező és egy "Hozzáadás" gomb.
 - Egyszerűen csak írjuk be az adott nyelvet, majd nyomjuk meg a "Hozzáadás" gombot az adat felviteléhez.
- Nyelv törlése
 - Az adott nyelv mellett található "Törlés" gombra kell kattintanunk, ezzel törölve az adott nyelvet a rendszerből.
- Végzettség hozzáadása
 - Bejelentkezés után egyszerűen a bal oldalon található menü segítségével válasszuk ki a "Egyéb adatok" menüpontot.

- Itt az "Oktatás" felirattól balra lent van egy beviteli mező és egy "Hozzáadás" gomb.
- Egyszerűen csak írjuk be az adott végzettséget, majd nyomjuk meg a "Hozzáadás" gombot az adat felviteléhez.
- Végzettség törlése
 - Az adott végzettség mellett található "Törlés" gombra kell kattintanunk, ezzel törölve az adott végzettséget a rendszerből.

Statisztika



Ebben a menüpontban az összes statisztika elérhető. Itt a bal oldalon beállíthatunk egyedi szűrést, de lehetőség van alapvetően napi, heti, havi statisztikák kiírására. Olyan statisztikák érhetünk el, mint például a foglalások száma, vagy épp a bevételünk mértéke. Lehetőségünk van még a bal felső sarokban egy legördülő menüben orvost vagy épp kórházat kiválasztani, esetleg céget, így megtekinthető az adott orvos vagy kórház akár cég statisztikája is.

Felhasználók meghívása

Admin felhasználóként lehetőségünk van email alapján új felhasználók meghívására. Ezek a felhasználók kapnak egy email, melynek segítségével regisztrálni tudják profiljukat.

Admin/moderátor felhasználói meghívó



Meghívó

Bábi Levente meghívta, hogy regisztráljon a CareCompass rendszerébe.
A meghívó elfogadásához töltse ki az adatait, majd kattintson a lenti gombra.

Regisztráció

Orvos felhasználó meghívó email, majd a regisztrációs oldal.

Meghívója érkezett

Önt meghívták, hogy regisztráljon a CareCompass adminisztratív rendszerébe.

A meghívó elfogadásához kattintson az alábbi gombra és töltse ki adatait.

Meghívó elfogadása

Amennyiben Ön nem regisztrált, kérjük hagyja figyelmen kívül az emailt.

© CareCompass. Minden jog fenntartva.



Meghívó

Bábi Levente meghívta, hogy regisztráljon a CareCompass rendszerébe.
A meghívó elfogadásához töltse ki az adatait, majd kattintson a lenti gombra.



Regisztráció

6 Konklúzió

A projekt célja egy olyan két részből álló rendszer megvalósítása volt, amely egyszerre biztosít online hozzáférést a magán egészségügyi szolgáltatásokhoz a weboldalon keresztül, illetve lehetőséget nyújt az adminisztrációs adatok hatékony kezelésére a különálló asztali (WinForms) alkalmazás segítségével.

A weboldal lehetővé teszi a páciensek számára, hogy egyszerűen és gyorsan időpontot foglaljanak különböző egészségügyi szolgáltatásokra, kiválaszthassák a számukra megfelelő intézményt és szakorvost. A felület felhasználóbarát kialakítása hozzájárul a kényelmes ügyintézéshez, bármilyen eszközről.

Az asztali alkalmazás célja az volt, hogy a háttérben dolgozó kórház, cég adminisztrátorok és orvosok hatékonyan kezelni tudják a saját adataikat, orvosi beosztásokat és szolgáltatásokat. A program Windows Forms technológiával készült, amely lehetővé tette a gyors fejlesztést és az egyszerű, stabil felhasználói felület kialakítását.

A fejlesztés során tapasztalatot szereztünk a webes és asztali alkalmazások összekapcsolásában, adatkezelésben, valamint felhasználói élmény szempontjából optimalizált felületek tervezésében is. A rendszer alapfunkciói sikeresen működnek, azonban a jövőben lehetőség van további fejlesztésekre, mint például értékelések, részletesebb szűrése, további adatmegjelenítés, blog.

Összességében a projekt elérte a célját: egy jól használható rendszert sikerült létrehozni, amely segíti a magán egészségügyi szolgáltatók és a páciensek közti gördülékeny kapcsolattartást és adminisztrációt.

7 Továbbfejlesztés

7.1 Értékelések

A felhasználók számára lehetőséget biztosítunk arra, hogy értékeljék az általuk igénybe vett szolgáltatásokat és orvosokat. Ez átláthatóságot teremt, és segíti az új pácienseket a döntéshozatalban. Az értékelések alapján a rendszer intelligens ajánlásokat is tud tenni, személyre szabott javaslatokat nyújtva. Az értékelések moderálása automatikusan és manuálisan is történik, hogy hiteles és hasznos visszajelzések maradjanak az oldalon. A pozitív visszajelzések motiválhatják az egészségügyi szolgáltatókat a folyamatos fejlődésre.

7.2 Blog

Egy szakmai blog indítása lehetőséget teremt arra, hogy orvosaink és szakértőink közérthető formában osszanak meg egészségügyi tanácsokat, híreket és aktualitásokat. A blog hozzájárul a páciensek edukálásához és bizalmuk növeléséhez. Emellett keresőoptimalizálási (SEO) szempontból is előnyös, hiszen növeli a weboldal láthatóságát. A tartalmak kategóriákba rendezve jelennek meg, és kulcsszavak alapján is kereshetők lesznek. A látogatók kommentelhetnek, kérdezhetnek, így valódi közösség is épülhet a platform köré.

7.3 Orvos adatlapjának kibővítése

Az orvosok adatlapjai részletesebbé válnak, tartalmazni fognak szakmai önéletrajzot, publikációkat, elismeréseket, és gyakori kérdéseket is. A cél, hogy a páciensek minél több információ birtokában tudjanak dönteni. Az adatlap része lesz továbbá az értékelések összegzése és egy dinamikus elérhetőség-naptár is. A vizuális elemeket is bővítjük, akár rövid bemutatkozó videók is megjelenhetnek. Így a rendszer még emberközelibbé válik.

7.4 További statisztikák

A szolgáltatásunkhoz kapcsolódó statisztikai modul továbbfejlesztése lehetővé teszi az orvosok és rendelők számára, hogy részletes adatokat lássanak például a foglalási arányokról, lemondásokról, népszerű időpontokról és visszatérő páciensekről. Ez segíti az erőforrások hatékonyabb beosztását. A rendszer automatikusan javaslatokat is tesz, például új időpontok megnyitására, ha egy időszak túlterhelt. A páciensek is láthatnak anonim módon statisztikákat, mint például az átlagos várakozási idő. Mindez átláthatóságot és tervezhetőséget biztosít mindkét oldalnak.

7.5 Internacionális fejlesztések

Célunk, hogy a CareCompass rendszert más országokban is elérhetővé tegyük. Ennek érdekében többnyelvű támogatást vezetnénk be, beleértve a platform teljes nyelvi lokalizációját. Az egészségügyi szabályozások országonként eltérnek, így jogi és technikai oldalról is biztosítanánk az alkalmazkodást. A külföldön élő magyarok számára is egyszerűbbé válik a hazai időpontfoglalás. A globális nyitás lehetővé teszi a nemzetközi együttműködéseket, például orvosi másodvélemények kérését.

7.6 Magasabb fokú személyre szabhatóság

A felhasználói élményt egyéni preferenciákhoz igazítjuk. A páciensek beállíthatják, milyen típusú szolgáltatásokat, orvosokat és rendelési időpontokat részesítenek előnyben. A kezdőfelület, értesítések és ajánlások is ezekhez az igényekhez igazodnak majd. Az orvosok és szolgáltatók is testre szabhatják profiljukat és foglalási rendszereiket. A cél, hogy minden felhasználó úgy érezze, a rendszer teljes mértékben neki készült.

7.7 Mobilalkalmazás

7.7.1 Időpontok megtekintése

A CareCompass mobilalkalmazás lehetőséget biztosít arra, hogy a páciensek bárhol és bármikor megtekinthessék aktuális és jövőbeli időpontjaikat. Az alkalmazás egyszerű,

áttekinthető felülettel jeleníti meg a foglалásokat. A felhasználók emlékeztetőt kapnak a közelgő időpontokról, valamint az esetleges változásokról. A naptár integráció révén a foglалások automatikusan bekerülhetnek a telefon naptárába. Így a páciensek biztosak lehetnek abban, hogy nem felejteneк el semmit.

7.7.2 Időpontfoglalás

A mobilalkalmazás lehetővé teszi új időpontok gyors és egyszerű foglalását. A felhasználók szűrhetnek szakterület, orvos, helyszín és időpont alapján. Az alkalmazás figyelembe veszi a felhasználó korábbi választásait, és intelligens ajánlásokat is kínál. A foglalás során egyetlen érintéssel kiválasztható a kívánt időpont, és push értesítésekben visszaigazolást is küldünk. A cél egy gördülékeny, papírmentes és kényelmes időpontfoglalási élmény biztosítása.

8 Irodalomjegyzék

8.1 Applikáció (winform)

Custom button:

Custom textbox: <https://www.youtube.com/watch?v=3wP6QqjiCCo>

Custom PictureBox: <https://www.youtube.com/watch?v=54347QjDisA>

Custom Toggle button: <https://www.youtube.com/watch?v=m7lv6xfjnuw>

8.2 Statisztika

<https://www.youtube.com/watch?v=qHyrJDd4sd8>

<https://www.youtube.com/watch?v=CPQKHGNWJBA>

8.3 Theme changer (Alap): ChatGPT

JSON: <https://stackoverflow.com/questions/59503393/is-using-the-system-text-jsonno-longer-possible-for-serializing-deserialiing>

8.4 Web applikáció

Tab control: https://www.w3schools.com/w3css/w3css_tabulators.asp

Naptár esemény (ICS generálás PHP-val):

<https://stackoverflow.com/questions/12739247/how-to-generate-ics-file-using-php-for-a-given-date-range-and-time>

Script általi kérés észlelése (AJAX felismerés):

<https://www.ozzu.com/snippets/608142/detect-an-ajax-request-with-php>

SQL JSON lekérdezés (JSON_EXTRACT) használat):

<https://www.beekeeperstudio.io/blog/sql-json-extract>

Script kérés ellenőrzés (fetch felismerés PHP-ben):

<https://stackoverflow.com/questions/44202593/detect-a-fetch-request-in-php>

Naptár inspiráció (HTML/CSS/JS alapú dinamikus naptár):

<https://dev.to/wizdomtek/creating-a-dynamic-calendar-using-html-css-and-javascript-29msetting-up-the-project>

Töltőanimáció (Uiverse animáció):

<https://uiverse.io/abrahamcalsin/serious-turkey-52>

Tab control (W3.CSS tabulátorok példával):

https://www.w3schools.com/w3css/w3css_tabulators.asp

8.5 További linkek:

C# dokumentáció: <https://learn.microsoft.com/en-us/dotnet/csharp/>

PHP dokumentáció: <https://www.php.net/docs.php>

JavaScript dokumentáció: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

CSS dokumentáció: <https://developer.mozilla.org/en-US/docs/Web/CSS>

HTML5 dokumentáció: <https://developer.mozilla.org/en-US/docs/Glossary/HTML5>

W3Schools: <https://www.w3schools.com/>

Stack Overflow: <https://stackoverflow.com/>

ChatGPT: <https://chat.openai.com/>

ClaudeAI: <https://claude.ai/>

9. Mellékletek

Github: <https://github.com/Marci13M/Mesterremek>

Api dokumentáció: Api_dokumentáció.pdf

Felhasználók: Felhasználók.pdf